COMP 252
Data Structures and Algorithms
Complete Course Notes

Yitian Zhang
Instructor: Luc Devroye

Winter 2026

Contents

Reading conventions vii

1 Models of Computation and Algorithm Analysis 1

1.1 Analysis of Algorithms and Models of Computation . . . 1

1.1.1 A Short History of Machine Models . . . 1

1.1.2 Time-Complexity Models . . . 2

1.1.3 Other Cost Models . . . 2

1.2 Landau Symbols . . . 3

1.3 Computing the Fibonacci Sequence . . . 4

2 Divide-and-Conquer Algorithms 6

2.1 The Divide-and-Conquer Paradigm and First Examples . . . 6

2.1.1 Fast Exponentiation . . . 6

2.1.2 Chip Testing . . . 6

2.1.3 Integer Multiplication . . . 8

2.1.4 Merge Sort . . . 9

2.1.5 Finding a Convex Hull . . . 9

2.1.6 Binary Search with Ternary/Binary Oracles . . . 10

2.1.7 Matrix Multiplication . . . 11

2.1.8 Euclid's Algorithm . . . 11

2.2 Solving Divide-and-Conquer Recurrences . . . 12

2.2.1 The Master Theorem . . . 12

2.2.2 Induction . . . 14

2.2.3 An Inductive Bound for Merge Sort . . . 14

2.3 Selection . . . 15

2.3.1 The Selection Problem . . . 15

2.3.2 Deterministic Selection: Median of Medians . . . 15

2.3.3 Median of Medians: Completing the Recurrence . . . 17

2.4 Refined Analyses of Earlier Algorithms . . . 18

2.4.1 Convex-Hull Implementation and Complexity . . . 18
2.4.2 Euclid's Algorithm in the Bit Model . . . 20
2.4.3 Binary Search with a Binary Oracle . . . 21

3 Fast Fourier Transform and Convolution **23**
3.1 Polynomial Multiplication and the Cooley–Tukey Idea . . . 23
3.2 The Fourier/Vandermonde Matrix . . . 24
3.3 The Recursive FFT Program . . . 24
3.4 Application: Rotating Two Circles of Bits . . . 25

4 Dynamic Programming **27**
4.1 Example 1: Binomial Numbers . . . 27
4.2 Example 2: Partitioning Numbers . . . 28
4.3 Example 3: Traveling-Salesman Problem . . . 28
4.4 Example 4: Longest Common Subsequence . . . 29
4.5 Example 5: Knapsack/Subset-Sum Problem . . . 31

5 Decision Trees, Adversaries, and Lower Bounds **32**
5.1 Worst-Case Running Time and Lower Bounds . . . 32
5.2 Decision Trees and Lower Bounds . . . 32
5.3 Merging Sorted Lists . . . 36
5.4 Adversary Lower Bounds . . . 36
5.5 Median of $2n + 1$ Elements . . . 40

6 Linear Abstract Data Types and Stack Applications **42**
6.1 Abstract Data Types . . . 42
6.2 Linked Lists . . . 43
6.3 Stacks . . . 44
6.4 Queues . . . 45
6.5 Deques . . . 45
6.6 Application: Matching Parentheses . . . 46
6.7 Stack Applications and Recursion . . . 46
6.7.1 Finding a path through a maze . . . 46

7 Rooted Trees, Traversals, and Succinct Encodings **47**
7.1 Rooted Trees . . . 47
7.1.1 Rooted and ordered trees . . . 47
7.1.2 Complete binary trees . . . 48

7.1.3 Tree representations . . . 48
7.1.4 Basic properties of trees . . . 49
7.2 Tree traversals . . . 49
7.2.1 Iterative traversal and traversal ranks . . . 50
7.2.2 Stack-based preorder . . . 50
7.2.3 Parsimonious stack traversal . . . 51
7.2.4 Preorder using parent pointers . . . 51
7.3 Compression of binary trees . . . 52
7.3.1 Root coding and the information-theoretic lower bound . . . 53
7.3.2 Catalan paths . . . 53
7.3.3 Reflection principle and the Catalan formula . . . 54
7.4 Expression trees and Polish notation . . . 54

8 Dictionary ADT, Balanced Search Trees, and Augmented Structures 56
8.1 The dictionary ADT and search trees . . . 56
8.1.1 Binary search trees . . . 56
8.1.2 BST search and insertion . . . 57
8.1.3 Deletion and successor . . . 58
8.1.4 Completing the k-successor analysis . . . 58
8.2 Red–black trees . . . 58
8.2.1 Red–black invariants and pods . . . 59
8.2.2 Height of a red–black tree . . . 59
8.2.3 Insertion . . . 60
8.2.4 Insertion: final fix . . . 60
8.2.5 Deletion strategies and repair cost . . . 61
8.2.6 Deletion repair . . . 61
8.2.7 Black-sibling cases . . . 62
8.3 Joining and Splitting Red–Black Trees . . . 63
8.3.1 Joining two trees . . . 63
8.3.2 Splitting a red–black tree . . . 64
8.4 Augmented Data Structures . . . 65
8.4.1 Two Red–Black Trees over Shared Data . . . 65
8.4.2 The Order-Statistic ADT . . . 65
8.4.3 Adding an Ordered List to a Tree . . . 67
8.4.4 Lists as Trees . . . 68
8.4.5 Interval Trees . . . 68
8.4.6 Application: Sweep-Line Algorithm for Rectangle Overlap . . . 69

9 Cartesian Trees, Treaps, and Random Binary Search Trees **71**
9.1 Cartesian Trees . . . 71
9.1.1 Construction Rules and Properties . . . 71
9.1.2 Special Cases . . . 71
9.1.3 Treap Operations . . . 72
9.1.4 Join and Split . . . 73
9.1.5 Probabilistic Analysis of Treap Depth . . . 73
9.2 Quicksort and Random BSTs . . . 75
9.2.1 Pivot Improvements . . . 75

10 Priority Queues, Heaps, and Tournament Trees **76**
10.1 The Priority-Queue ADT . . . 76
10.1.1 Binary Heaps . . . 76
10.2 Building and Updating a Heap . . . 77
10.3 HeapSort . . . 79
10.4 Tournament Trees . . . 80

11 Amortized Analysis and Self-Adjusting Search Trees **82**
11.1 Amortized Analysis . . . 82
11.2 Weight-Balanced Search Trees (Scapegoat Trees) . . . 84
11.3 Splay Trees . . . 86

12 String Data Structures and Pattern Matching **90**
12.1 Tries . . . 90
12.2 Patricia Trees . . . 91
12.3 Suffix Tries and Suffix Trees . . . 92
12.4 Pattern Matching When Neither Input Is Preprocessed . . . 94

13 Information Theory, Huffman Coding, and Lempel–Ziv **98**
13.1 Information theory and data compression . . . 98
13.1.1 Lossless and lossy compression . . . 98
13.1.2 Codes viewed as binary trees . . . 98
13.1.3 Optimizing expected codeword length . . . 99
13.2 Huffman Coding . . . 101
13.2.1 Huffman Trees and the Greedy Algorithm . . . 101
13.2.2 Optimal Merging of Sorted Files . . . 102
13.2.3 Generating a Weighted Random Integer . . . 103
13.3 Shannon's Theory . . . 103

13.3.1 Two Facts Used in the Entropy Bound . . . 104
13.3.2 Proof of the Lower Bound . . . 105
13.3.3 Shannon–Fano Code . . . 105
13.4 Lempel–Ziv Parsing . . . 106
13.4.1 Digital Search Tree for the Phrases . . . 107

14 Graph Foundations, Search, and Shortest Paths 108
14.1 Graphs and Their Representations . . . 108
14.1.1 Basic Graph Terminology . . . 108
14.1.2 Directed Acyclic Graphs . . . 109
14.1.3 Graph Representations . . . 110
14.2 Depth-First Search . . . 110
14.2.1 The Algorithm . . . 110
14.2.2 Classification of Edges . . . 112
14.2.3 Cycle Detection . . . 115
14.3 Euler Tours . . . 116
14.4 Breadth-First Search . . . 118
14.5 Weighted Graphs and Shortest Paths . . . 119
14.5.1 Dijkstra's Algorithm . . . 119

15 Fibonacci Heaps 122
15.1 Overview and Structure . . . 122
15.2 Simple Operations and the Potential Function . . . 123
15.3 Decrease-Key, Delete-Min, and Delete . . . 124
15.4 Marked Nodes and Cascading Cuts . . . 126
15.5 The Maximum-Degree Bound . . . 127

16 Spanning Trees and Advanced Directed-Graph Algorithms 129
16.1 Minimum spanning trees . . . 129
16.1.1 Greedy strategies for MSTs . . . 130
16.1.2 A 2-approximation for metric TSP . . . 132
16.2 All-pairs shortest paths: Floyd–Warshall . . . 132
16.3 The transitive-closure problem . . . 133
16.4 Directed acyclic graphs (DAGs) . . . 134
16.4.1 Examples of DAGs . . . 134
16.4.2 Topological sorting / consistent labeling . . . 134
16.4.3 Activity monitors (PERT) . . . 135
16.5 Strongly Connected Components . . . 136

16.5.1 Decomposing every directed graph . . . 136
16.5.2 Kosaraju–Sharir's algorithm . . . 136
16.6 NIM: A Game on a DAG . . . 137

17 Network Flow **139**
17.1 Flow Networks . . . 139
17.2 Residual Networks . . . 140
17.3 Augmenting Paths and Ford–Fulkerson . . . 141
17.4 Cuts and the Max-Flow/Min-Cut Theorem . . . 143
17.4.1 Proof of max-flow/min-cut . . . 143
17.5 Complexity . . . 144

A Worked Practice Examples **145**
A.1 Minimum Spanning Trees: Prim–Dijkstra and Kruskal . . . 145
A.2 Maximum Flow, Residual Network, and Minimum Cut . . . 146
A.3 Augmenting Along a Reverse Residual Edge . . . 147

Reading conventions

Color is semantic: blue records supplementary reasoning, red records warnings or corrections, orange records emphasized derivations, and pink records highlighted examples or conclusions. Crossed-out work is retained when it illustrates an educationally useful false start. Every diagram is redrawn in TikZ from the handwritten original; where a redraw also corrects a slip in the original, a red note says so.

Chapter 1

Models of Computation and Algorithm Analysis

1.1 Analysis of Algorithms and Models of Computation

COMP 252 – Data Structures and Algorithms

Yitian Zhang (261254611) Instructor: Luc Devroye

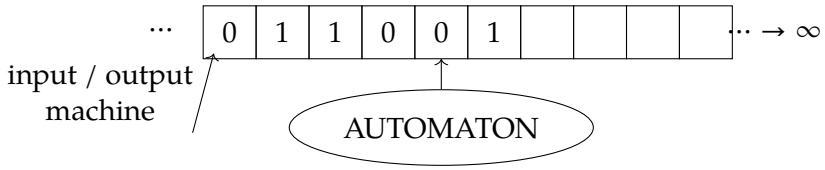
Textbooks. Jeff Erickson, *Algorithms*; and CLRS, *Introduction to Algorithms*, 3rd ed.

1.1.1 A Short History of Machine Models

1940s: Alan Turing and the Turing machine

A Turing machine consists of a tape and an automaton that executes the following unit operations:

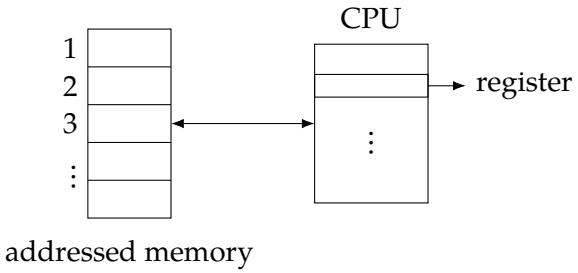
1. read or write the current tape cell;
2. move left;
3. move right.



Related course: COMP 273.

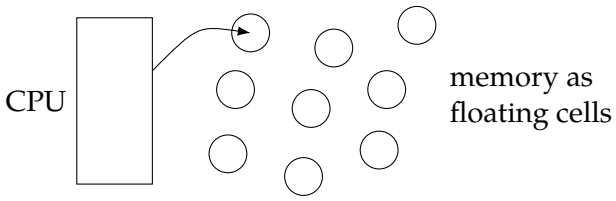
1950s: random-access machines

Memory locations have addresses, and the CPU has registers. In the *RAM* (random-access-memory) model, the CPU can fetch from any location in memory.



1960s: pointer-based machines

Memory is viewed as floating cells reached through pointers rather than through a contiguous addressed array.



1.1.2 Time-Complexity Models

(1) **RAM model:** assume that all operations take $O(1)$ time.
(2) **Bit model (logarithmic model):** reading, writing, moving, and erasing a bit each take $O(1)$ time. For example,

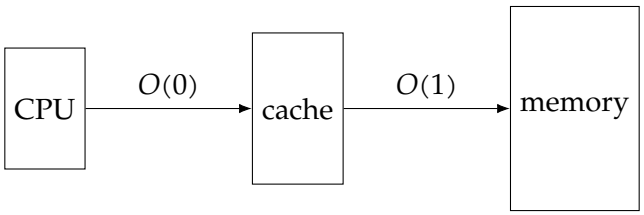
$$\begin{array}{r} 1101 \\ +\ 11 \\ \hline 10000 \end{array} \qquad \rightsquigarrow \qquad 6 + 1 + 2 + 1 + 1 = 11 \text{ units of time.}$$

(3) **Oracles:** "miracle" black boxes whose access time is assumed to be 1.

- **Scale oracle.** A ternary comparison tells whether $x < y$, $x = y$, or $x > y$ in unit time. A binary comparison tells whether $x \leq y$ or $x > y$ in unit time.
- **Function oracle.** An input x passes through an $O(1)$ oracle and returns $f(x)$.

1.1.3 Other Cost Models

Cache. In this cache model, the CPU accesses a nearby cache at cost $O(0)$ and main memory at cost $O(1)$:



Communication complexity. A network edge between two computers is charged one time unit.

1.2 Landau Symbols

Let $\{a_n\}$, $\{b_n\}$, and $\{c_n\}$ be sequences of positive numbers.

Big O and big Ω

$$a_n = O(b_n) \iff \exists C > 0,\, n_0 :\, a_n \le C b_n \quad \forall n \ge n_0,$$

and

$$a_n = \Omega(c_n) \iff \exists C' > 0,\, n_1 :\, a_n \ge C' c_n \quad \forall n \ge n_1.$$

Example

For

$$a_n = 7n^3 - 200n^2 + 50n - 70,$$

we have $a_n \le 7n^3$ for all n, hence $a_n = O(n^3)$. Also

$$a_n \ge 7n^3 - 270n^2 \ge n^3 \qquad \text{if} \qquad n \ge \frac{270}{6} =: n_0,$$

so $a_n = \Omega(n^3)$.

Tight, little, and asymptotic notation

$$\begin{aligned}
a_n = \Theta(b_n) &\iff a_n = O(b_n) \text{ and } a_n = \Omega(b_n),\\
a_n = o(b_n) &\iff \lim_{n\to\infty} \frac{a_n}{b_n} = 0 \qquad (a_n \ll b_n),\\
a_n = \omega(b_n) &\iff \lim_{n\to\infty} \frac{a_n}{b_n} = \infty \qquad (a_n \gg b_n),\\
a_n \sim b_n &\iff \lim_{n\to\infty} \frac{a_n}{b_n} = 1.
\end{aligned}$$

Example

$$n^{1000} = o(1.000001^n), \qquad \log_2(n!) \sim n \log_2 n.$$

For the second relation, recall that $e_n = (1 + 1/n)^n$ increases to e. Thus

$$e^n \ge e_1 e_2 \cdots e_n = \frac{(n+1)^n}{n!} \ge \frac{n^n}{n!}, \qquad \text{so} \qquad n! \ge \frac{n^n}{e^n}.$$

Consequently,

$$1 \ge \frac{\ln(n!)}{\ln(n^n)} \ge \frac{\ln(n^n) - \ln(e^n)}{\ln(n^n)} = 1 - \frac{1}{\ln n} \longrightarrow 1,$$

by the squeeze theorem.

For an algorithm A on n inputs, the notation used is

$$T(x_1, \ldots, x_n; A).$$

Its worst-case running time is

$$T_n(A) = \max_{x_1,\dots,x_n} T(x_1,\dots,x_n;A),$$

while its average-case running time is

$$T_n(A) = \frac{1}{\#\{(x_1,\dots,x_n)\}} \sum_{x_1,\dots,x_n} T(x_1,\dots,x_n;A).$$

Common complexity classes, from smaller to larger, include

$$\Theta(1), \quad \Theta(\log n), \quad \Theta(n), \quad \Theta(n\log n), \quad \Theta(n^2), \quad O(n^k)\ (k\in\mathbb{N}).$$

Examples in the preceding hierarchy include reading one array entry in the RAM model, binary search, and merge sort. Beyond polynomial growth come

$$\Theta(c^n)\ (c>1), \qquad \Theta(n!), \qquad \dots$$

1.3 Computing the Fibonacci Sequence

Let

$$x_0,x_1,x_2,x_3,x_4,x_5,\dots = 0,1,1,2,3,5,8,13,21,\dots$$

and suppose that we want to find x_n.

Closed-form calculation

Consider the closed form

$$x_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{2}.$$

Problem: a computer truncates numerical values. This makes the direct floating-point calculation unsuitable as an exact algorithm.

1. Direct recursion

`FIB(`n`):`
- `if` $n=0$, `output 0;`
- `if` $n=1$, `output 1;`
- `if` $n\geq 2$, `output FIB(`$n-1$`)+FIB(`$n-2$`).`

In the RAM model,

$$T_0 = T_1 = 1, \qquad T_n = 1 + T_{n-1} + T_{n-2}.$$

Therefore

$$T_n + 1 = (1 + T_{n-1}) + (1 + T_{n-2}),$$

which has the same recurrence as $x_n = x_{n-1} + x_{n-2}$. The first terms line up as follows:

	T_0+1	T_1+1	T_2+1	T_3+1	T_4+1
	2	2	4	6	10
$2x_1,\dots,2x_5$	2	2	4	6	10

Thus

$$T_n + 1 = 2x_{n+1}, \qquad T_n = 2x_{n+1} - 1.$$

Since

$$x_n = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right),$$

the recursive algorithm is exponentially slow: BAD.

2. Dynamic programming

Store an array $x[0], x[1], \ldots$ and compute

```
x[0] ← 0;  x[1] ← 1;
for i ← 2 to n do
    x[i] ← x[i − 1] + x[i − 2].
```

This takes $\Theta(n)$ time: BETTER.

3. Fast exponentiation

The Fibonacci recurrence can be written

$$\begin{bmatrix} x_n \\ x_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x_{n-1} \\ x_{n-2} \end{bmatrix} = \cdots = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} x_1 \\ x_0 \end{bmatrix}, \qquad x_1 = 1,\ x_0 = 0.$$

Chapter 2

Divide-and-Conquer Algorithms

How long does it take to compute M^n? Repeated squaring gives

$$M^2 \longrightarrow M^4 \longrightarrow M^8 \longrightarrow \cdots \longrightarrow M^n,$$

with one unit of work at each arrow, for approximately $\log_2 n$ steps. Hence the matrix method above runs in $\Theta(\log_2 n)$ matrix operations: GREAT.

2.1 The Divide-and-Conquer Paradigm and First Examples

Divide and conquer means dividing larger problems into smaller ones.

2.1.1 Fast Exponentiation

To calculate x^n in the RAM model, define

```
P(x,n):
  if n = 0, return 1;
  if n = 1, return x;
  if n ≥ 2 is even, return P(x,n/2)^2;
  if n ≥ 2 is odd, return x P(x,(n-1)/2)^2.
```

Its running time satisfies

$$T_n = \begin{cases} 1, & n = 0 \text{ or } 1, \\ T_{n/2} + 1, & n \geq 2 \text{ even}, \\ T_{\lfloor n/2 \rfloor} + 2, & n \geq 2 \text{ odd}. \end{cases}$$

If $n = 2^k$ for some k (otherwise, round up to the next power of two), then

$$T_n = T_{n/2} + 1 = T_{n/4} + 2 = \cdots = T_{n/2^k} + k = 1 + k = 1 + \log_2 n.$$

2.1.2 Chip Testing

Chips test one another. Assumption: good chips report truthfully; bad chips lie and might say that the other chip is good even when it is bad.

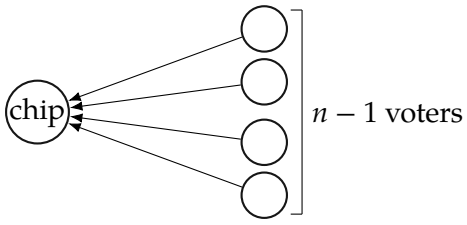
Let G be the set of good chips and B the set of bad chips, with

$$|G| > |B|, \qquad |G| + |B| = n.$$

Problem. Find all good chips.

Model. The cost is the number of times the tester (oracle) is used. Once one good chip is known, it can identify all the other chips.

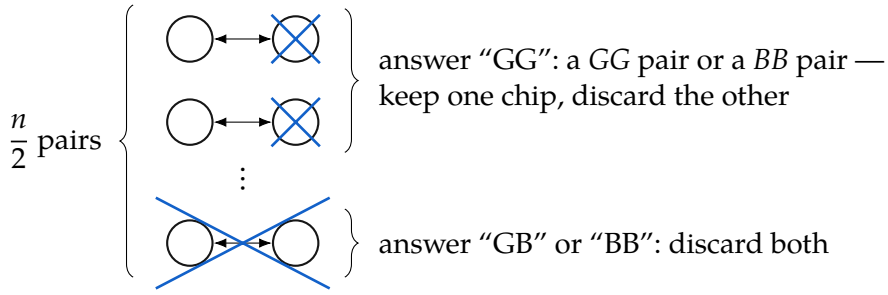
Random selection. Select one chip. Since more than 50% of the chips are good, the selected chip is good with probability greater than $1/2$. Test it by voting: ask the other $n-1$ chips about the selected chip; a strict majority gives the truthful result.



At most $2(n-1)$ voting tests are expected before a good chip is found; that good chip is then used to test all chips. Thus

$$T_n^{\text{avg}} \le 2(n-1).$$

Worst-case $\Theta(n)$ time: a divide-and-conquer alternative. First suppose n is even. Pair the chips randomly and have the two chips in each pair test one another.



For an answer "GG", the pair is either two good chips or two bad chips, so retain one representative. For an answer "GB" or "BB", discard both chips. In the worst such discard, a GB pair is removed; the other possibility is a BB pair. Hence the strict majority of good chips is preserved, since the surviving counts satisfy

$$|G'| = \frac{|G|}{2} > \frac{|B|}{2} + 1 = |B'|,$$

so $|G'| > |B'|$ remains true.

Repeat until one chip remains. In the worst case all pairs answer "GG", so only half are removed, and

$$T_n \le \frac{n}{2} + T_{n/2} \le \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots \le n.$$

If n is odd, pick a chip A and test whether it is good. If $A \in G$, stop; if $A \notin G$, remove it and the remaining number of chips is even. The resulting recurrence is

$$T_n = \frac{3n}{2} + T_{\lfloor n/2 \rfloor} = O(n).$$

2.1.3 Integer Multiplication

Model: the bit model, in which every operation takes one unit. Ordinary long multiplication of two n-bit integers uses $\Theta(n^2)$ bit operations:

$$\begin{array}{r} 101\cdots01 \\ \times\ 11\cdots01 \\ \hline 101\cdots01 \\ 101\cdots01 \\ \vdots \\ \hline \end{array} \qquad \Theta(n^2).$$

2.1.3.1 Karatsuba's Algorithm

Write

$$a = (a_n \cdots a_0), \qquad b = (b_m \cdots b_0),$$

padding the shorter input with zeros so that the two bit strings have the same length.

Split each n-bit input in half:

$$a = \underbrace{(a_n \cdots a_{n/2})}_{\alpha_1}\underbrace{(a_{n/2-1} \cdots a_0)}_{\alpha_2} = \alpha_1 2^{n/2} + \alpha_2,$$

$$b = \underbrace{(b_n \cdots b_{n/2})}_{\beta_1}\underbrace{(b_{n/2-1} \cdots b_0)}_{\beta_2} = \beta_1 2^{n/2} + \beta_2.$$

Then

$$ab = \alpha_1\beta_1 2^n + \alpha_2\beta_2 + (\alpha_1\beta_2 + \alpha_2\beta_1)2^{n/2}.$$

Four half-size multiplications

Computing all four products separately gives

$$T_n \le 4T_{n/2} + c_1 n = O(n^2),$$

so nothing has been simplified asymptotically.

The key identity is

$$\alpha_1\beta_2 + \alpha_2\beta_1 = (\alpha_1 - \alpha_2)(\beta_2 - \beta_1) + \underbrace{\alpha_1\beta_1}_{\text{already calculated}} + \underbrace{\alpha_2\beta_2}_{\text{already calculated}}.$$

Only three half-size multiplications are now needed, so

$$T_n \le 3T_{n/2} + c_2 n = O\left(n^{\log_2 3}\right) \subseteq O(n^{1.58}).$$

Cook–Toom algorithm. Splitting each input into three blocks gives

$$T_n = 5T_{n/3} + cn = O\left(n^{\log_3 5}\right) \subseteq O(n^{1.47}).$$

2007 result

$$T_n = O(n \log n).$$

2.1.4 Merge Sort

Under an oracle model, sort the two halves A and B recursively and then merge them:

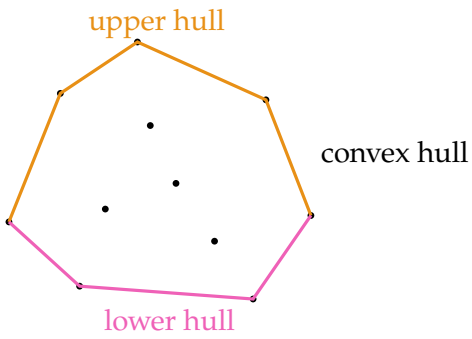
```
sort A
sort B
merge(A, B)
```

At most $n - 1$ comparisons are needed for the merge, hence

$$T_n \leq (n-1) + T_{\lfloor n/2 \rfloor} + T_{\lceil n/2 \rceil} = O(n \log n).$$

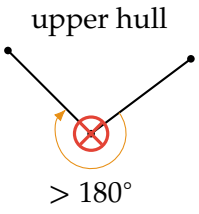
2.1.5 Finding a Convex Hull

First sort the points by x-coordinate using merge sort; then merge them into the hull. The merge is the important part.



Maintain two vertex lists $V_1[0], V_1[1], \ldots$ and $V_2[0], V_2[1], \ldots$, with input indices i, j and output index k.

Convex-hull merge. Start with the leftmost vertex in x; the vertices should already be sorted by x-coordinate. Advance one position in the list. If the angle at the current endpoint is at least 180°, remove the start of the current edge; keep removing until the angle is below 180°. Otherwise, add the end of the edge to the list.



the angle at the middle vertex is $> 180°$:
remove it, and keep removing until it is $< 180°$

Implementation remark: in `/comp 252/FindConvexHull.py`, "start" would be more appropriate at the indicated step.

2.1.6 Binary Search with Ternary/Binary Oracles

Given

$$x_1 < x_2 < \cdots < x_n \in \mathbb{R},$$

binary-search for a query $x \in \mathbb{R}$. Return either the index i for which $x = x_i$, or the interval containing x (including the cases $x < x_1$ and $x > x_n$).

```
BS(x;i,j):
  if j < i, return the interval (x_{i-1}, x_i);
  if j = i, call Ternary(x, x_i):
    if x = x_i, return i;
    if x < x_i, return (x_{i-1}, x_i);
    if x > x_i, return (x_i, x_{i+1});
  if j > i:
    k ← ⌊(i + j)/2⌋; call Ternary(x, x_k);
    if x < x_k, return BS(x;i,k-1);
    if x > x_k, return BS(x;k+1,j);
    otherwise return k.
```

The base costs are $T_0 = 0$ and $T_1 = 1$. For even n, with

$$k = \left\lfloor \frac{n+1}{2} \right\rfloor = \frac{n}{2},$$

the recurrence is

$$T_n = \begin{cases} 1, & x = x_k, \\ T_{n/2} + 1, & x > x_k, \\ T_{n/2-1} + 1, & x < x_k. \end{cases}$$

For odd n,

$$T_n = \begin{cases} 1, & x = x_k, \\ T_{(n-1)/2} + 1, & x \neq x_k, \end{cases} \qquad k = \left\lfloor \frac{n+1}{2} \right\rfloor = \frac{n+1}{2}.$$

Define the monotone envelope

$$M_n := \max_{\text{inputs}} (T_n, T_{n-1}, \ldots, T_0).$$

Then, by substitution,

$$M_n \leq 1 + M_{\lfloor n/2 \rfloor} \leq 1 + M_{\left\lfloor \frac{\lfloor n/2 \rfloor}{2} \right\rfloor}.$$

The substitution uses the fact that for positive integers a, b,

$$\left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor.$$

Thus

$$M_n \leq 2 + M_{\lfloor n/4 \rfloor} \leq \cdots \leq k + M_{\lfloor n/2^k \rfloor} \leq k + 0$$

whenever $2^k > n$. Equivalently $2^k \geq n + 1$, so the least such integer is

$$k = \left\lceil \log_2 (n+1) \right\rceil.$$

2.1.7 Matrix Multiplication

Let $A_n = B_n C_n$. The standard algorithm computes

$$a_{ij} = \sum_{k=1}^{n} b_{ik} c_{kj}$$

in the RAM model, where $+, -, \times$, and $/$ have unit cost. There are n^2 entries, each requiring n multiplications and $n-1$ additions:

$$\underbrace{n^2 \cdot n}_{\text{multiplications}} + \underbrace{(n-1)n^2}_{\text{additions}} = \Theta(n^3).$$

2.1.7.1 Strassen's Multiplication Algorithm

Assume $n = 2^k$; otherwise pad the matrices with zeros, which takes n^2 time. Partition

$$B = \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix}, \qquad C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}.$$

The ordinary block product is

$$BC = \begin{bmatrix} B_1C_1 + B_2C_3 & B_1C_2 + B_2C_4 \\ B_3C_1 + B_4C_3 & B_3C_2 + B_4C_4 \end{bmatrix}.$$

If T_n is the time to multiply two $n \times n$ matrices, this direct blocking gives

$$T_n = 8T_{n/2} + 4n^2 \quad \Longrightarrow \quad T_n = \Theta\left(n^{\log_2 8}\right) = \Theta(n^3),$$

an instance of the "monster theorem" calculation.

Using the same rewriting idea as in Karatsuba integer multiplication, the number of half-size multiplications is reduced from 8 to 7. Therefore

$$T_n = \Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807}).$$

2.1.8 Euclid's Algorithm

Find $\gcd(n, m)$ for $n > m$ by repeatedly replacing (n, m) by $(m, n \bmod m)$:

n	m	$n \bmod m$
83	68	15
68	15	8
15	8	7

The number of lines is at most $2 \log_2 n$: if $m > n/2$, it takes two lines to reduce to at most $n/2$; if $m \le n/2$, it takes one line.

Euclid's example concludes

n	m	$n \bmod m$
8	7	1
7	1	0

Exercise (bit model): show that the complexity is

$$O(\log n \cdot \log m).$$

2.2 Solving Divide-and-Conquer Recurrences

2.2.1 The Master Theorem

For

$$T_n = aT_{n/b} + f(n), \qquad a \geq 1, \quad b > 1, \quad f(n) \geq 0,$$

the Master theorem gives the following three cases.

Master theorem

For some $\varepsilon > 0$:

1. If $n^{\log_b a - \varepsilon} \geq f(n)$, then
$$T_n = \Theta\left(n^{\log_b a}\right).$$
2. If $n^{\log_b a + \varepsilon} \leq f(n)$, then
$$T_n = \Theta(f(n)).$$
3. If $n^{\log_b a} = \Theta(f(n))$, then
$$T_n = \Theta\left(n^{\log_b a} \log n\right).$$

The base of a logarithm does not matter inside a Landau symbol, because

$$\frac{\log_b n}{\log_c n} = \log_b c$$

is constant.

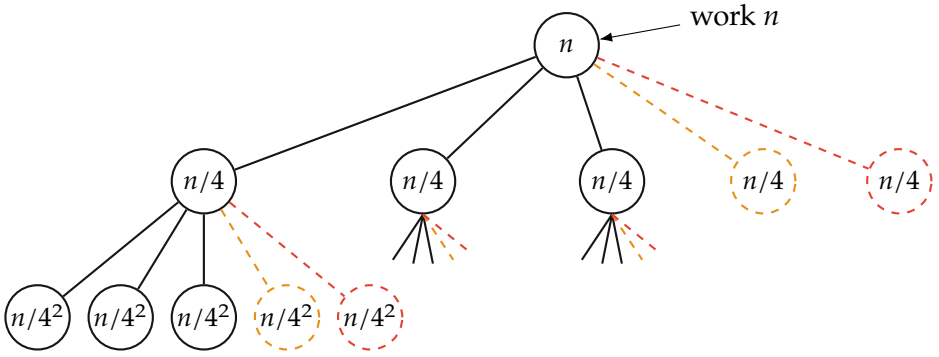
Algorithm	Recurrence	$n^{\log_b a}$ versus $f(n)$	Result
Fast exponentiation	$T_n \leq T_{n/2} + 1$	1 versus 1	$\Theta(\log n)$
Binary search	$T_n \leq T_{n/2} + 1$	1 versus 1	$\Theta(\log n)$
Merge sort / convex hull	$T_n \leq 2T_{n/2} + n$	n versus n	$\Theta(n \log n)$
Karatsuba $*$	$T_n \leq 3T_{n/2} + n$	$n^{\log_2 3}$ versus n	$\Theta(n^{\log_2 3})$
Cook–Toom $*$	$T_n \leq 5T_{n/3} + n$	$n^{\log_3 5}$ versus n	$\Theta(n^{\log_3 5})$
Strassen $\times$	$T_n \leq 7T_{n/2} + n^2$	$n^{\log_2 7}$ versus n^2	$\Theta(n^{\log_2 7})$
Chip testing	$T_n \leq T_{n/2} + \frac{3n}{2}$	1 versus n	$\Theta(n)$

2.2.1.1 Recursion-Tree Justification

Consider

$$T_n = 3T_{n/4} + n,$$

where the additive term is the work at each node. Assuming $n = 4^k$, the tree has $k = \log_4 n$ nontrivial levels. The dashed orange and red children show what changes if the recurrence had $a = 4$ or $a = 5$ children of size $n/4$ instead of three.



dashed orange: a hypothetical 4th child ($a = 4$), $n^{\log_4 4} = n = \Theta(f(n))$: balanced case (case 3)
dashed red: a hypothetical 5th child ($a = 5$), $n^{\log_4 5} > n = f(n)$: recursion-dominated case (case 1)

At level i of the recursion tree there are 3^i nodes, each with work $n/4^i$. Thus:

level	# nodes	total work
0	1	n
1	3	$\frac{3}{4}n$
2	3^2	$\left(\frac{3}{4}\right)^2 n$
$\vdots$	$\vdots$	$\vdots$
k	3^k	$\left(\frac{3}{4}\right)^k n$

In the balanced case (Master-theorem case 3), each level contributes n, so

$$T_n = n \cdot \#\text{levels} = n \log_4 n = \Theta(n \log n).$$

For the recursion-dominated case illustrated with five children of size $n/4$,

$$\begin{aligned} T_n &= n + \frac{5}{4}n + \cdots + \frac{5^k}{4^k}n \\ &= n\frac{5^k}{4^k}\left[\left(\frac{4}{5}\right)^k + \cdots + \frac{4}{5} + 1\right] \\ &\le n\frac{5^k}{4^k}\frac{1}{1-4/5} = \frac{5^k}{1-4/5} \asymp 5^{\log_4 n} = n^{\log_4 5} = n^{\log_b a}. \end{aligned}$$

For an additive-work-dominated case,

$$T_n = n\left(1 + \frac{3}{4} + \left(\frac{3}{4}\right)^2 + \cdots + \left(\frac{3}{4}\right)^k\right) \le n\sum_{i\ge 0}\left(\frac{3}{4}\right)^i = 4n.$$

Remarks

The main part is focused at the first few levels if $f(n)$ dominates. This explains why merge sort / quicksort can be switched to $O(n^2)$ sorting at the base cases. The Master theorem does not apply to every recurrence.

2.2.2 Induction

To prove that a statement holds for every $n \in \mathbb{N}$:

1. verify it for $n \leq n_0$;
2. take $n > n_0$ and assume it is true up to $n - 1$;
3. verify it for n.

2.2.2.1 Binary Search with a Ternary Oracle

We know from the substitution calculation that

$$T_0 = 0, \qquad T_1 = 1, \qquad T_n \leq 1 + T_{\lfloor n/2 \rfloor},$$

and claim

$$T_n \leq \left\lceil \log_2(n+1) \right\rceil.$$

The case $n = 1$ is immediate. Under the induction hypothesis,

$$T_n \leq 1 + T_{\lfloor n/2 \rfloor} \leq 1 + \left\lceil \log_2\left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \right\rceil.$$

If n is even, this becomes

$$\begin{aligned} T_n &\leq 1 + \left\lceil \log_2\left(\frac{n+2}{2} \right) \right\rceil \\ &= 1 + \left\lceil \log_2(n+2) - \log_2 2 \right\rceil \\ &= \left\lceil \log_2(n+2) \right\rceil = \left\lceil \log_2(n+1) \right\rceil, \end{aligned}$$

where the last equality uses that n is even, hence $n + 1$ is odd.

If n is odd, the binary-search induction continues as

$$\begin{aligned} T_n &\leq 1 + \left\lceil \log_2\left(\frac{n-1}{2} + 1 \right) \right\rceil \\ &= 1 + \left\lceil \log_2\left(\frac{n+1}{2} \right) \right\rceil \\ &= 1 + \left\lceil \log_2(n+1) \right\rceil - \log_2 2 \\ &= \left\lceil \log_2(n+1) \right\rceil. \end{aligned}$$

2.2.3 An Inductive Bound for Merge Sort

Work in the ternary/binary-oracle model. The base values are

$$T_0 = T_1 = 0,$$

and the known recurrence is

$$T_n \leq T_{\lfloor n/2 \rfloor} + T_{\lceil n/2 \rceil} + n - 1.$$

Use the induction hypothesis

$$T_n \leq Cn \log_2 n.$$

The base cases are true. For $n > 1$, both subproblem sizes are smaller than n, so

$$T_n \le C\left\lfloor \frac{n}{2} \right\rfloor \log_2 \left\lfloor \frac{n}{2} \right\rfloor + C\left\lceil \frac{n}{2} \right\rceil \log_2 \left\lceil \frac{n}{2} \right\rceil + n - 1.$$

If n is even,

$$\begin{aligned} T_n &\le Cn \log_2 \frac{n}{2} + n - 1 \\ &= Cn(\log_2 n - 1) + n - 1 \\ &= Cn \log_2 n + (1 - C)n - 1 \\ &\le Cn \log_2 n \qquad \text{if } C \ge 1. \end{aligned}$$

If n is odd,

$$\begin{aligned} T_n &\le C\frac{n-1}{2} \log_2 \frac{n-1}{2} + C\frac{n+1}{2} \log_2 \frac{n+1}{2} + n - 1 \\ &= \frac{Cn}{2} \log_2 \frac{n^2-1}{4} + \frac{C}{2} \log_2 \frac{n+1}{n-1} + n - 1 \\ &\le \frac{Cn}{2} \log_2 \frac{n^2}{4} + \frac{C}{2} + n - 1 \\ &= Cn \log_2 n + n(1 - C) + \left(\frac{C}{2} - 1 \right) \\ &\le Cn \log_2 n \end{aligned}$$

provided $C \ge 1$ and $C \le 2$. Thus $1 \le C \le 2$, and the strongest choice is $C = 1$.

Binary search with a binary oracle

Show

$$T_n \le 2 + \left\lceil \log_2 n \right\rceil.$$

2.3 Selection

2.3.1 The Selection Problem

Given: distinct values

$$\{x_1, \dots, x_n\} \subseteq \mathbb{Z} \text{ or } \mathbb{R}, \qquad k \in \mathbb{Z},$$

under the comparison-oracle model.

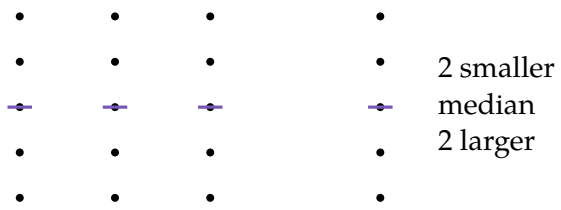
Problem: find the kth smallest element.

2.3.2 Deterministic Selection: Median of Medians

Let $\textsc{Select}(k, S)$ return the kth smallest member of $S \subseteq \{x_1, \dots, x_n\}$.

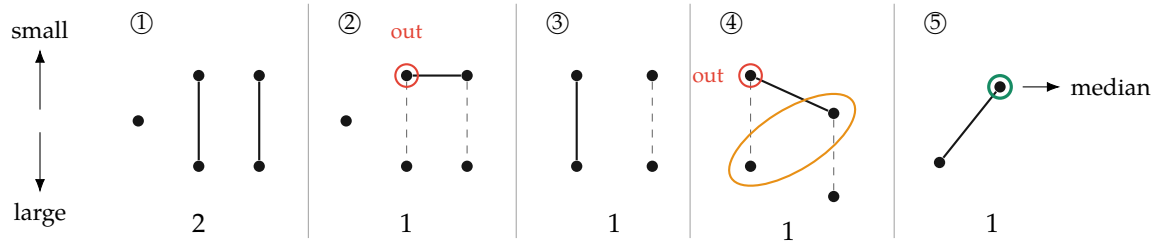
Step 0. If $|S| \le 5$, sort S and return its kth element.

Step 1. Partition S into groups of five. Let M be the set of medians of those groups. A median of five has two smaller and two larger group members and can be found with six comparisons, hence constant time per group.



one median per group

The detailed six-comparison construction has the following comparison topology. Small elements are drawn above large ones; a solid segment is the comparison made in the current step, and a dashed segment is an order that is already known.



$2 + 1 + 1 + 1 + 1 = 6$ comparisons

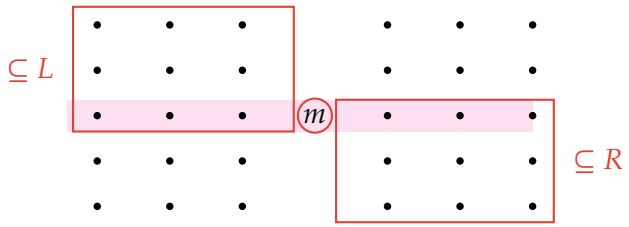
In step ② the smaller of the two minima lies below three elements, so it cannot be the median and is discarded. Its partner is then compared with the untouched element (step ③). In step ④ the smaller of the two minima is again below three of the remaining elements, which leaves the two candidates circled in orange; the smaller of those two is the median (step ⑤).

Step 2. Compute

$$m \leftarrow \textsc{Select}(|M|/2, M),$$

the median of the medians.

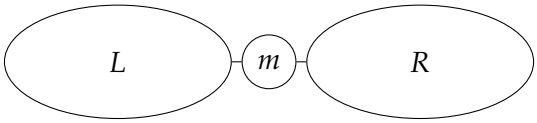
After this step, arrange the list so that group medians $\leq m$ are on the left and group medians $\geq m$ are on the right. The medians are separated, not fully sorted.



Graph 1

Step 3. Form, using comparisons,

$$L \leftarrow \{x \in S : x < m\}, \qquad R \leftarrow \{x \in S : x > m\}.$$



Step 4. Return according to

$$\begin{cases} m, & k = |L| + 1, \\ \textsc{Select}(k, L), & k \leq |L|, \\ \textsc{Select}(k - |L| - 1, R), & k > |L| + 1. \end{cases}$$

Analysis

From Graph 1, in Step 3,

$$|R|, |L| \geq \frac{3}{5} \cdot 50\% \, n = \frac{3}{10} n,$$

and therefore

$$|R|, |L| \leq \frac{7}{10} n,$$

because the complement is occupied. Let

$$T_n = \max\{\text{number of comparisons on an input of size } n\}.$$

2.3.3 Median of Medians: Completing the Recurrence

For groups of five, the running time satisfies

$$T(n) \leq \begin{cases} 7, & n \leq 5, \\ \frac{6n}{5} + T\left(\frac{n}{5}\right) + (n-1) + T\left(\frac{7n}{10}\right), & n > 5. \end{cases}$$

The four terms in the second line are marked as steps ①–④. The last recursive call is the worst case, when either side of the partition has size $7n/10$.

Step ③ can be improved by keeping track of the elements already compared in step ①: its cost becomes $\frac{4n}{10} - 1$. The elements already placed in L and R need not be compared with the median m again, since they are already known to be below or above m.

Thus

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \frac{8n}{5} \leq Cn.$$

Induction claim. $T(n) \leq Cn$.

For $1 \leq n \leq 5$,

$$T(n) \leq 7 \leq Cn,$$

provided $C \geq 7$. For $n > 5$, the induction hypothesis gives

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \frac{8n}{5} \\ &\leq C\left(\frac{n}{5} + \frac{7n}{10}\right) + \frac{8n}{5} \\ &= \left(\frac{9}{10}C + \frac{8}{5}\right) n \leq Cn \end{aligned}$$

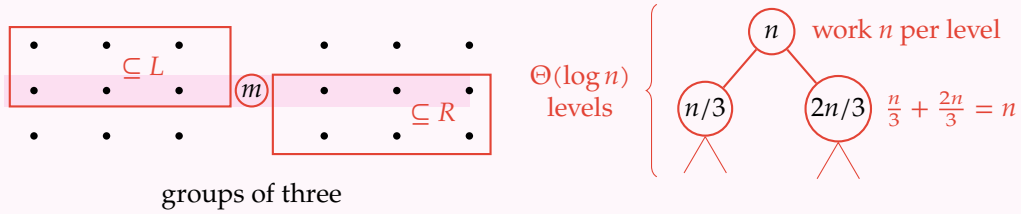
when $C \geq 16$.

If the groups had size three

Partitioning into groups of three instead gives

$$T(n) \leq n + T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) = \Theta(n \log_2 n) \qquad \text{(to show)}.$$

The recursion tree has n work at each level, split into $n/3$ and $2n/3$, over $\Theta(\log n)$ levels.



For the size-three median grouping, the total work is $n \log_2 n$.

2.4 Refined Analyses of Earlier Algorithms

2.4.1 Convex-Hull Implementation and Complexity

Complexity annotation

The sorting phase takes $O(n \log n)$. The merging phase initially appears to use two nested loops and hence to take $O(n^2)$. However, a vertex is either added in constant time or participates in checks before being pushed and popped only once. The cost breakdown is $2cn$ checking time for removed vertices, $2n$ worst-case add/remove time, and cn for the first check on each vertex:

$$T_n = O(n \log n) + 2cn + 2n + cn = O(n \log n) + (3c + 2)n = O(n \log n).$$

The collinearity portion below is marked ×. Its inconsistent list names are intentionally preserved so that the flawed approach remains visible for study.

```
import math

def findconvexhull(coords):
    assert type(coords) == dict, \
        "findconvexhull requires a dict"  # input validation
    lst = []
    for k in coords.keys():
        lst.append(k)
    if lst == [] or len(lst) == 1:
        return coords
    if not (type(lst[0]) == float or type(lst[0]) == int):
        raise TypeError
    lst = sorted(lst)
    lst_copy = lst[:]

    upperlist = []
    while len(lst) > 0:
        if len(upperlist) <= 1:
            upperlist.append(lst[0])
            lst.remove(lst[0])
            continue
        v1_cross_v2 = 0
        while v1_cross_v2 <= 0:
```

```
            next = float(lst[0])
            curr = upperlist[len(upperlist)-1]
            currprev = upperlist[len(upperlist)-2]
            v1 = [next-curr, coords[next]-coords[curr]]
            v2 = [-currprev+curr,
                  -coords[currprev]+coords[curr]]
            v1_cross_v2 = v1[0]*v2[1] - v1[1]*v2[0]
            # calculate the z-coordinate of v1 x v2
            # print(next, curr, v1, v2, v1_cross_v2)
            if v1_cross_v2 > 0:  # convex orientation
                upperlist.append(next)
                lst.remove(next)
                break
            elif len(upperlist) == 2:
                upperlist.remove(curr)
                upperlist.append(next)
                lst.remove(next)
                break
            elif v1_cross_v2 == 0:  # collinearity
                upperlist.remove(curr)
                upperlist.append(next)
                lst.remove(next)
                break
            else:
                # the previous vertex may also need removal;
                # continue in the while loop to check
                upperlist.remove(curr)

    lowerlist = []
    while len(lst_copy) > 0:
        if len(lowerlist) <= 1:
            lowerlist.append(lst_copy[0])
            lst_copy.remove(lst_copy[0])
            continue
        v1_cross_v2 = 0
        while v1_cross_v2 <= 0:
            next = float(lst_copy[0])
            curr = lowerlist[len(lowerlist)-1]
            currprev = lowerlist[len(lowerlist)-2]
            v1 = [next-curr, coords[next]-coords[curr]]
            v2 = [-currprev+curr,
                  -coords[currprev]+coords[curr]]
            v1_cross_v2 = v1[0]*v2[1] - v1[1]*v2[0]
            # print(next, curr, v1, v2, v1_cross_v2)
            if v1_cross_v2 < 0:
                lowerlist.append(next)
                lst_copy.remove(next)
                break
            elif len(lowerlist) == 2:
                lowerlist.remove(curr)
                lowerlist.append(next)
                lst_copy.remove(next)
                break
            elif v1_cross_v2 == 0:  # collinearity
                upperlist.remove(curr)
                upperlist.append(next)
```

```
                lst.remove(next)
                break
            else:
                lowerlist.remove(curr)

    upperlist.reverse()
    upperlist.remove(upperlist[0])
    upperlist.remove(upperlist[len(upperlist)-1])

    returndict = {}
    for k in lowerlist:
        returndict[k] = coords[k]
    for k in upperlist:
        returndict[k] = coords[k]

    return returndict
```

2.4.2 Euclid's Algorithm in the Bit Model

2.4.2.1 First Attempt

Analyze the running time of $\gcd(n, m)$ in the bit model. To divide n by m, let

$$k = \log_2 n, \qquad \ell = \log_2 m,$$

and write the inputs in binary as

$$n = (a_k a_{k-1} \cdots a_1 a_0)_2, \qquad m = (b_\ell b_{\ell-1} \cdots b_1 b_0)_2.$$

2.4.2.2 Cost of Binary Long Division

In long division, the $(\ell + 1)$-bit divisor is shifted along the $(k + 1)$-bit dividend. Reading the relevant dividend block costs ℓ, and a subtraction costs $\ell + 1$; this occurs at most k times. Therefore one remainder computation has the initial bound

$$T_{\text{mod}} \le k(2\ell + 1) = O(k\ell) = O(\log_2 n \, \log_2 m).$$

Trying to insert this bound directly into the Euclidean recursion gives a recurrence of the form

$$T(n) = O(\log_2 n \, \log_2 m) + T(n/2),$$

which the stated version of the Master Theorem does not cover. Substitution (or a different Master Theorem) gives only the loose bound

$$T(n) = O\left(\log_2^2 n \, \log_2 m\right).$$

Correction to the first attempt

The problem is that the bound on a remainder operation is not tight enough. Write all of the

Euclidean divisions as

$$\begin{aligned} n &= q_1 m + r_1, \\ m &= q_2 r_1 + r_2, \\ r_1 &= q_3 r_2 + r_3, \\ &\vdots \\ r_{N-2} &= q_N r_{N-1} + 0, \end{aligned}$$

and suppose there are N divisions in total.

Two key observations are needed.

(**a**) The ith remainder operation has the tighter bound

$$T_i(\text{mod}) = O\big(\log_2 m \, \log_2 q_i\big) .$$

In binary long division, every block writes one bit of the quotient. Hence there are at most $\log_2 q_i$ blocks (the number of binary bits of q_i), and

$$T_i(\text{mod}) \le (2\ell + 1) \log_2 q_i = O\big(\log_2 m \, \log_2 q_i\big) .$$

2.4.2.3 Completing the Bound

(**b**) The quotients satisfy

$$q_1 q_2 \cdots q_N \le n.$$

This follows by back-substitution:

$$\begin{aligned} n &= q_1 m + r_1 \\ &= q_1 (q_2 r_1 + r_2) + r_1 \\ &= q_1 (q_2 (q_3 r_2 + r_3) + r_2) + r_1 \\ &= \cdots = q_1 q_2 \cdots q_N + \text{positive constant integers.} \end{aligned}$$

Consequently,

$$\begin{aligned} T(n) &\le \sum_{1 \le i \le N} T_i(\text{mod}) \\ &\le \sum_{1 \le i \le N} (2\ell + 1) \log_2 q_i \\ &= (2\ell + 1) \log_2 \left(\prod_{1 \le i \le N} q_i \right) \\ &\le (2\ell + 1) \log_2 n = O(\log_2 m \, \log_2 n). \end{aligned}$$

2.4.3 Binary Search with a Binary Oracle

Theorem

For every $n \ge 1$, binary search with a binary oracle satisfies

$$T_n \le 2 + \lceil \log_2 n \rceil.$$

The base case is

$$n = 1: \qquad T_1 = 2 \le 2 + \lceil \log_2 1 \rceil.$$

Assume the claim holds for every $n < n_0$. The procedure below does not depend on how the endpoints i, j themselves are compared:

BS$(x; i, j)$

```
if i > j:
  return x ∈ (x_{i−1}, x_i)
if i = j:
  Binary(x, x_i)
  if x > x_i: return x ∈ (x_i, x_{i+1})
  Binary(x_i, x)
  if x_i ≤ x: return i
  if x_i > x: return x ∈ (x_{i−1}, x_i)
if j > i:
  k ← ⌊(i + j)/2⌋
  Binary(x, x_k)
  if x > x_k: return BS(x; k + 1, j)
  if x ≤ x_k: return BS(x; i, k)
```

The recorded recurrences are

$$T_n = \begin{cases} 2, & i = j, \\ 1 + T_{n/2}, & n \text{ even}, \end{cases}$$

and, for odd n,

$$T_n = \begin{cases} 2, & i = j, \\ 1 + T_{(n-1)/2}, & x > x_k, \\ 1 + T_{(n+1)/2}, & x \le x_k. \end{cases}$$

Binary-search induction step

Suppose $n_0 > 1$. For even n,

$$\begin{aligned} T_{n,\text{even}} &\le 1 + T_{n/2} \\ &\le 1 + 2 + \left\lceil \log_2 \frac{n}{2} \right\rceil \\ &= 1 + 2 - 1 + \lceil \log_2 n \rceil = 2 + \lceil \log_2 n \rceil. \end{aligned}$$

For odd n, the larger subproblem has size $(n + 1)/2$, so

$$\begin{aligned} T_{n,\text{odd}} &\le 1 + T_{(n+1)/2} \\ &\le 1 + 2 + \left\lceil \log_2 \frac{n+1}{2} \right\rceil \\ &= 1 + 2 - 1 + \lceil \log_2 (n+1) \rceil \\ &= 2 + \lceil \log_2 n \rceil, \end{aligned}$$

where the last equality uses the fact that n is odd. Hence, by induction,

$$T_n \le 2 + \lceil \log_2 n \rceil$$

for binary search with a binary oracle.

Chapter 3

Fast Fourier Transform and Convolution

3.1 Polynomial Multiplication and the Cooley–Tukey Idea

Given

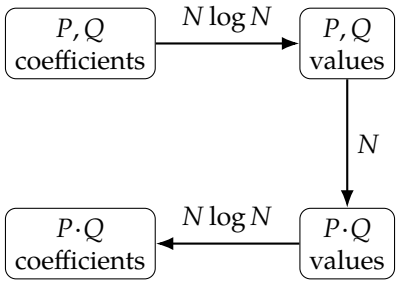
$$P(x) = \sum_{i=0}^{N-1} p_i x^i, \qquad Q(x) = \sum_{i=0}^{N-1} q_i x^i,$$

the goal is to compute $P(x)Q(x)$. Directly calculating every coefficient takes $O(N^2)$ time; the target is $O(N \log N)$.

A polynomial may be represented in two equivalent ways:

1. by coefficients, $P(x) = \sum_i a_i x^i$;
2. by values $P(a_0), \dots, P(a_{N-1})$ at distinct points $a_0, \dots, a_{N-1}$ (Lagrange interpolation).

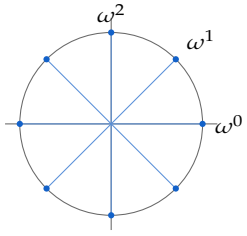
The FFT converts between these representations in $O(N \log N)$ time for a special choice of evaluation points.



Recall

$$\omega = e^{i2\pi/N}, \qquad N \in \mathbb{N}.$$

Then ω is an Nth root of unity, and $\omega^0, \omega^1, \dots, \omega^{N-1}$ are all N roots of $x^N = 1$.



3.2 The Fourier/Vandermonde Matrix

Wanted: coefficients ⟶ values in $N \log N$ time. Evaluate a degree-$(N-1)$ polynomial at the powers of ω:

$$\begin{bmatrix} P(\omega^0) \\ P(\omega^1) \\ \vdots \\ P(\omega^{N-1}) \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & \omega & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \cdots & \omega^{(N-1)^2} \end{bmatrix}}_{V,\ V_{jk}=\omega^{jk}} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{N-1} \end{bmatrix}, \qquad 0 \le j,k \le N-1.$$

Values ⟶ coefficients in $N \log N$ time.

$$\begin{bmatrix} p_0 \\ \vdots \\ p_{N-1} \end{bmatrix} = V^{-1} \begin{bmatrix} P(\omega^0) \\ \vdots \\ P(\omega^{N-1}) \end{bmatrix}.$$

Claim

$$(V^{-1})_{jk} = \frac{\omega^{-jk}}{N}, \qquad 0 \le j,k \le N-1.$$

Indeed,

$$\begin{aligned} (VV^{-1})_{jk} &= \sum_{\ell=0}^{N-1} V_{j\ell}(V^{-1})_{\ell k} \\ &= \frac{1}{N}\sum_{\ell=0}^{N-1} \omega^{j\ell}\omega^{-\ell k} = \frac{1}{N}\sum_{\ell=0}^{N-1} \omega^{\ell(j-k)} \\ &= \begin{cases} 1, & j = k, \\ \dfrac{1}{N}\left[1 + \omega^{j-k} + \cdots + (\omega^{j-k})^{N-1}\right], & j \ne k \end{cases} \\ &= \begin{cases} 1, & j = k, \\ \dfrac{1}{N}\dfrac{(\omega^{j-k})^N - 1}{\omega^{j-k} - 1} = 0, & j \ne k. \end{cases} \end{aligned}$$

Thus $VV^{-1} = I$.

Only one program is needed for one direction; the other direction follows by replacing ω with ω^{-1} (together with the $1/N$ normalization displayed above).

Programming exercise: write a program that performs the coefficient-to-value conversion.

3.3 The Recursive FFT Program

To convert values back to coefficients, use the same program with ω replaced by ω^{-1}, which gives the inverse transform above.

Assume $N = 2^k$; if not, pad the input. The input is N coefficients, and the output is the N values at $\omega^0, \ldots, \omega^{N-1}$.

Split the polynomial into its even- and odd-degree terms:

$$P(x) = P'(x) + xP''(x),$$

where

$$P'(x) = p_0 + p_2x^2 + \cdots + p_{N-2}x^{N-2},$$
$$P''(x) = p_1 + p_3x^2 + \cdots + p_{N-1}x^{N-2}.$$

Compress the exponents by defining

$$Q'(x) := p_0 + p_2x + \cdots + p_{N-2}x^{N/2-1},$$
$$Q''(x) := p_1 + p_3x + \cdots + p_{N-1}x^{N/2-1}.$$

Evaluating P at the N points $\omega^0, \omega^1, \ldots, \omega^{N-1}$ is therefore reduced to the following:

1. Recursively evaluate Q' and Q'' at $N/2$ points
$$\psi = \omega^2 : \qquad \psi^0, \psi^1, \ldots, \psi^{N/2-1}.$$
2. Transcribe those values into values of P' and P'' at powers of ω.
3. Use $P(x) = P'(x) + xP''(x)$ to calculate the values of P.

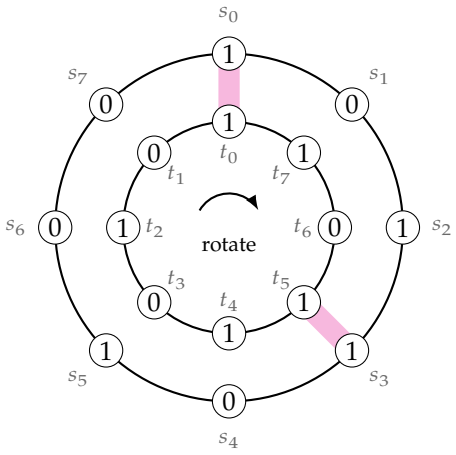
Both the transcription and the final calculation take $O(N)$ time. Hence

$$T(N) = 2T(N/2) + O(N) = \Theta(N \log N).$$

3.4 Application: Rotating Two Circles of Bits

Problem. Two circles each contain n bits. How should the inner circle be rotated to maximize the number of matching 1s between the circles?

The figure shows the setting for $n = 8$. The outer circle carries the bits $s_0, \ldots, s_{n-1}$, the inner circle carries $t_0, \ldots, t_{m-1}$ (here $m = n$), and the inner circle is rotated against the outer one. Two aligned bits form a match exactly when both are 1, that is, when the product $t_j s_k$ equals 1; in the position drawn there are two matches.



outer circle: $(s_0, \ldots, s_{n-1}) \in \{0,1\}^n$,
indices increasing clockwise

inner circle: $(t_0, \ldots, t_{m-1}) \in \{0,1\}^m$,
indices increasing counterclockwise

pink bars: aligned pairs $(1,1)$, i.e. products $t_j s_k = 1$;
the score of this position is 2

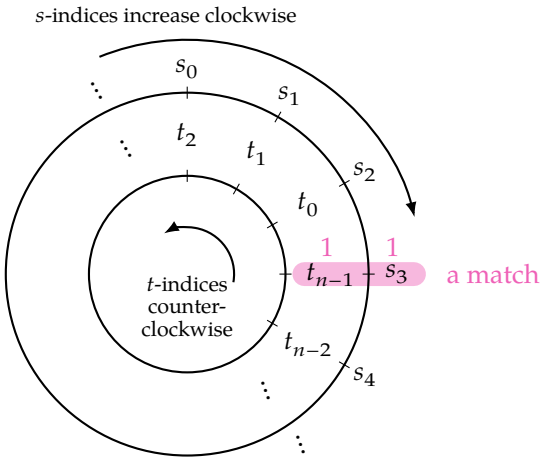
goal: rotate the inner circle to maximize the score

Write S for the outer sequence followed by a copy of itself, $(s_0, \ldots, s_{n-1}, s_n, \ldots, s_{2n-1})$ with $s_{n+k} = s_k$, and T for the inner sequence. The number of matches at one alignment is then one coefficient of the polynomial product ST:

$$[ST]_{m+i} = t_{m-1}s_{i+1} + t_{m-2}s_{i+2} + t_{m-3}s_{i+3} + \cdots + t_0 s_{m+i}.$$

The solution below carries out this computation with $m = n$.

Circle-matching solution



opposite directions: the s-indices increase clockwise, the t-indices counterclockwise

position $i = 2$: t_0 sits on s_2, and in general t_j sits on s_{i-j} (indices mod n)

a match: t_{n-1} and s_3 are aligned and both bits are 1, so $t_{n-1}s_3 = 1$

Taking a product detects a match: 0 means no match and 1 means a match. Checking every alignment directly uses $n \times n = n^2$ products.

For the faster method, duplicate the outer sequence and write

$$\begin{aligned} S(x) &= s_0 + s_1 x + s_2 x^2 + \cdots + s_{n-1} x^{n-1} \\ &\quad + \underbrace{s_n}_{s_0} x^n + \underbrace{s_{n+1}}_{s_1} x^{n+1} + \cdots + \underbrace{s_{2n-1}}_{s_{n-1}} x^{2n-1}, \\ P(x) &= t_0 + t_1 x + t_2 x^2 + \cdots + t_{n-1} x^{n-1}. \end{aligned}$$

Use the FFT to multiply S and P in $O(n \log n)$ time. For $0 \le i \le n-1$, inspect the coefficient of x^{n+i}:

$$\begin{aligned} [x^{n+i}]\, S(x)P(x) &= t_0 s_{n+i} + t_1 s_{n+i-1} + \cdots + t_{n-1} s_{i+1} \\ &= t_0 s_i + t_1 s_{i-1} + \cdots + t_{n-1} s_{i+1}, \end{aligned}$$

where the s-indices are interpreted cyclically. This is exactly the number of matches when t_0 is aligned with $s_{n+i} = s_i$. Looking at these n coefficients therefore gives the best rotation.

Chapter 4

Dynamic Programming

Dynamic programming solves all subproblems and records their answers in a table, so that no subproblem is solved twice.

4.1 Example 1: Binomial Numbers

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \qquad \text{RAM time } O(\min\{k, n-k\}) \quad \text{(direct computation)}.$$

The dynamic-programming table is Pascal's triangle:

$n\backslash k$	0	1	2	3	4
0	1				
1	1	1			
2	1	2	1		
3	1	3	3	1	
4	1	4	6	4	1

Fill the table one subproblem at a time in increasing rows. Pascal's triangle.

The recurrence is

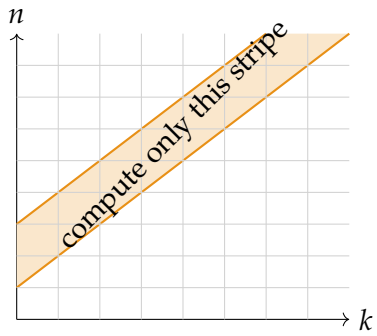
$$\binom{n}{k} = \underbrace{\binom{n-1}{k-1}}_{\text{choosing the } n\text{th element}} + \underbrace{\binom{n-1}{k}}_{\text{not choosing the } n\text{th element}}.$$

Exercise: binomial-coefficient dynamic programming

The standard dynamic-programming algorithm for a binomial coefficient runs in $O(nk)$. Refine it so that it runs in

$$O((n-k)k).$$

Hint: compute only the diagonal stripe of the table needed to obtain $\binom{n}{k}$.



4.2 Example 2: Partitioning Numbers

These numbers are related to cycle structures and conjugacy in S_n. Let

$$\left\{ n \atop k \right\}$$

denote the number of ways to partition n elements into k parts. The boundary values are

$$\left\{ n \atop 1 \right\} = \left\{ n \atop n \right\} = 1,$$

and the recurrence is

$$\left\{ n \atop k \right\} = \left\{ n-1 \atop k-1 \right\} + k \left\{ n-1 \atop k \right\}.$$

The table begins

$n\backslash k$	1	2	3	4	5
1	1				
2	1	1			
3	1	3	1		
4	1	7	6	1	
5	1	15	25	10	1

4.3 Example 3: Traveling-Salesman Problem

Bellman–Held–Karp algorithm. There are n cities, with

$$D[i,j] = d(i,j) \geq 0.$$

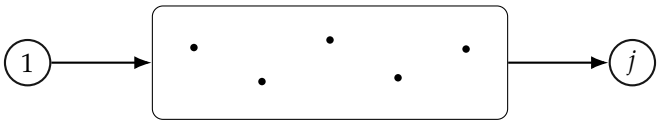
The triangle inequality is not required. The problem is to find a least-cost tour. In the symmetric case, the number of tours is $(n-1)!/2$, while the dynamic program satisfies the target bound $T_n \leq n^2 2^n$.

For

$$S \subseteq \{2, \ldots, n\} \setminus \{j\},$$

define

$$L(1,S,j) := \text{the length of a path that starts at 1, visits } S, \text{ and ends at } j.$$



$$S \subseteq \{2, \dots, n\} \setminus \{j\}$$

Filling the table for all possible S gives at most 2^{n-2} possibilities. For each of the $n-1$ possible endpoints j, this stores

$$2^{n-2}(n-1)$$

results.

Bellman–Held–Karp recurrence

The initial condition is

$$L(1, \emptyset, j) = D[1, j].$$

Table-filling order

```
for |S| = 1 to n − 2, with
S ⊆ {2, ..., n} \ {j}:
  for every j = 2 to n:
    for every set S with j ∉ S:
      L(1, S, j) ← min_{ℓ∈S}{L(1, S \ {ℓ}, ℓ) + D[ℓ, j]}
```

The previously stored entry is $L(1, S \setminus \{\ell\}, \ell)$; appending the last edge contributes $D[\ell, j]$.

The time accounting is

$$\underbrace{2^{n-2}}_{\text{subsets } S} \cdot \underbrace{(n-1)}_{\text{values of } j} \cdot \underbrace{n}_{\text{comparisons for a minimum}} = O(n^2 2^n).$$

Finally, close the tour by returning to city 1:

$$\text{Best} = \min_{2 \le j \le n} \left[L(1, \{\text{all nodes}\} \setminus \{1, j\}, j) + D[j, 1] \right].$$

This final minimum costs $O(n)$, so the total remains

$$T_n = O(n^2 2^n).$$

4.4 Example 4: Longest Common Subsequence

A subsequence need not be a substring. Consider the example strings

$$X = 001011111 \quad (|X| = n), \qquad Y = 1011000 \quad (|Y| = m).$$

Find the maximum ℓ for which there are index sequences

$$1 \le i_1 < \cdots < i_\ell \le n, \qquad 1 \le j_1 < \cdots < j_\ell \le m,$$

Longest common subsequence recurrence

such that

$$x_{i_k} = y_{j_k}, \qquad 1 \le k \le \ell.$$

The $O(nm)$ dynamic program is

LCS table

```
for i = 0 to n:
  for j = 0 to m:
    if ij = 0:
      L[i,j] ← 0
    else if x_i = y_j:
      L[i,j] ← L[i−1,j−1] + 1
    else:
      L[i,j] ← max{L[i−1,j], L[i,j−1]}
```

Thus the running time is $O(nm)$.

When $x_i \neq y_j$, the recurrence goes into one of two preceding subproblems: $(i-1, j)$ or $(i, j-1)$. These two alternatives connect to the corresponding prefixes of the two strings.

Here is a top-down OCaml implementation:

```
let lcs s1 s2 =
  let n = String.length s1 in
  let m = String.length s2 in
  let memo = Hashtbl.create (n*m) in
  let rec go i j =
    if i = 0 || j = 0 then ""
    else match Hashtbl.find_opt memo (i, j) with
      | Some res -> res
      | None ->
          let res =
            if s1.[i-1] = s2.[j-1] then
              go (i-1) (j-1) ^ String.make 1 s1.[i-1]
            else
              let left = go i (j-1) in
              let up = go (i-1) j in
              if String.length up >= String.length left
              then up else left
          in Hashtbl.add memo (i, j) res;
          res
  in (go n m, String.length (go n m))
```

It outputs the common subsequence as a string together with its length. This top-down recursive filling-in of the table is called *memoization*.

Exercise

Write an $O(n+m)$ program that outputs the indices in $x_1, \ldots, x_n$ and $y_1, \ldots, y_m$ at which the matches occur.

4.5 Example 5: Knapsack/Subset-Sum Problem

The knapsack has capacity $k \in \mathbb{N}$, and the item sizes are $x_1, \ldots, x_n \in \mathbb{N}$. The decision question is

$$\exists S \subseteq \{1, \ldots, n\} \quad \text{such that} \quad \sum_{i \in S} x_i = k \ ?$$

The output is the Boolean table entry $P[n,k]$ ($0 =$ no, $1 =$ yes).

Knapsack/subset-sum table

Fill the table in increasing i and j order:

Subset-sum dynamic program

```
for i = 0 to n:                                          // elements x_1,...,x_i
  for j = 0 to k:                                            // capacity
    if j = 0 and i ≥ 0:
      P[i,j] ← 1
    else if i = 0 and j > 0:
      P[i,j] ← 0
    else if x_i = j:
      P[i,j] ← 1
    else if x_i > j:
      P[i,j] ← P[i − 1,j]
    else:                                                      // x_i < j
      P[i,j] ← max{P[i − 1,j], P[i − 1,j − x_i]}
```

The two entries in the maximum mean, respectively, *do not take* x_i and *take* x_i. The running time is

$$O(kn).$$

Exercises

1. Given that $P[n,k] = 1$, output the items to place in the knapsack.
2. Given the length of an optimal TSP tour, output one optimal route. Hint: store a reference (pointer) to the previous city on the best path from city 1 to the current node.

Chapter 5

Decision Trees, Adversaries, and Lower Bounds

5.1 Worst-Case Running Time and Lower Bounds

Recall that the worst-case running time of an algorithm A is

$$\max_{x_1,\dots,x_n} T(A; x_1, \dots, x_n) = T_n(A).$$

Definition

Given a problem Pb, its lower bound is

$$\min_A T_n(A).$$

5.2 Decision Trees and Lower Bounds

The goal is to lower-bound the *worst-case time of the best algorithm*. A decision tree is one way to show such a lower bound. Each oracle intrinsically has its own number of possible outputs.

Decision tree

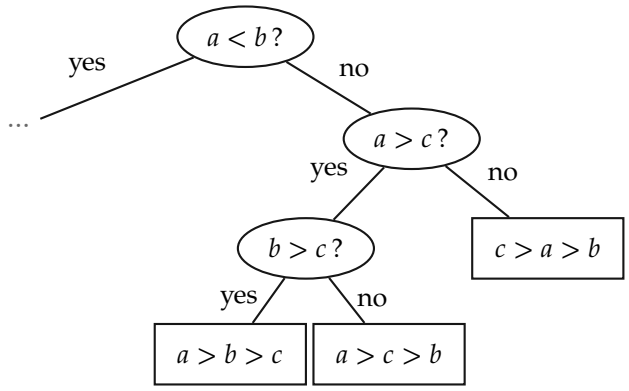
A *decision tree* for an algorithm A is a k-ary tree that visualizes the oracle calls made by A. Here

$$k = \#\{\text{possible outputs of one oracle call}\}.$$

Internal nodes are oracle questions, edges are possible answers, and leaves are answers to the problem.

Example: sorting three elements

For sorting (a, b, c) with a binary comparison oracle, $k = 2$. The following is the portion of the decision tree reached when the first comparison answers $a \not< b$; the other side continues symmetrically.



The longest root-to-leaf path represents the worst case; a shorter path can represent a best case.

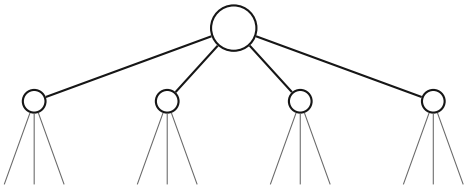
Leaf bound for a k-ary tree

If a k-ary tree has L leaves and height h, then

$$L \leq k^h.$$

Proof. Induct on h. If $h = 0$, then $L = 1 \leq k^0 = 1$. Suppose the claim is true up to height $h - 1$. The root has at most k children, hence at most k subtrees; each subtree has at most k^{h-1} leaves. Therefore

$$L \leq k\, k^{h-1} = k^h. \quad \square$$



k children

k subtrees, each with at most k^{h-1} leaves

The leaf bound gives

$$h \geq \lceil \log_k L \rceil \qquad \Longrightarrow \qquad \min_A T(A) \geq \lceil \log_k L \rceil,$$

where L is the number of answers to the problem and k is the number of outputs of the oracle.

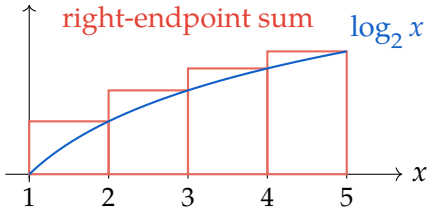
Problem	lower bound	L = # answers
Find all elements $\leq x$ in $\{x_1, \ldots, x_n\}$, using a binary oracle	$\lceil \log_2 2^n \rceil = n$	2^n
Sort $\{x_1, \ldots, x_n\}$, using a binary oracle	$\lceil \log_2 (n!) \rceil$	$n!$
Successful search in $x_1 \leq \cdots \leq x_n$; it is known that the key is present, and the answer is its index	$\lceil \log_2 n \rceil$	n
Unsuccessful search; it is known that the key is absent, and the answer is its interval	$\lceil \log_2 (n+1) \rceil$	$n+1$
General search	$\lceil \log_2 (2n+1) \rceil$	$2n+1$

For unsuccessful search, the n sorted elements determine $n + 1$ gaps:

$$\square\, x_1 \,\square\, x_2 \,\square\, \cdots\, x_n \,\square.$$

Sorting lower bound

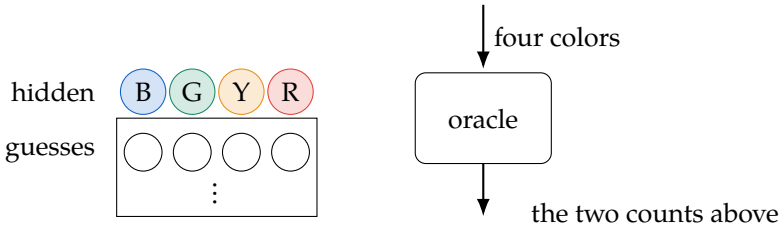
$$\log_2(n!) = \log_2\left(\prod_{i=1}^{n} i\right) = \sum_{i=1}^{n} \log_2 i$$
$$\geq \int_1^n \log_2 x \, dx = n \log_2 n - (n-1)\log_2 e.$$



Mastermind with six colors

A hidden code and every guess contain four colors. The oracle returns the pair

(# correct colors in the correct position, # correct colors in a wrong position).



Mastermind: counting oracle outputs

View an oracle output as a pair (i, j). With four positions, the candidate outputs are

$i+j$	possible pairs
0	$(0,0)$
1	$(0,1), (1,0)$
2	$(2,0), (1,1), (0,2)$
3	$(0,3), (1,2), \ldots, (3,0)$
4	$(0,4), (1,3), \ldots, (3,1), (4,0)$

The red pair $(3,1)$ is impossible: once three positions are exactly correct, the sole remaining position cannot contain a correct color in a different position. Thus

$$k = (1+2+3+4+5) - 1 = 15 - 1 = 14.$$

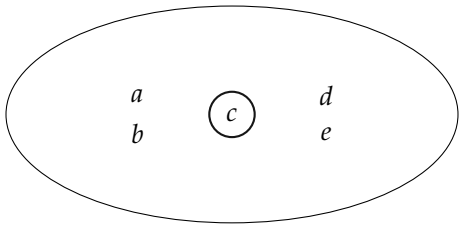
There are 6^4 hidden codes, so

$$\text{lower bound} = \left\lceil \log_{14}(6^4) \right\rceil = \left\lceil 4 \log_{14} 6 \right\rceil = 3.$$

Median of five

The oracle is a binary comparison oracle, so $k = 2$. To specify an answer, choose the median and then choose which two of the remaining four elements lie below it:

$$L = 5\binom{4}{2} = 5 \cdot 6 = 30 \qquad \text{(not merely 5)}.$$



$\binom{4}{2} \times 5 = 30$ answer configurations

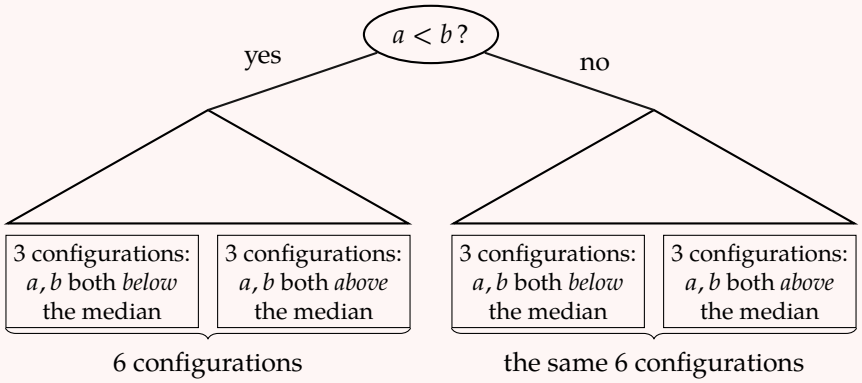
The naive leaf count gives

$$\lceil \log_2 30 \rceil = 5.$$

Why is the true comparison lower bound 6?

Why the count 30 is not yet enough

At a node asking $a < b$?, a branch can leave both a and b on the same side of the median. The comparison then fails to distinguish two orders that lead to the same median answer. Schematically, both branches retain duplicated answer configurations:



the same six configurations appear under both branches:
six duplicate leaves, hence $L \geq 30 + 6 = 36$

The duplicate-leaf correction follows next.

There are six duplicate leaves. The question $a < b$? did not eliminate these extra leaves because a and b can lie on the same side of the median. Hence the relevant decision tree has

$$L = 30 + 6 = 36, \qquad \text{LB} = \lceil \log_2 36 \rceil = 6.$$

5.3 Merging Sorted Lists

Let

$$x_1 < \cdots < x_n, \qquad y_1 < \cdots < y_m.$$

A merged order is determined by choosing the m positions occupied by the y-elements among the $m+n$ output positions. Therefore

$$L = \binom{m+n}{m} = \binom{m+n}{n} = \frac{(m+n)!}{m!\,n!}.$$

Moreover,

$$\begin{aligned}\binom{m+n}{m} &= \frac{m+n}{m} \cdot \frac{m+n-1}{m-1} \cdots \frac{n+1}{1} \\ &\geq \left(\frac{m+n}{m}\right)^m = \left(1+\frac{n}{m}\right)^m,\end{aligned}$$

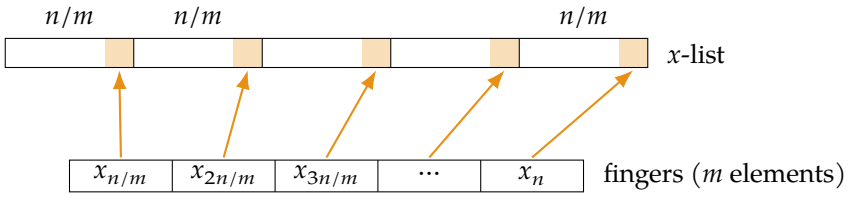
because $(m+n)/m$ is the smallest of the displayed factors. Thus

$$\mathrm{LB} \geq \left\lceil \log_2 \binom{m+n}{m} \right\rceil \geq \left\lceil m \log_2\left(1+\frac{n}{m}\right) \right\rceil.$$

Approaching the bound with finger lists

Assume $m \mid n$ and $n \geq m$.

Step 1. Divide the x-list into m blocks of size n/m, and retain one "finger" per block, for example $x_{n/m}, x_{2n/m}, \ldots, x_n$.



Step 2. Perform an ordinary merge of the finger list (size m) with the list of m y-elements. This takes $T \leq 2m$. It identifies, in terms of blocks, where every y-element belongs.

Step 3. Insert the m y-elements into their respective blocks by binary search. Each block has size n/m, so

$$T = m \left\lceil \log_2\left(1+\frac{n}{m}\right) \right\rceil.$$

5.4 Adversary Lower Bounds

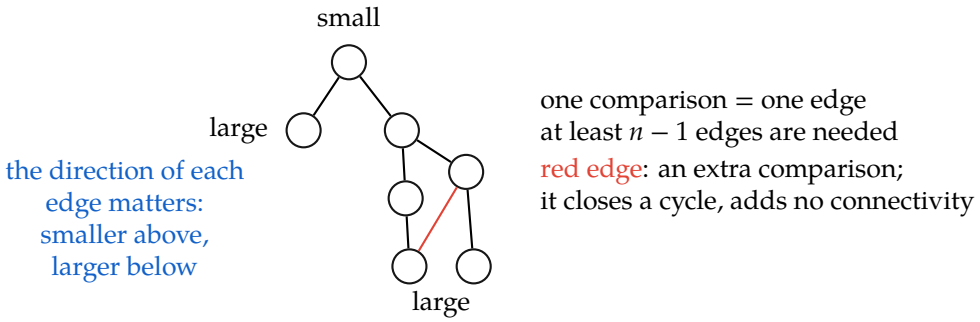
Finding a minimum

For finding $\min(x_1, \ldots, x_n)$, the mere answer count is $L = n$, so the binary decision-tree argument gives only

$$\lceil \log_2 n \rceil,$$

even though the best bound is $n-1$ comparisons.

Instead, form a comparison (Hasse) graph: one comparison contributes one edge. To prove which element is the minimum, the comparison graph must connect all n elements; otherwise an element in another component could still be the minimum. A connected graph on n vertices has at least $n-1$ edges. Hence finding the minimum requires at least $n-1$ comparisons.



Guessing an n-bit password

With a binary identification oracle, an adversary can keep answering "wrong" until the 2^nth guess. Thus the worst case is at least 2^n:

$$\min_{A} \max_{x \in \{0,1\}^n} T(A;x) \geq 2^n.$$

Finding both the minimum and the maximum

A direct method takes

$$(n-1) + (n-2) = 2n-3$$

comparisons. A better method first compares elements in pairs. Every pair produces a candidate for the minimum (the smaller member) and a candidate for the maximum (the larger member).

For even n, the displayed count is

$$\underbrace{\frac{n}{2}}_{\text{compare pairs}} + \underbrace{\left(\frac{n}{2}-1\right)}_{\text{find minimum}} + \underbrace{\left(\frac{n}{2}-1\right)}_{\text{find maximum}} = \frac{3n}{2} - 2 = \left\lceil \frac{3n}{2} \right\rceil - 2.$$

For odd n, pair $n-1$ elements and leave one extra element:

$$\frac{n-1}{2} + \frac{n-1}{2} + \frac{n-1}{2} + 2 = \left\lceil \frac{3n}{2} \right\rceil,$$

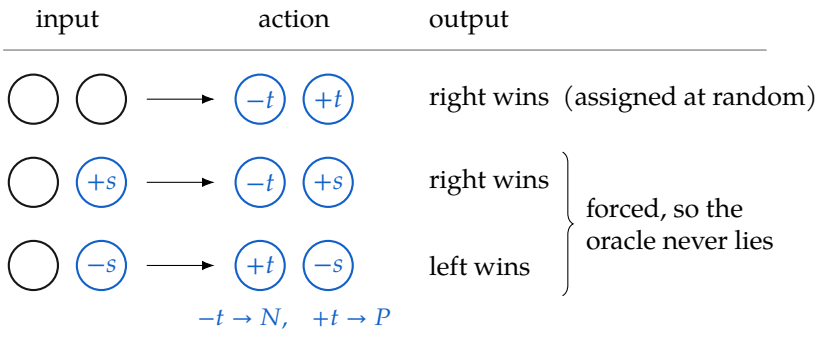
where the final two terms are comparisons involving the extra element. The construction is then claimed to be optimal.

Discrepancy between the displayed counts

The odd-n display above has no "-2", whereas the adversary conclusion in the next section is $\lceil 3n/2 \rceil - 2$.

Adversary setup

Let t be a clock. Initially every item is unclassified. When two unclassified items are compared, the adversary assigns one to the negative set N (a possible minimum) and one to the positive set P (a possible maximum), with signs $-t$ and $+t$; the announced winner is chosen so that these assignments remain consistent. Comparisons between already classified items are answered without tying the two candidate sets together. The counting argument follows next.



To empty the starting set of unclassified elements, the adversary forces at least $\lceil n/2 \rceil$ comparisons. It then takes $|P| - 1$ comparisons to identify the maximum within the positive set P, and $|N| - 1$ comparisons to identify the minimum within the negative set N.

The sets P and N contain all distinct elements that were ever placed in them. Every number is classified as positive or negative and is placed in P or N once, so

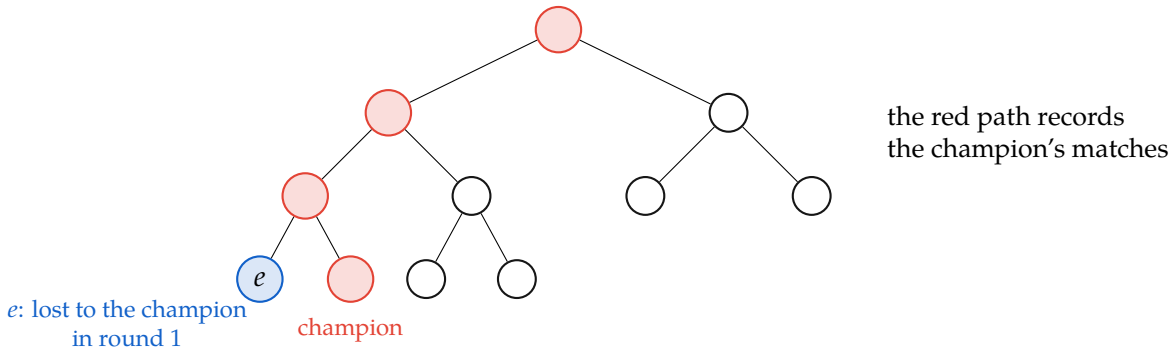
$$|P| + |N| = n.$$

Consequently,

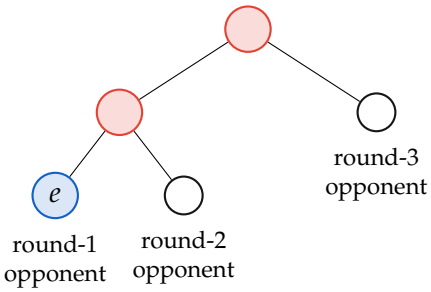
$$\#\text{comparisons} \geq \left\lceil \frac{n}{2} \right\rceil + (|P| - 1) + (|N| - 1) = \left\lceil \frac{3n}{2} \right\rceil - 2.$$

Finding the two best players

Put the n players at the leaves of a complete binary tournament tree. The tournament has at most $n - 1$ internal nodes, so finding the best player takes $n - 1$ matches.



To find the second-best player, remove the champion and replay only the matches along the champion's path. Matches unaffected by the removal need not be played again. The champion met at most $\lceil \log_2 n \rceil$ opponents, so the rematch among those opponents takes at most $\lceil \log_2 n \rceil - 1$ comparisons.



rematch after removing the champion: only its $\lceil \log_2 n \rceil$ opponents play again, in $\lceil \log_2 n \rceil - 1$ matches (red)

$$\#\text{matches} \leq (n-1) + \left\lceil \log_2 n \right\rceil - 1.$$

Also, a complete binary tree with n leaves has at most $2n-1$ total nodes and height $\lfloor \log_2(2n-1) \rfloor = \lceil \log_2 n \rceil$.

Yao's adversary oracle

Each item carries a pair

$$(\text{assigned value}, \text{weight}).$$

Initially no value is assigned and every item has weight 1, so $\sum w = n$. Let t be a clock. At the end, one item has weight n and the other $n-1$ items have weight 0. The second-largest element is chosen to be the weight-zero item having the largest assigned clock value.

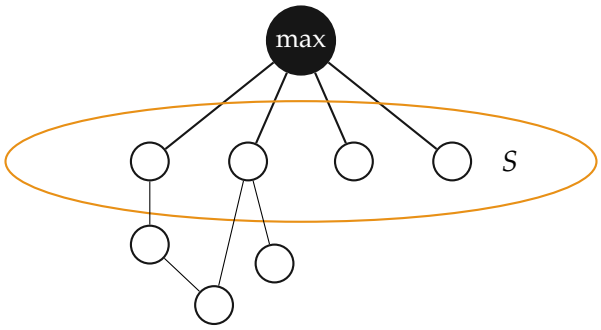
The oracle behaves as follows.

Input states	Internal action	Output
$(?, w)$ and $(?, w')$, with $w, w' > 0$	If the weights differ, the larger-weight item wins; equal weights are tie-broken arbitrarily. The winner gets weight $w + w'$, while the loser is assigned value t and weight 0.	The chosen winner *eats* the loser's weight.
$(s, 0)$ and $(s', 0)$	No change.	The larger of s, s' wins.
$(s, 0)$ and $(?, w')$, with $w' > 0$	No change.	The nonzero-weight item wins.

A larger positive weight therefore eats a smaller one; equal positive weights are tie-broken arbitrarily, and every positive-weight item beats every zero-weight item.

In the resulting comparison (Hasse) diagram, let

$$S = \{\text{all items eaten directly by the maximum}\}.$$



Fact 1

$$|S| \geq \lceil \log_2 n \rceil .$$

Indeed, after the maximum has eaten k elements directly, its weight can be at most 2^k: it cannot eat a weight larger than its own.

# eaten directly	0	1	2	$\cdots$	k
weight of maximum	1	≤ 2	≤ 4	$\cdots$	$\leq 2^k = n$

The weight argument yields

$$k \geq \lceil \log_2 n \rceil .$$

Fact 2

The set S contains the second-largest element.

The last element eaten directly by the maximum receives the largest clock time among those elements, because it is eaten last. By the adversary's final value assignment, that element is the second largest. Therefore, after the $n-1$ comparisons needed to identify the maximum, an algorithm must still find the maximum of S. Hence

$$\#\text{comparisons} \geq (n-1) + \left(\lceil \log_2 n \rceil - 1 \right).$$

5.5 Median of $2n+1$ Elements

There are

$$L = (2n+1)\binom{2n}{n}$$

possible median certificates: choose the median, then choose which n of the remaining $2n$ items lie below it. The central binomial coefficient is the largest term in

$$(1+1)^{2n} = \sum_{k=0}^{2n} \binom{2n}{k} = 4^n.$$

Since the sum has $2n+1$ terms,

$$\binom{2n}{n} \geq \frac{4^n}{2n+1}, \qquad (2n+1)\binom{2n}{n} \geq 4^n.$$

The decision-tree count is too weak

The binary leaf count gives only

$$\log_2 L \geq \log_2(4^n) = 2n.$$

This $2n$ conclusion is insufficient; the stronger adversary argument follows.

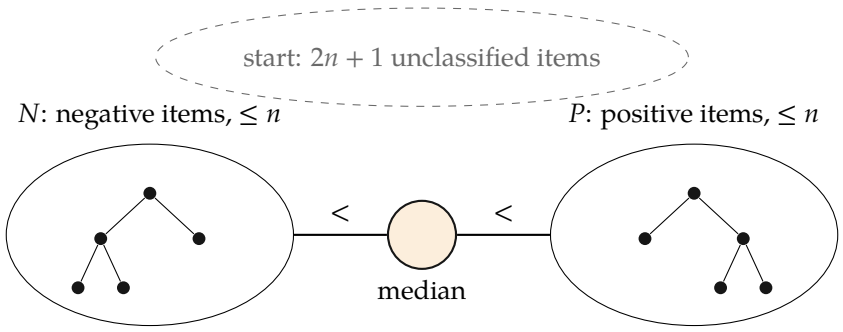
Adversary oracle

Start with $2n + 1$ unclassified items. The adversary builds three regions:

- a negative set N of at most n items;
- one eventual median;
- a positive set P of at most n items.

This is the min/max adversary adapted so that newly assigned items go to the correct set:

$$-t \longrightarrow N, \qquad +t \longrightarrow P.$$



Correction/warning. A naive "no change" response can overflow a set when $|P| = n$ (and, symmetrically, when $|N| = n$). The oracle must assign a newly classified item to the other available set so that both size caps remain valid. The comparison count is completed next.

For the median adversary, at least n comparisons are needed to fill P and N by comparing pairs from the starting set. A further $2n$ comparison witnesses in the Hasse trees for P and N are needed to prove that the remaining element really has n elements below it and n above it. Thus

$$\#\text{comparisons} \geq \underbrace{n}_{\text{fill } P, N} + \underbrace{2n}_{\text{witness edges in } P, N} = 3n.$$

Chapter 6

Linear Abstract Data Types and Stack Applications

6.1 Abstract Data Types

Examples include lists, stacks, queues, deques, and (later) trees.

Abstract data type (ADT)

An ADT is an object containing data together with a set of operations.

Lists

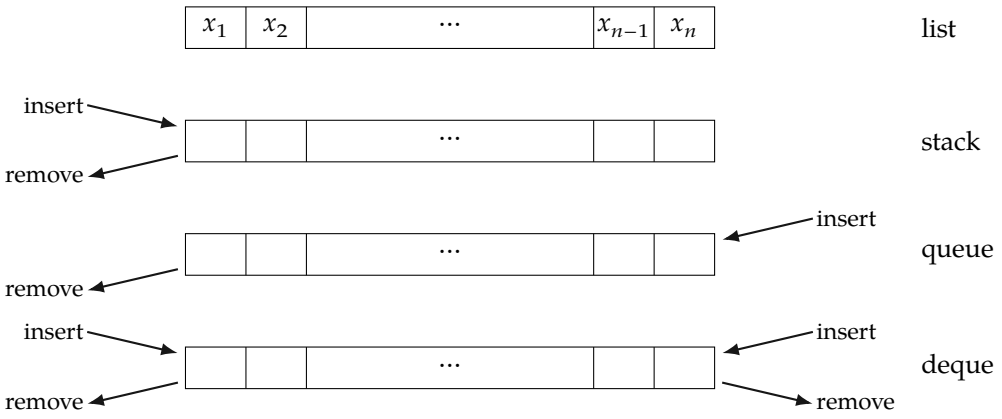
A list is a linearly ordered set

$$L = [x_1, x_2, \dots, x_n].$$

Two operations are atomic, meaning that every other list operation can be built from them.

1. **Sublist:**

$$\begin{aligned} \text{Sublist}(L, [i, \dots]) &= [x_i, \dots, x_n], \\ \text{Sublist}(L, [\dots, i]) &= [x_1, \dots, x_i], \\ \text{Sublist}(L, [i, \dots, j]) &= [x_i, \dots, x_j]. \end{aligned}$$



The second atomic list operation is

2. **Concatenation:**

$$\text{Concatenation}(L_1, L_2) = L_1 \,\&\, L_2 = [L_1\ L_2].$$

Common operations can be expressed using only sublists and concatenation:

$$\begin{aligned}
\text{Push}(x, L) &= [x] \,\&\, L,\\
\text{Pop}(L) &= \text{Sublist}(L, [2, \ldots]),\\
\text{Access}(i, L) &= \text{Sublist}(L, [i, \ldots, i]),\\
\text{Delete}(i, L) &= \text{Sublist}(L, [\ldots, i-1]) \,\&\, \text{Sublist}(L, [i+1, \ldots]),\\
\text{Insert}(x, j, L) &= \text{Sublist}(L, [\ldots, j]) \,\&\, [x] \,\&\, \text{Sublist}(L, [j+1, \ldots]).
\end{aligned}$$

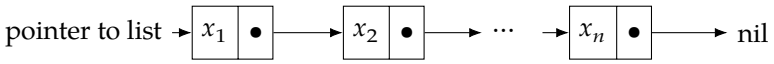
The last operation inserts x after the jth element.

Data structure	Sublist	Concatenate
Array	$O(1)$	$\Theta(n)$
Linked list	$\Theta(n)$	$O(1)$
BST	$O(\log n)$	$O(\log n)$

6.2 Linked Lists

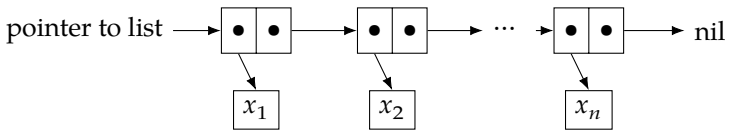
Singly linked list

A singly linked list is a collection of cells, each holding a data object and a unique pointer to another cell called its *successor*.



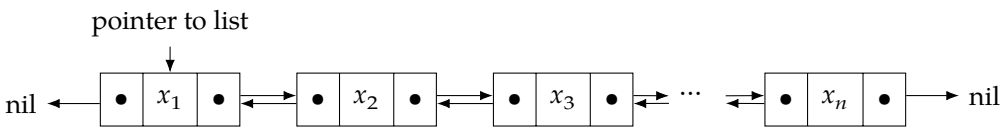
The representation may be

- *endogenous*: the data is stored in the cell; or
- *exogenous*: the data is stored outside, and the cell contains a pointer to it.



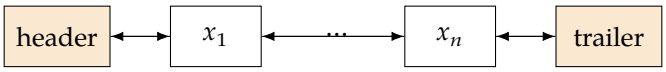
Doubly linked list

A doubly linked list is a collection of cells, each containing a data object, a pointer to its successor, and a pointer to its predecessor.

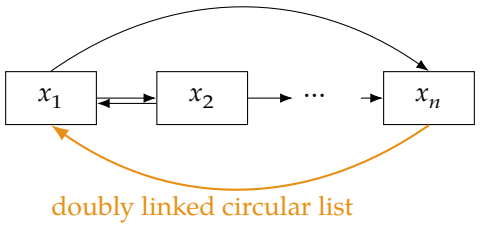


Dummy head and tail

To avoid the special case of an empty list, use a dummy header and trailer.



A header node often stores useful aggregate information such as the list's size, maximum, or average. A doubly linked list can also be made circular:

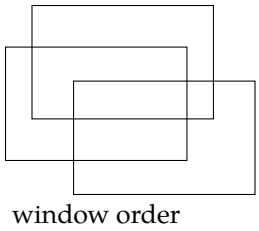


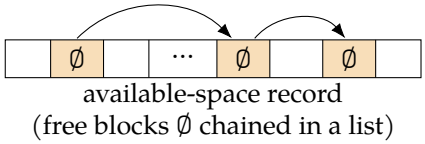
Applications of linked lists

- **Polynomial derivatives.** Represent a polynomial by the coefficient list $[a_0, a_1, \ldots, a_n]$. Then

$$P' = [0a_0, 1a_1, \ldots, na_n].$$

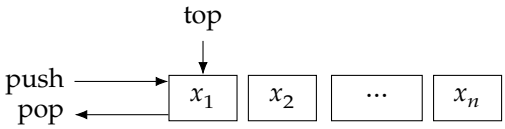
- **Window manager.** A linked list represents the order in which windows are open.
- **Memory management.** A linked list records available regions of memory.





6.3 Stacks

Stacks support push and pop in **FILO** order: first in, last out.



Operations include

$$\begin{aligned} \text{Top}(S): &\ \text{return the top element,} \\ \text{MakeNull}(S): &\ \text{create a new empty list.} \end{aligned}$$

The remaining stack operation is

$$\text{Empty}(S): \text{ return true iff } |S| = 0.$$

With linked lists, all the stack operations above take constant time $O(1)$.

6.4 Queues

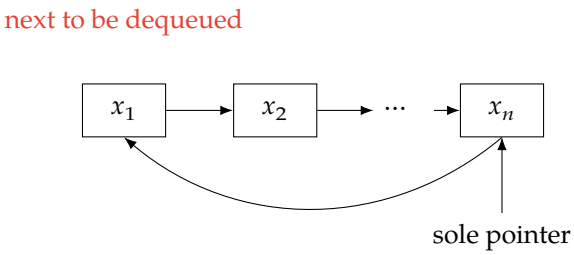
Queues support enqueue and dequeue in **FIFO** order: first in, first out. Using the list primitives,

$$\text{Enqueue}(x, Q) = Q \,\&\, [x],$$
$$\text{Dequeue}(Q) = \text{Pop}(Q).$$

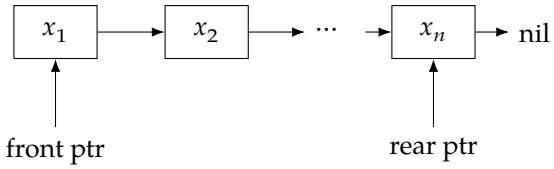
The other operations are MakeNull(Q) and Empty(Q).

Queue implementations

Single-pointer implementation. Use a circular singly linked list and store a single pointer to the rear. The rear's successor is the front, the next item to be dequeued.



Double-pointer implementation. Use a singly linked list with a front pointer to x_1 and a rear pointer to x_n.



6.5 Deques

A deque is a *doubly ended queue*. Its operations are

left end	right end
push(x, D)	inject(x, D) (push right)
pop(D)	eject(D) (pop right)

A doubly linked circular list implements all these operations in $O(1)$ time.

6.6 Application: Matching Parentheses

Use a stack to evaluate/parse an expression and match parentheses.

```
Parse(str):
  c ← str's first character
  MakeNull(S)
  while c ≠ nil:
    if c is a left parenthesis:
      Push(c,S)
    else if c is a right parenthesis:
      if Empty(S):
        return error
```

The delimiter-matching procedure concludes next.

6.7 Stack Applications and Recursion

The delimiter-matching scan concludes as follows:

Balanced-delimiter scan (conclusion)

```
else
  c' ← Pop(S)
  if c′ and c do not match, return error
c ← next character in the string
if not Empty(S), return error
```

Other important uses of a stack include:

- evaluating postfix, prefix, and infix expressions;
- depth-first traversal;
- recursion and the call stack.

On a function call, a frame is pushed onto the call stack. Execution returns to the caller only after the callee has returned and its frame has been popped.

6.7.1 Finding a path through a maze

EscapePath(Maze, start)

```
MakeNull(S)
Push(start, S)
Mark(start) // mark the node as visited
while Top(S) ≠ exit:
  if Top(S) has an unmarked neighbour v:
    Push(v,S); Mark(v)
  else:
    Pop(S)
```

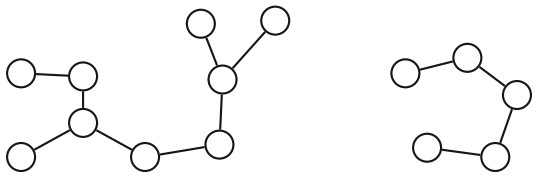
Chapter 7

Rooted Trees, Traversals, and Succinct Encodings

7.1 Rooted Trees

Definition

A *free tree* is a connected acyclic graph. A *rooted tree* is a free tree with one distinguished node, called the *root*, denoted Root(T).



a free tree

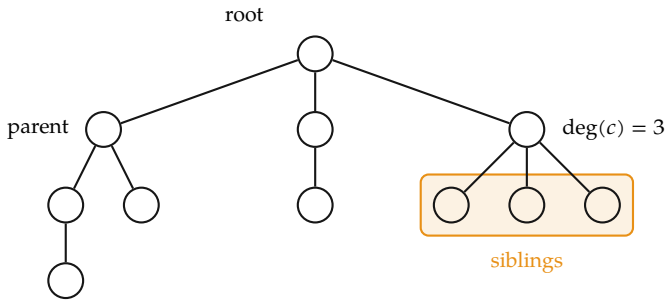
cycle ⇒ not a tree

7.1.1 Rooted and ordered trees

Equivalently, a rooted tree consists of:

1. a single distinguished node called the root; and
2. the remaining nodes, partitioned into disjoint sets $T_1, \ldots, T_m$, each itself a tree. These are the *subtrees* of the root.

An *ordered rooted tree*, or *plane tree*, is a rooted tree in which the order of the subtrees matters. A *k-ary tree* is a rooted tree in which every node has at most k subtrees (equivalently, degree at most k in the usual rooted-tree sense).



Terminology: root, parent, child, oldest/youngest child, siblings, ancestors, descendants, leaves (nodes with no children), edges, and the subtree rooted at a node. A root-to-leaf path has length equal to the depth of its terminal node; the maximum depth is the tree's height.

7.1.2 Complete binary trees

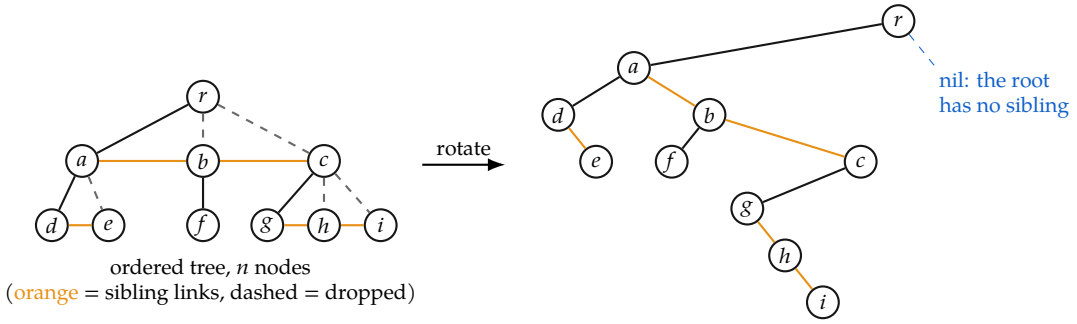
A *complete binary tree* has every level full except possibly the last, which is filled from left to right. With n nodes and height h,

$$1 + 2 + \cdots + 2^{h-1} < n \leq 1 + 2 + \cdots + 2^h,$$

so

$$2^h \leq n < 2^{h+1}, \qquad h = \lfloor \log_2 n \rfloor.$$

There is a bijection between binary trees on $n-1$ nodes and ordered trees on n nodes, obtained using the left-child/right-sibling representation. Question: why does the shift from $n-1$ to n occur, and why does the binary-tree root have no right sibling?



7.1.3 Tree representations

A binary-tree node can store (data, left, right), optionally with a parent pointer. A general ordered tree can instead store (data, oldest child, next sibling).

Implicit representation of a complete binary tree. Label nodes level by level starting at 1. For a node at array index i,

$$\text{left}(i) = 2i, \qquad \text{right}(i) = 2i + 1, \qquad \text{parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor.$$

Thus a complete binary tree can be stored directly in an array.

The node's position can also be read from the binary representation of its index. Remove the leading 1; among the remaining h bits, 0 means "go left" and 1 means "go right." For example,

$$11 = (1011)_2 \quad \rightsquigarrow \quad 011,$$

so the path is left, right, right.

7.1.4 Basic properties of trees

1. Every tree with n nodes has $n - 1$ edges.
   Proof. Induct on n. Adding a node while preserving the tree property requires exactly one new edge: zero would disconnect the node, while two or more would create a cycle. Hence $n + 1$ nodes have n edges. □
2. If n_i is the number of nodes with i children, then counting edges by their parent gives

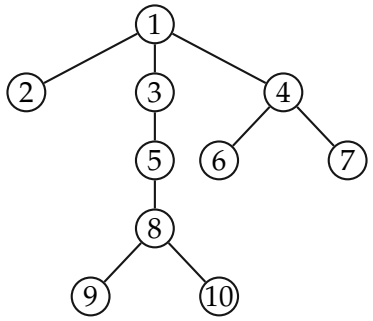
$$n - 1 = n_1 + 2n_2 + 3n_3 + \cdots = \sum_{i=1}^{\infty} i\, n_i.$$

Note

A third property, proved later, is that the number of binary-tree shapes with n nodes is a Catalan number (with the index shifted by one under the course's convention).

7.2 Tree traversals

The main traversals are preorder, postorder, level order, and—for binary trees—inorder.



Preorder

See a node from its left window: visit the root, then traverse its subtrees from left to right. For the tree above,

$$1, 2, 3, 5, 8, 9, 10, 4, 6, 7.$$

Postorder

See a node from its right window: traverse subtrees left to right, then visit the root. The order is

$$2, 9, 10, 8, 5, 3, 6, 7, 4, 1.$$

Inorder (binary trees)

See a node from a window below it: traverse the left subtree, visit the node, then traverse the right subtree. A tree with root 1, left child 2, and a right chain 3, 4 has order $2, 1, 3, 4$.

Level order / breadth first

Visit nodes level by level. For the displayed tree the order is $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$.

Recursive preorder

```
Preorder(t):
  if t ≠ nil:
    visit(t)
    for every child v of t, from left to right:
      Preorder(v)
```

7.2.1 Iterative traversal and traversal ranks

LevelOrder(t)

```
MakeNull(Q) // queue
Enqueue(t, Q)
while |Q| ≥ 1:
  v ← Dequeue(Q)
  visit(v)
  for every child w of v, from left to right:
    Enqueue(w, Q)
```

Convention: u, v, w denote vertices and Q denotes a queue.

For the example above, node 5 has preorder rank 4 and postorder rank 5. Storing both ranks lets us determine the relationship of two nodes directly. Relative to a fixed node v, the four regions are its left side L, right side R, ancestors A, and descendants D:

region of u	comparison with $\text{pre}(v)$	comparison with $\text{post}(v)$
L	$\text{pre}(u) < \text{pre}(v)$	$\text{post}(u) < \text{post}(v)$
R	$\text{pre}(u) > \text{pre}(v)$	$\text{post}(u) > \text{post}(v)$
A	$\text{pre}(u) < \text{pre}(v)$	$\text{post}(u) > \text{post}(v)$
D	$\text{pre}(u) > \text{pre}(v)$	$\text{post}(u) < \text{post}(v)$

Note

The pair $(\text{pre}, \text{post})$ therefore answers ancestor/descendant and left/right questions without walking through the tree.

7.2.2 Stack-based preorder

The recursive call stack can be made explicit in the following algorithm.

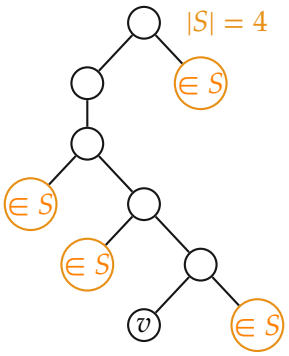
StackBasedTraversal(t)

```
MakeNull(S) // stack
Push(t, S)
while |S| ≥ 1:
  v ← Pop(S)
  visit(v)
  for every child w of v, from right to left:
    Push(w, S)
```

Children are pushed right-to-left so that the leftmost child is popped first. The difficulty is that the stack may become large.

7.2.3 Parsimonious stack traversal

Suppose subtree sizes are stored. At a branch, continue immediately into the smaller subtree and push the larger subtree for later. In the binary case: always stack the larger subtree.



If v is the current node and S is the stack, then

$$1 \le |\text{subtree}(v)| \le \frac{n}{2^{|S|}}.$$

Indeed, v lies in the current subtree. Every time a subtree is pushed, the algorithm continues into the other, smaller subtree, whose size is at most half the current size. Consequently $|S| \le \lfloor \log_2 n \rfloor$.

7.2.4 Preorder using parent pointers

The following parent-pointer idea is an intentionally incomplete draft. It is retained because the exercise asks us to find its bugs.

Draft algorithm: preorder without a stack or recursion

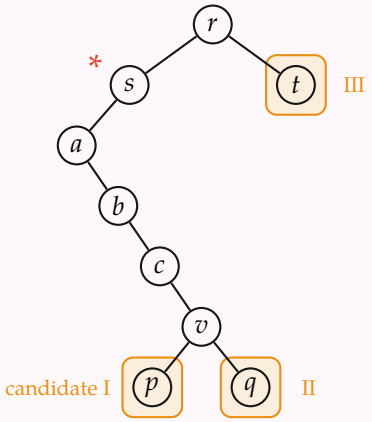
```
Preorder(t):
  while t ≠ nil:
    visit(t)
    t ← next(t)

next(v):
  if left[v] ≠ nil: return left[v]
  else if right[v] ≠ nil: return right[v]
  else:
    w ← parent[v]
    while w ≠ nil and v = right[w]:
      v ← w
      w ← parent[w]
    if right[w] ≠ nil: return right[w]
```

Exercise: fix two errors

1. Trace the code when the traversal reaches its last node. What can happen?
2. What happens when w climbs to the node marked $*$ in the tree below?

The traversal is at v; the orange nodes are the candidates for next(v).



7.3 Compression of binary trees

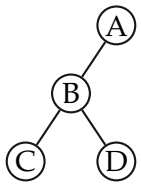
For a binary node, encode its child pattern with two bits:

00	no children	10	left child only
01	right child only	11	both children

Reading these codes in preorder stores the tree shape in $2n$ bits. For the tree with preorder A, B, C, D where A has only a left child B and B has both children C, D, the code is

10 11 00 00.

The tree can be reconstructed in $O(n)$ time.

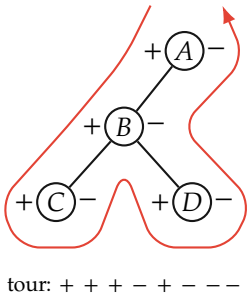


7.3.1 Root coding and the information-theoretic lower bound

An alternative is *root coding*: write + when a node is first entered and − when it is left. The preceding four-node example gives

$$+\ +\ +\ -\ +\ -\ -\ -,$$

again $2n$ symbols/bits, and the tree is reconstructible in $O(n)$ time. The handwritten notes write this code as $+\ +\ +\ -\ -\ +\ -\ -$, which is the code of a different tree (A with children $B(C)$ and D); the Euler tour drawn around $A(B(C,D))$ gives the code above.



tour: + + + − + − − −

Lower bound

Representing the shape of an arbitrary binary tree on n nodes requires $2n - O(\log n)$ bits in the worst case; hence $2n$ bits is asymptotically optimal.

The number of binary-tree shapes of size n is the Catalan number

$$C_n = \frac{1}{n+1}\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi}\, n^{3/2}}.$$

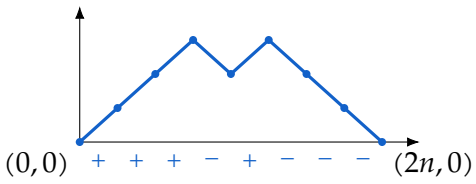
Distinguishing C_n possible objects needs at least

$$\begin{aligned}\log_2 C_n &\sim \log_2\left(\frac{4^n}{\sqrt{\pi}\, n^{3/2}}\right) \\ &= 2n - \log_2 \sqrt{\pi} - \frac{3}{2}\log_2 n = 2n - O(\log n)\end{aligned}$$

bits. The dominant term is $2n$.

7.3.2 Catalan paths

View the tree code as a walk with + an up-step and − a down-step. A valid code is a path from $(0,0)$ to $(2n,0)$ that never goes below the axis (a Dyck path). There are $\binom{2n}{n}$ unrestricted walks with n up-steps and n down-steps; invalid paths are subtracted by the reflection argument below.



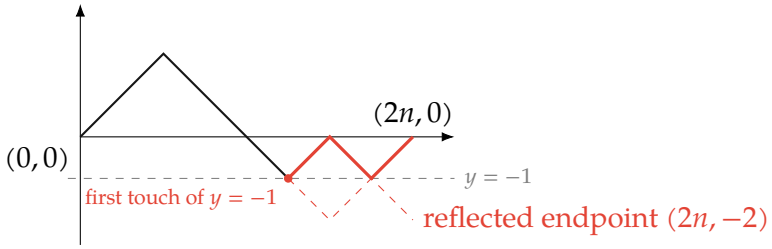
7.3.3 Reflection principle and the Catalan formula

The invalid paths from $(0,0)$ to $(2n,0)$—those that go below the axis—are in bijection with unrestricted paths from $(0,0)$ to $(2n,-2)$. Reflect the suffix beginning with the first step that reaches $y=-1$. The reflected path has $n+1$ down-steps and $n-1$ up-steps, so there are

$$\binom{2n}{n+1}$$

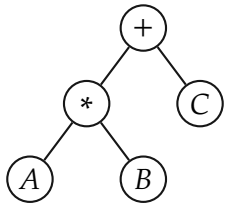
invalid paths. (These nonnegative walks are the Dyck paths.) Therefore

$$\binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1}\binom{2n}{n} = C_n.$$



7.4 Expression trees and Polish notation

For the expression $(A * B) + C$, the operator is the parent of its operands:



Postorder gives postfix (reverse Polish) notation $AB*C+$; preorder gives prefix (Polish) notation $+*ABC$. Both encode the tree without parentheses.

Evaluate a postfix expression

```
Postfix(E):
  MakeNull(S) // stack
  for every symbol s ∈ E, left to right:
    if s is an operand:
      Push(s, S)
    else:
```

```
    β ← Pop(S)
    α ← Pop(S)
    // combine as specified below
```

The postfix evaluation concludes with

Evaluate a postfix expression (conclusion)

```
    Push(α s β, S)
  return Pop(S)
```

For example, if $s = *$ and the two popped values are A and B, the combined value is $A * B$.

Chapter 8

Dictionary ADT, Balanced Search Trees, and Augmented Structures

8.1 The dictionary ADT and search trees

Dictionary

The model contains items that can be compared. Operations include *Search*, *Insert*, *Delete*, *MakeNull*, *Minimum*, *Maximum*, *Successor*, *Predecessor*, and sorting.

Dictionary implementations include:

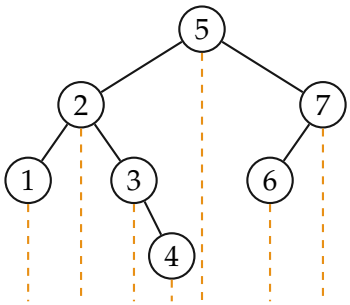
- search trees: ordinary BSTs and balanced BSTs of height $O(\log n)$, including red–black trees, AVL trees, and HB-k trees;
- static BSTs (for example, complete BSTs);
- randomized BSTs (for example, treaps);
- amortized BSTs, including splay trees and weight-balanced trees;
- string structures such as tries and suffix trees;
- hashing.

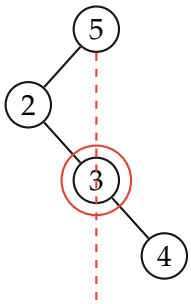
8.1.1 Binary search trees

Every node v stores a key and satisfies

$$\text{all keys in the left subtree} < \text{key}(v) < \text{all keys in the right subtree.}$$

The separating line through v can be drawn so that it intersects no subtree edge (dashed lines below every node of the tree on the left; the small tree on the right breaks this convention, since 3 is drawn on the line below 5).





bad: 3 sits on the line below 5

8.1.2 BST search and insertion

Recall: sorting with a heap

Starting with a chaotic unsorted list, a heap can be built in $O(n)$ time and then drained into sorted order within the overall $n \log_2 n$ bound. A $\frac{1}{2} n \log_2 n$ portion motivates the side question "Why does it take $\frac{1}{2} n \log_2 n$?"; the corresponding divide-and-conquer view repeatedly pairs and merges smaller lists.

Rule 1: an inorder traversal of a BST is in sorted order.

Rule 2: if a BST is flattened horizontally in the way it is normally drawn, the keys appear in sorted order as well.

TreeSearch(t, k)

```
x ← t
while x ≠ nil and k ≠ key[x]:
  if k < key[x]:
    x ← left[x]
  else:
    x ← right[x]
return x
```

Insert(t, k), where k is not already in t

```
x ← t;  y ← nil
while x ≠ nil:
  if k < key[x]:
    y ← x;  x ← left[x]
  else:
    y ← x;  x ← right[x]
CreateNode(x)
parent[x] ← y
left[x], right[x] ← nil
key[x] ← k
if k < key[y]:
  left[y] ← x
else:
  right[y] ← x
```

8.1.3 Deletion and successor

To delete node x from a BST:

1. If x is a leaf, detach it from its parent y and replace the corresponding child pointer by nil.
2. If x has exactly one child, bypass x: connect its parent y directly to that child.
3. If x has two children, choose
$$y \leftarrow \text{Maximum}(\text{left}[x]),$$
copy key$[y]$ into x, and delete y. The predecessor y has at most one child, so this reduces to case 1 or 2.

Successor(x, t)

```
if right[x] ≠ nil:
  return Minimum(right[x])
else:
  y ← parent[x]
  while y ≠ nil and right[y] = x:
    x ← y
    y ← parent[y]
  return y
```

If the loop returns nil, then x has no successor. Otherwise it stops at the first ancestor for which the upward path entered from the left.

In the RAM model these operations take $O(\text{height})$ time. Repeating the successor operation k times takes

$$O(\text{height} + k),$$

not $O(\text{height} \cdot k)$; the upward portions telescope and each reported successor contributes only constant additional work.

8.1.4 Completing the k-successor analysis

Between a node x and its kth successor y, there are $k - 1$ intervening nodes. Every step within those intervening subtrees is traversed at most twice, and each of the two boundary paths has length at most the height h. Therefore

$$T \leq 2k + 2h = O(h + k).$$

8.2 Red–black trees

A red–black tree is a BST whose node records contain

$$(\text{parent}, \text{left}, \text{right}, \text{key}, \text{color} \in \{\text{red}, \text{black}\}, \text{rank} \in \mathbb{N}_0).$$

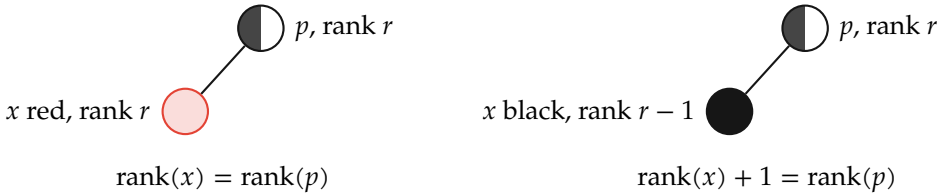
The drawings use black circles for black nodes, red circles for red nodes, squares for external (nil) nodes, and half-shaded circles when a color is not yet known.

Rank/color rule

If x is a child of p, then

$$\operatorname{rank}(p) = \begin{cases} \operatorname{rank}(x), & x \text{ is red}, \\ \operatorname{rank}(x) + 1, & x \text{ is black}. \end{cases}$$

Thus rank counts black nodes on a downward path.



8.2.1 Red–black invariants and pods

Two immediate consequences/requirements are:

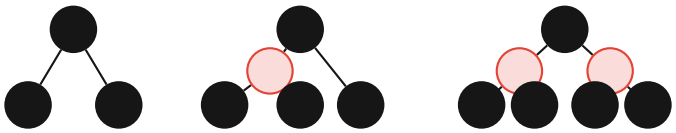
1. red nodes cannot have red children. Equivalently, for every node x whose grandparent exists,

$$\operatorname{rank}(\text{grandparent}(x)) > \operatorname{rank}(x);$$

2. every external leaf is black and has rank 0.

When the explicit external leaves are included, a tree with n internal nodes has $n + 1$ external nodes and $2n + 1$ total nodes. Indeed, every internal node has two child edges, so there are $2n$ edges; a tree with $2n + 1$ total nodes also has $2n$ edges.

Pods. Group together a black node and its red children. Nodes of a common black rank form one layer; a pod has either two, three, or four black child pods:



8.2.2 Height of a red–black tree

Because every pod has two, three, or four black children, red–black trees are also called *2–3–4 trees*.

Let $r = \operatorname{rank}(\text{root})$ and let h be the ordinary height. A path can have no red nodes at all, or at most one red node between successive black-rank layers. Hence

$$r \le h \le 2r.$$

Theorem

For a red–black tree with n internal nodes and root rank r,

$$2^r \le n + 1 \le 4^r.$$

Lower bound. Induct on r. Rank 0 has no internal node and one external leaf. A rank-r root has at least two child pods of rank $r-1$, so

$$n+1 \geq 2^{r-1} + 2^{r-1} = 2^r.$$

Upper bound. A rank-r pod has at most four child pods of rank $r-1$, so

$$n+1 \leq 4 \cdot 4^{r-1} = 4^r.$$

Consequently

$$h \leq 2r \leq 2\log_2(n+1) = O(\log n).$$

8.2.3 Insertion

Insertion has two stages:

1. perform ordinary BST insertion;
2. repair the red–black invariants.

The new internal node is colored red and receives rank 1; its two newly created external children are black of rank 0.

If the parent is black, nothing is broken. A red parent can create a red–red edge, which is why the second stage is needed. Notation:

$$p[x] = \text{parent}(x), \qquad p^2[x] = \text{grandparent}(x), \qquad u[x] = \text{uncle}(x).$$

Fix(x): upward recoloring phase

```
while p[x] ≠ nil,  p²[x] ≠ nil,
      color[p[x]] = color[u[x]] = red:
    x ← p²[x] // move to grandparent
    rank[x] ← rank[x] + 1
    color[x] ← red
    color[left[x]], color[right[x]] ← black
if p[x] = nil: // x is root
    color[x] ← black
    rank[x] ← rank[x] + 1
else if color[p[x]] = black:
    done
else:
    continue with the final rotation
```

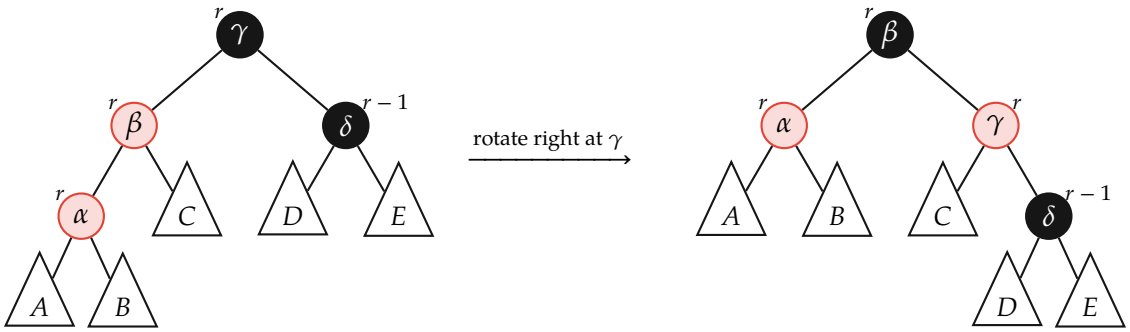
Note

The repair does not change keys: it changes only colors/ranks until the final rotation. The BST ordering is therefore preserved.

8.2.4 Insertion: final fix

When the parent and its sibling are both red, increment the rank of their parent and recolor both children black. If this reaches the root, the root becomes black. Otherwise a final constant-time rotation repairs the remaining red–red edge.

In the outside case, with red nodes α, β on the same side below γ, rotate at γ and recolor:



This is a right rotation at γ (with the left/right-symmetric case handled analogously). The inorder sequence

$$A, \alpha, B, \beta, C, \gamma, D, \delta, E$$

is unchanged, so the BST order is maintained.

In the inside, or zigzag, case, first rotate the parent to convert the shape to the outside case, then apply the same final fix. The total final-fix cost is $O(1)$.

8.2.5 Deletion strategies and repair cost

Two deletion strategies are:

1. ordinary BST deletion followed by DeleteFix;
2. lazy deletion: mark an element deleted and rebuild the entire tree when 50% of its nodes are marked.

Each rotation in Fix takes $O(1)$ time. Its loop moves x to its grandparent, so it iterates at most the tree height. Thus

$$T(\text{Fix}) = O(\log n).$$

8.2.6 Deletion repair

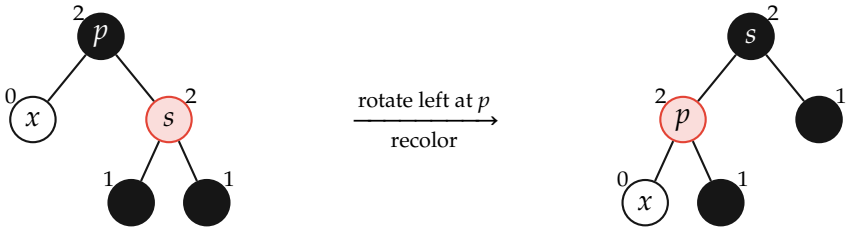
Reminder. Ordinary BST deletion physically removes either a leaf or a node with one child. When a node with two children is requested, its key is first exchanged with a predecessor/successor, reducing the physical removal to one of those two cases.

Where the repair is attached. When BST deletion replaces a node's key, a red–black tree keeps that node's rank and color unchanged; only the key is copied. The repair is therefore attached to the node that is *physically removed*. That node has at most one nonexternal subtree. In a red–black tree it is either a black node of rank 1 or a red node of rank 1.

DeleteFix(t, x): setup

```
if color[deleted node] = red:
  done // remove it directly
else:
  x ← left[deleted] or right[deleted]
  // x carries a one-black rank deficit
  while p[x] ≠ nil and the deficit remains:
    inspect sibling[x] and its children
```

Case 1: the sibling is red. Rotate at $p[x]$ toward x (left/right symmetrically), recolor the old parent red and the former sibling black, and keep x as the deficient subtree. This does not finish the repair; it transforms the configuration into one of the black-sibling cases.



8.2.7 Black-sibling cases

Case 2: the sibling and both of its children are black. Color the sibling red and set

$$x \leftarrow p[x],$$

which pushes the missing black rank upward. If x reaches the root, lower the root rank by one and finish. Otherwise repeat the loop.

Case 3: a far red child. Assume x is a left child and the right (far) child of its black sibling is red; the other orientation is symmetric. Rotate left at $p[x]$, preserve the old outside color by giving it to the new subtree root, make $p[x]$ black, and make the far child black. In the corresponding rank notation,

$$\begin{aligned} \text{color}(p^2[x]) &\leftarrow \text{color}(p[x]), \\ \text{color}(p[x]) &\leftarrow \text{black}, \\ \text{rank}(p[x]) &\leftarrow \text{rank}(x) + 1, \\ \text{rank}(p^2[x]) &\leftarrow r + 2, \\ \text{color}(u[x]) &\leftarrow \text{black}. \end{aligned}$$

From outside the repaired subtree, ranks and colors appear unchanged, while the internal deficit has disappeared.

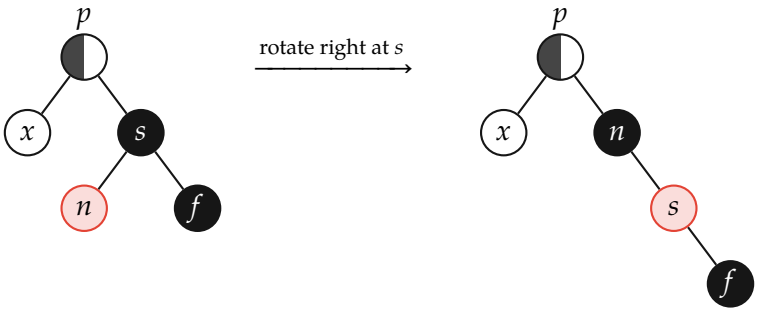
Note

The root case is reached only after repeatedly applying the recoloring case and pushing the deficit upward.

Near red child, far black child. Suppose x is a left child, the sibling is black, its left (near) child is red, and its right (far) child is black. Rotate right at the sibling and exchange the relevant colors. This converts the configuration into the preceding far-red-child case:

$$\text{sibling black, right[sibling] red.}$$

All left/right reflections are handled symmetrically.



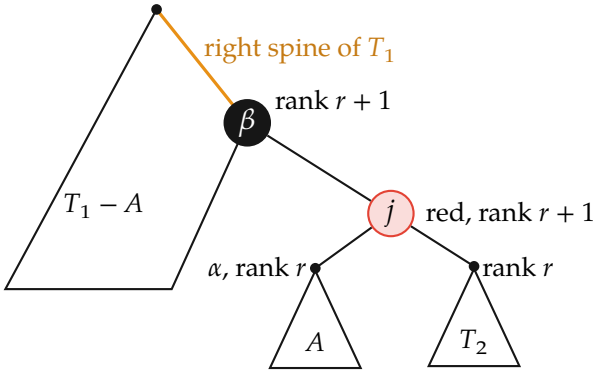
8.3 Joining and Splitting Red–Black Trees

8.3.1 Joining two trees

Suppose every key in T_1 is smaller than every key in T_2. To join them, remove an appropriate boundary key j (for instance the minimum of T_2), and attach the two trees around j. If the root rank of T_2 is r, locate on the right spine of T_1 a black node of rank r and splice in

$$\mathrm{Join}(A, j, T_2),$$

where A is the displaced subtree. The splice creates a red node of rank $r+1$, after which the standard fix procedure restores the invariants. In the figure, α is that black node of rank r, A the subtree it roots, and $\beta = p[\alpha]$ its parent on the spine.



8.3.1.1 Join algorithm

Join(T_1, T_2)

```
j ← FindMin(T2)  // O(log n)
Delete(j, T2)  // ordinary BST deletion
DeleteFix(T2, {j})  // O(log n)
B ← the repaired right tree
r ← rank[root[B]]
find on the right spine of T1 the black node α of rank r
β ← p[α];  A ← subtree of T1 rooted at α
color[j] ← red;  rank[j] ← r + 1
left[j] ← α;  right[j] ← root[B];  p[j] ← β
update all affected parent/child pointers
```

```
Fix(j)
```

The search, deletion repair, splice, and final repair all take $O(\log n)$, so

$$\text{Join}(T_1, T_2) = O(\log n), \qquad n = |T_1| + |T_2|.$$

One may also write $\text{Join}(T_1, x, T_2)$ when $T_1 < x < T_2$; then x itself is the joining key and there is no need to remove the minimum of T_2.

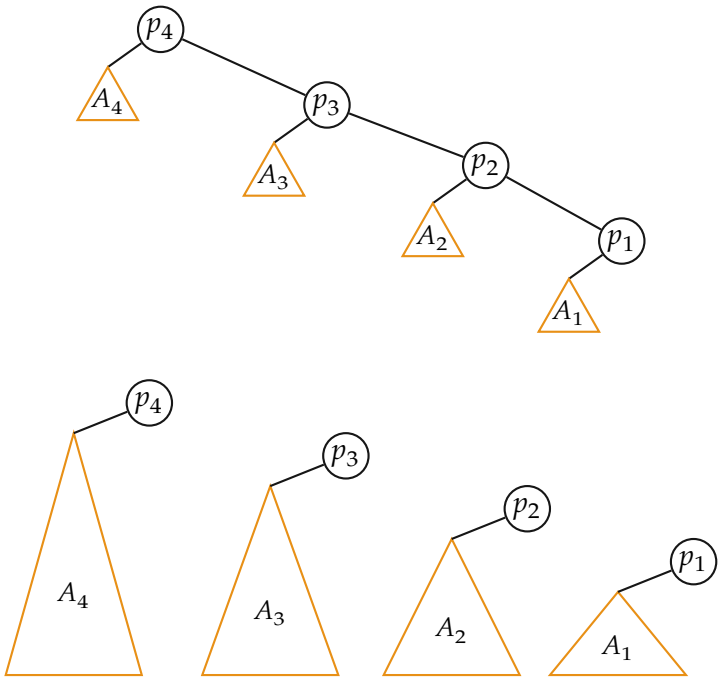
Note

The deletion repair used inside Join is not literally identical to the earlier insertion fix; it requires a different repair procedure.

8.3.2 Splitting a red–black tree

Split(T, k) separates T into the keys at most k and the keys larger than k.

1. Search for k: $x \leftarrow \text{Search}(T, k)$.
2. Travel from x to the root. Keep the path nodes $p_1, p_2, \ldots, p_\ell$ and the side subtrees $A_1, A_2, \ldots, A_\ell$ that peel away from the search path. On one side of the path, the pairs satisfy $\text{key}(p_i) \le k$; the other side is treated symmetrically.
3. Expose the actual forest, including the rank of every p_i and A_i, then reassemble it bottom-up with Join.



Step 3: the actual forest, drawn to show the ranks (heights decrease from A_4 to A_1)

8.3.2.1 Reassembling the split

One side can be rebuilt in the bottom-up order

Bottom-up reconstruction

```
Insert(p_1, A_1)
S ← Join(A_2, p_2, A_1)
S ← Join(A_3, p_3, S)
S ← Join(A_4, p_4, S)
return S
```

and symmetrically for the other side.

Theorem

Split takes $O(\log n)$ time.

The important point is the order of joining. The work paid while joining A_i is charged to the rank/height increase before the next join; these increments telescope along the original search path. Hence the total is

$$O(\text{total rank increase}) = O(\text{height}(T)) = O(\log n).$$

Wrong approach / correction

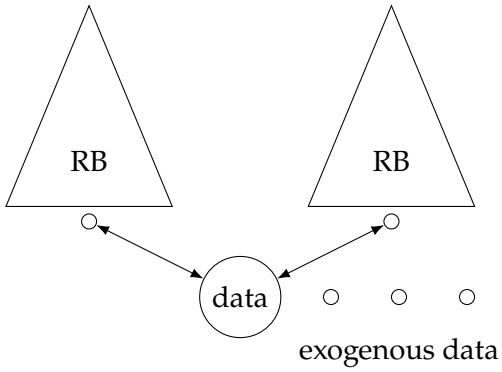
Joining a large side tree to the accumulating result in the wrong order can cost $O(\log n)$ at every one of $O(\log n)$ joins, yielding $O((\log n)^2)$. The bottom-up order above avoids this.

8.4 Augmented Data Structures

An augmented data structure stores needed information inside an existing data structure.

8.4.1 Two Red–Black Trees over Shared Data

Two red–black trees may be ordered with respect to different parameters while their nodes keep two-way references to the same exogenous data records.



8.4.2 The Order-Statistic ADT

The operations are

Search, Insert, Delete, Rank, Select.

- RANK(x) returns the order of x in the list: the smallest item has rank 1, and the largest has rank n.
- SELECT(k) returns the kth smallest item.

8.4.2.1 Order-Statistic Tree

Implement the ADT with a red–black tree. Each internal node stores

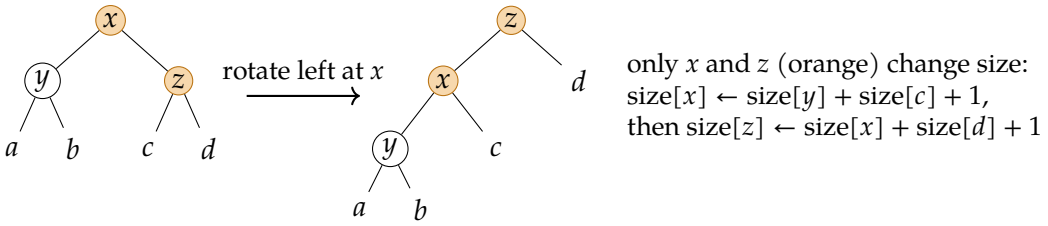
`parent`, `color`, `key`, `left`, `right`, and `size` = size of its subtree.

The subtree size includes the node itself and excludes external (nil) nodes.

Two issues must be handled:

1. maintain `size` during red–black-tree operations;
2. implement RANK and SELECT.

Maintaining sizes. During insertion or deletion, update the size by $+1$ or -1 along the search path; a newly inserted node has size 1. This costs $\Theta(\log n)$. During a rotation, recompute the sizes of the nodes whose subtrees change.



only x and z (orange) change size:
size[x] ← size[y] + size[c] + 1,
then size[z] ← size[x] + size[d] + 1

8.4.2.2 Select

For a subtree rooted at t, the rank of its root is

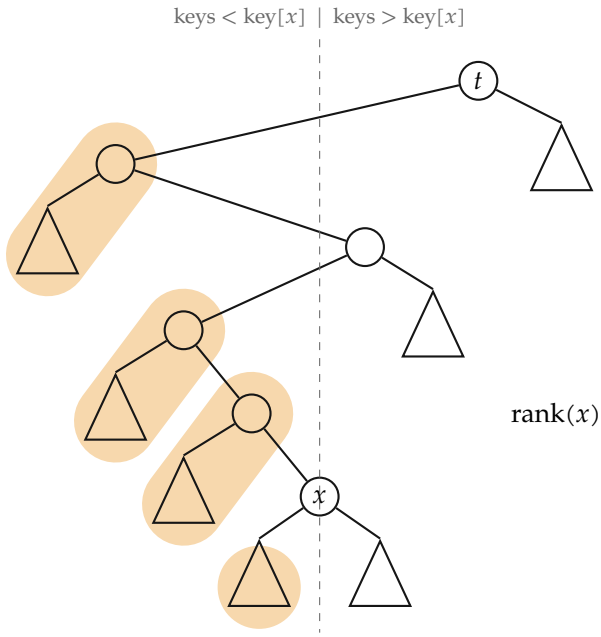
$$r = 1 + \text{size}(\text{left}[t]),$$

with $r = 1$ when the left child is nil.

```
SELECT(k,t):
  if t = nil, return ``gibberish'';
  if left[t] = nil, r ← 1;
  else r ← 1 + size(left[t]);
  if r > k, return SELECT(k,left[t]);
  if r = k, return t;
  if r < k, return SELECT(k − r,right[t]).
```

8.4.2.3 Rank

To compute the rank of a node x in the tree rooted at t, begin with the size of x's left subtree plus one. Whenever the upward path comes from a right child, add the parent's left-subtree size plus one.



$$\text{rank}(x) = 1 + \text{size}(\text{left}[x]) + \sum_{\text{highlighted } y} (1 + \text{size}(\text{left}[y]))$$

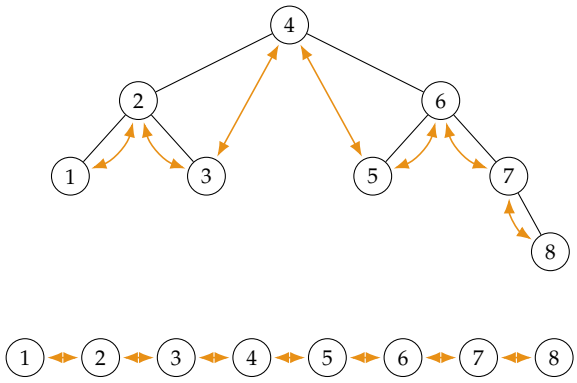
```
RANK(x,t):
  if left[x] = nil,  r ← 1;
  else r ← 1 + size(left[x]);
  y ← x;
  while y ≠ t:
    if right[parent[y]] = y:
      r ← r + 1 + size(left[parent[y]]);
    y ← parent[y];
  return r.
```

Example

Convince yourself that all these algorithms take $O(\log n)$ time.

8.4.3 Adding an Ordered List to a Tree

Augment every red–black-tree node with `prev` and `next` fields in addition to its usual parent, color, key, left, and right fields. These two pointers thread the nodes into a doubly linked list in key order.



Example

Write a program that maintains `next` and `prev` when a new node is inserted.

What happens during rotations? The key order does not change, so there is no need to fix the list pointers at all.

8.4.4 Lists as Trees

Recall list concatenation and splitting: for an ordinary linked list, one of these operations would be expensive. Instead, represent list positions $1, \dots, n$ by an augmented red–black tree storing subtree sizes.

The "key" is the node's position in the tree, **but the position is not stored**. To find a position, use SELECT from the order-statistic example; this takes $O(\log n)$ time. If list positions were stored explicitly, splitting would take $O(n)$ time because every affected node would need to be fixed. In contrast, the stored subtree sizes do not change merely because the list is split.

8.4.5 Interval Trees

An interval tree supports INSERT, DELETE, and OVERLAP for interval objects

$$I_i = [a_i, b_i] = [\text{low}(I_i), \text{high}(I_i)].$$

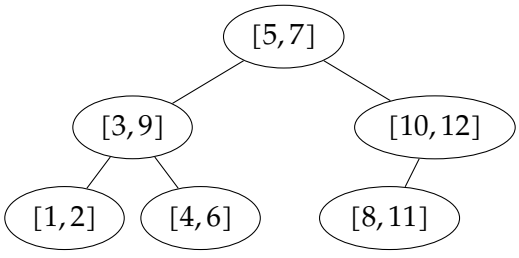
The question is: how can overlap be detected efficiently?

Implementation. Use a red–black tree whose key is the low endpoint. Each node stores

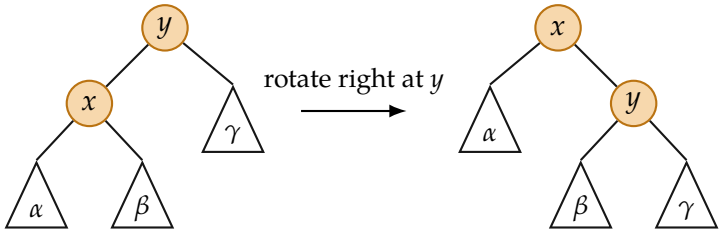
`low, high, max, color,`

where

$$\max[t] = \max\{\text{high}(I) : I \text{ is stored in the subtree of } t\}.$$



It is straightforward to maintain `max` during a rotation: only the nodes whose represented subtrees change need their maximum recomputed; all other nodes are unaffected.



only the two rotated nodes (orange) can change their `max`: recompute y first, then x

8.4.5.1 Overlap Query

```
OVERLAP(I, t):
  if t = nil, return false;
  if high[I] < low[t],
    return OVERLAP(I, left[t]);
  if I overlaps the interval stored at t, return true;
  if max[left[t]] ≥ low[I],
    return true;
  else return OVERLAP(I, right[t]).
```

When I lies to the right of the interval at t, the tempting rule

$$\underline{\text{return OVERLAP}(I, \text{right}[t])}$$

is wrong: an interval in the left subtree can extend far enough right to overlap I. If

$$\max[\text{left}[t]] \geq \text{low}[I],$$

such an overlap exists; otherwise search the right subtree.

Example

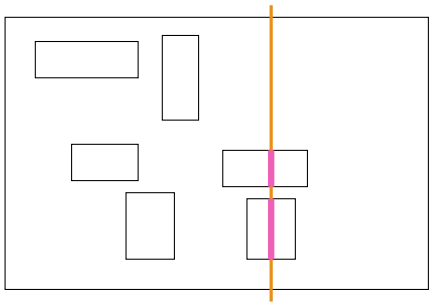
Convince yourself that the overlap query takes $\Theta(\log n)$ time.

8.4.6 Application: Sweep-Line Algorithm for Rectangle Overlap

Problem: given a layout of axis-aligned rectangles

$$[a_i, b_i] \times [c_i, d_i],$$

determine whether any two rectangles overlap.



active at time t: the two vertical intervals $[c_i, d_i]$ marked on the sweep line

At sweep time t, store the vertical intervals $[c_i, d_i]$ of all active rectangles. This reduces the two-dimensional problem to a one-dimensional one: if two rectangles overlap in the plane while both are active, their vertical intervals overlap.

Algorithm.

1. Sort the a_i values; sort the b_i values; initialize an empty interval tree T for vertical intervals.
2. Sweep from left to right through the birth and death events.
   - At a birth event a_i, if $\textsc{Overlap}([c_i, d_i], T)$ is true, halt and report "overlap"; otherwise insert $[c_i, d_i]$ into T.
   - At the death event b_i, delete $[c_i, d_i]$ from T.

One might worry about a configuration in which a birth line and a death line fail to scan an overlap. This possibility is eliminated by sorting the a_i and b_i events and processing them from left to right.

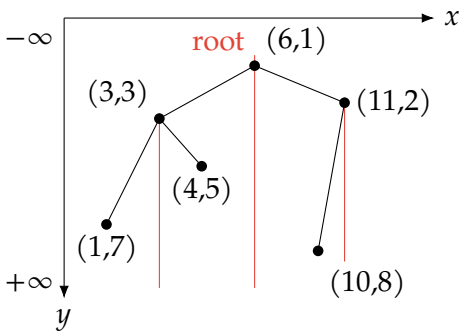
Chapter 9

Cartesian Trees, Treaps, and Random Binary Search Trees

9.1 Cartesian Trees

The data are distinct points

$$(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^2.$$



9.1.1 Construction Rules and Properties

1. The point with the smallest y-coordinate becomes the root.
2. Drop a vertical "rope" through the root, dividing the remaining points into left and right sets.
3. Repeat from Step 1 independently on the left and right sets.

A Cartesian tree is simultaneously

- a search tree with respect to the x-coordinate; and
- a min-heap with respect to the y-coordinate: the parent's y-value is smaller than the current node's.

9.1.2 Special Cases

1. **Ordinary BST:**

$$(x_1, 1), (x_2, 2), \dots, (x_n, n).$$

2. **Random BST:**

$$(x_{\sigma(1)}, 1), (x_{\sigma(2)}, 2), \dots, (x_{\sigma(n)}, n), \qquad \sigma \in S_n,$$

where σ is a permutation of the n elements. Equivalently,

$$(x_1, \tau(1)), (x_2, \tau(2)), \ldots, (x_n, \tau(n)), \qquad \tau \in S_n.$$

Randomization avoids the pathological ordered case in (1).

3. **Treap:**

$$(x_1, T_1), \ldots, (x_n, T_n),$$

where $T_1, \ldots, T_n$ are independent random numbers, for example generated uniformly from $[0, 1]$.

Benefit of a treap. When the next element arrives, there is no need to permute all elements again as in case (2); simply generate T_{n+1}.

9.1.3 Treap Operations

Each node stores x, y, parent, left, and right. A conventional BST insertion may leave a node v incorrectly positioned with respect to its y-coordinate.

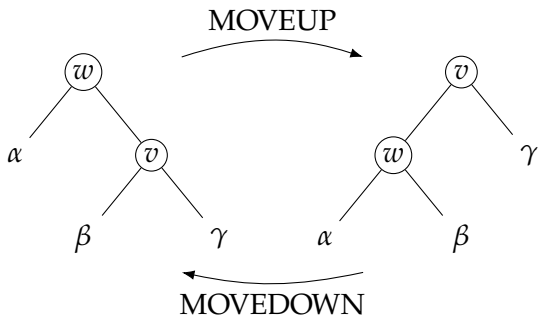
```
FIX(v,t):
  if parent[v] ≠ nil and
y[v] < y[parent[v]],
    FIXUP(v,t);
  else FIXDOWN(v,t).
```

```
FIXUP(v,t):
  while parent[v] ≠ nil and
y[parent[v]] > y[v]:
    MOVEUP(v)   // one rotation.
```

```
FIXDOWN(v,t):
  w ← n ∈ {v, left[v], right[v]}
such that y[n] is minimum among the three;
  while w ≠ v:   // otherwise v is fixed
    MOVEDOWN(v,w);
    recompute w as the minimum-y member of
{v, left[v], right[v]}.
```

For missing children, use priority $+\infty$.

Move-up / move-down rotation (left case).



The opposite rotation direction is symmetric.

Insertion.

```
INSERT(k,t,y):
  v ← the node produced by classical BST insertion of key k;
  x[v] ← k; y[v] ← y;
  FIX(v,t);
  return v.
```

Deletion. Deletion is much easier than in a red–black tree:

```
DELETE(v,t):
  y[v] ← +∞;
  FIX(v,t);   // v is now a leaf
  cut off the fixed node v at the leaf.
```

Join. For two treaps $t_1 < t_2$ (all keys in t_1 are smaller than all keys in t_2), introduce a temporary root above them and then delete that temporary root.

9.1.4 Join and Split

For $t_1 < t_2$, join the two treaps as follows:

```
JOIN(t1,t2):
  make a new node v;   // temporary fake root
  left[v] ← t1;
right[v] ← t2;
  t ← v;   // root
  DELETE(v,t).
```

To split at a key $k \notin t$, insert a temporary highest-priority node:

```
SPLIT(k,t):
  v ← INSERT(k,t,−∞);
  return left[v] and right[v].
```

The artificial node rises to the root, and its left and right subtrees are precisely the two parts.

9.1.5 Probabilistic Analysis of Treap Depth

Assume the nodes are labeled by x-values $1, \dots, n$. Let

$$D_k = \text{depth of node } k = \text{distance from the root},$$

and seek $\mathbb{E}[D_k]$ for $k \in \{1, \dots, n\}$.

Define indicator variables

$$X_{jk} = \begin{cases} 1, & j \text{ is an ancestor of } k, \\ 0, & \text{otherwise}, \end{cases} \qquad j \neq k.$$

Then

$$D_k = \sum_{j \neq k} X_{jk}$$

and, by linearity of expectation,

$$\mathbb{E}[D_k] = \sum_{j \neq k} \mathbb{E}[X_{jk}] = \sum_{j \neq k} \Pr(j \text{ is an ancestor of } k).$$

For $j < k$, the ancestor criterion is

$$j \text{ is an ancestor of } k \quad \Longleftrightarrow \quad y[j] = \max\{y[j], y[j+1], \ldots, y[k]\}.$$

Priorities outside the index interval from j to k do not affect this ancestor relation. There are $k - j + 1$ priorities from j through k. Any one of them is equally likely to be the relevant extremum, so

$$\begin{aligned}
\mathbb{E}[D_k] &= \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} \\
&= \left(\frac{1}{2} + \cdots + \frac{1}{k}\right) + \left(\frac{1}{2} + \cdots + \frac{1}{n-k+1}\right) \\
&\leq \ln k + \ln(n-k+1) \\
&\leq 2 \ln n = 1.38\ldots \log_2 n.
\end{aligned}$$

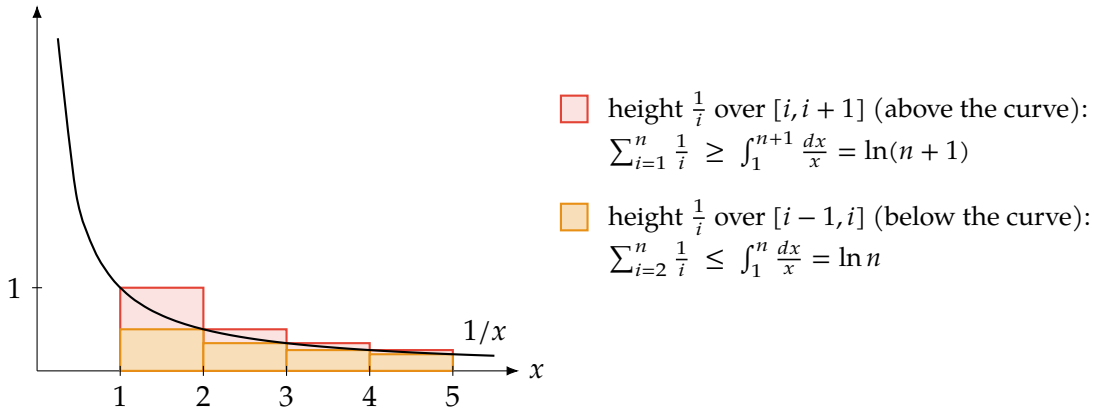
9.1.5.1 A Harmonic-Series Bound

The integral comparison used above gives

$$\ln(n+1) \leq 1 + \frac{1}{2} + \cdots + \frac{1}{n} \leq \ln n + 1,$$

because

$$\int_1^n \frac{dx}{x} = \ln n.$$

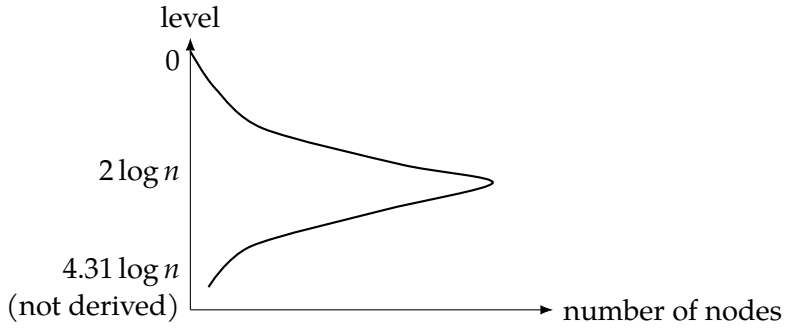


9.1.5.2 Average Treap Profile

The average number of nodes at each level has the profile sketched below. The average depth is on the order of $2 \log n$, with a lower tail around $4.31 \log n$; the latter value is not derived here.

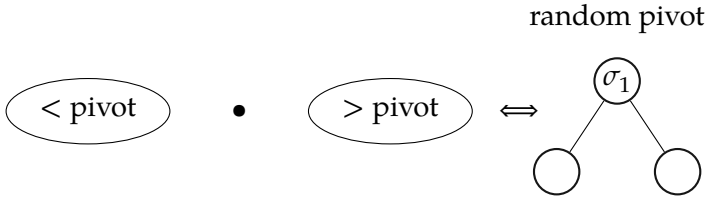
For the extreme node $k = 1$, there are no $j < k$, so

$$\mathbb{E}[D_1] = \sum_{j=2}^{n} \frac{1}{j} \leq \log n.$$



9.2 Quicksort and Random BSTs

Quicksort's recursive pivot partitions have the same topology as a random binary-search tree: the first random pivot is the root, and the recursive partitions become its subtrees.



Viewing a fixed item k as a witness, it is compared once with the pivot at each recursive level containing it. Thus it participates in D_k comparisons, where D_k is its depth in the corresponding random BST. Therefore

$$C_n := \#\text{ comparisons in quicksort} = \sum_{k=1}^{n} D_k,$$

the sum of all depths. Hence

$$\begin{aligned}\mathbb{E}[C_n] &= \sum_{k=1}^{n} \mathbb{E}[D_k] \\ &= 2(n+1)H_n - 4n \\ &\sim 2n \ln n = 1.38\ldots\, n \log_2 n.\end{aligned}$$

9.2.1 Pivot Improvements

- Pick three elements at random and use their median as the pivot:

$$\mathbb{E}[C_n] = \frac{12}{7} n \ln n.$$

- Pick five elements at random and use their median as the pivot:

$$\mathbb{E}[C_n] = \frac{60}{37} n \ln n.$$

Chapter 10

Priority Queues, Heaps, and Tournament Trees

10.1 The Priority-Queue ADT

The operations are Insert and DeleteMin; ~~Search~~ is not part of the stated interface.

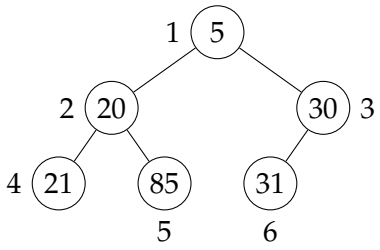
One use is operating-system CPU scheduling: jobs wait in a queue and may be served in round-robin order. Further uses of priority queues are:

2. heap sort;
3. construction of optimal trees – Dijkstra's algorithm, Prim–Dijkstra's algorithm, and Huffman trees all use priority queues;
4. discrete-event simulation.

Implementations include a binary heap or a tournament tree.

10.1.1 Binary Heaps

A binary heap is a complete binary tree. In a min-heap, keys increase along every path away from the root.



The actual representation is an array H with

$$\text{parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor, \qquad \text{left}(i) = 2i, \qquad \text{right}(i) = 2i + 1,$$

and current size heapsize$[H]$. The key at position i is $H[i]$.

10.1.1.1 Atomic Operations

SiftUp repairs a node that is too small relative to its parent; Heapify repairs a node that is too large relative to its children.

```
SIFTUP(i,H):
  while i ≠ 1:
    if H[⌊i/2⌋] > H[i]:
      swap H[i] and H[⌊i/2⌋];
      i ← ⌊i/2⌋;
    else break.
```

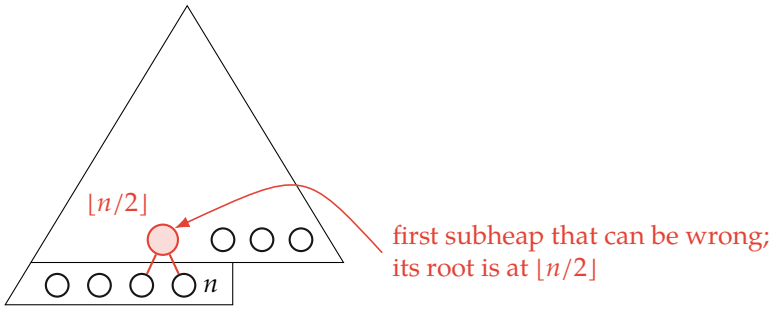
```
HEAPIFY(i,H):
  y ← the position whose key is
min{H[i],H[2i],H[2i+1]};
  // if a child does not exist, treat its key as +∞
  while y ≠ i:
    swap the keys at i and y;
    i ← y.
```

In Heapify, y denotes the node containing the minimum among the current node and its children.

10.2 Building and Updating a Heap

```
MakeHeap(H):
  n ← size(H)
  for i ← ⌊n/2⌋ downto 1:
    Heapify(i,H)
```

Why start at $\lfloor n/2 \rfloor$? It is the first root of a subheap that can be wrong. Every array position after it is a leaf, and the parent of the last node is at $\lfloor n/2 \rfloor$.



Cost of MakeHeap

At level i above the leaves, there are at most $n/2^i$ nodes, and a Heapify call costs at most $2i$ comparisons. The cost table is therefore

level	comparison cost
4	8 $(n/16)$
3	6 $(n/8)$
2	4 $(n/4)$
1	2 $(n/2)$
0	0

and

$$\#\text{comparisons} \leq \sum_{i=1}^{\infty} 2i\frac{n}{2^i} = 2n\sum_{i=1}^{\infty}\frac{i}{2^i} = 4n.$$

The last identity follows by expanding

$$\sum_{i=1}^{\infty}\frac{i}{2^i} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \cdots = 2.$$

Insertion and deletion

```
Insert(k,H):
  heapsize(H) ← heapsize(H)+1
  H[heapsize(H)] ← k
  SiftUp(heapsize(H),H)
```

$$\text{cost}(\textsc{Insert}) \leq \log_2 n.$$

```
DeleteMin(H):
  report/save H[1]
  H[1] ← H[heapsize(H)]
  heapsize(H) ← heapsize(H)−1
  Heapify(1,H)
```

The operation costs at most $2\log_2 n$ comparisons.

k-ary heaps

In a k-ary heap, an insertion follows one parent path, whereas deleting the minimum must find the smallest among k children at each level:

operation	cost
Insert	$\log_k n$
DeleteMin	$k\log_k n$

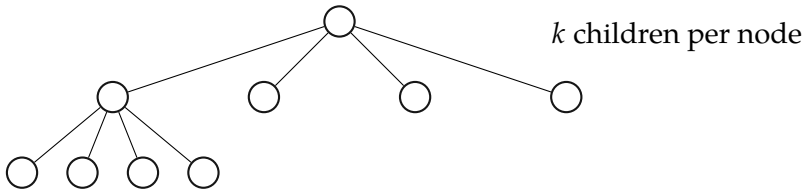
Thus the total cost of one insertion and one deletion is

$$(k+1)\log_k n.$$

Among integer arities, this is minimized at

$$\boxed{k = 4},$$

giving quaternary heaps.



10.3 HeapSort

```
HeapSort(H):
  MakeHeap(H) // at most 4n comparisons
  for i ← heapsize(H) downto 2:
    swap keys of items 1 and i
    heapsize(H) ← heapsize(H)-1
    Heapify(1,H)
```

The extracted minima accumulate in sorted order at the end of the array. The comparison count is

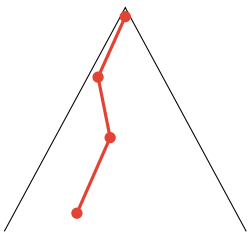
$$T_n \le 4n + \sum_{i=1}^{n} 2\log_2 i \sim 2n\log_2 n.$$

For comparison, the leading $n\log_2 n$ constants are

1 for MergeSort, 1.38 for average-case QuickSort.

Bottom-up HeapSort optimization

Each swap followed by heapification eventually identifies a downward path in the heap. This path supports the following bottom-up optimization.

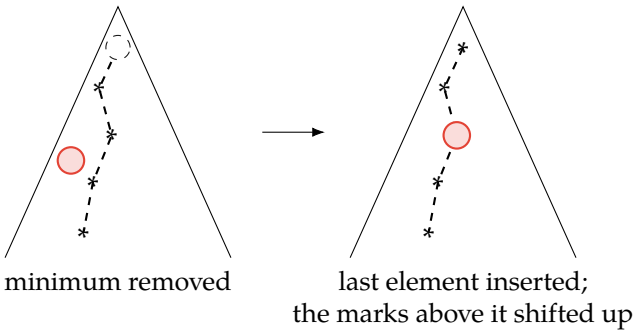


On the downward path, each marked node is the smaller child after comparing it with its sibling. This costs one comparison per level; ordinary heapification would use two comparisons to find the minimum among a parent and its two children.

After removing the minimum, the path moves up. Heapifying the final array element is now equivalent to inserting that element into the path and shifting the elements above it by one position. A binary search finds the insertion place along the path. The cost becomes

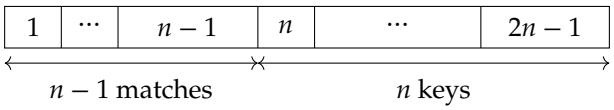
$$\underbrace{\log_2 n}_{\text{find the path}} + \underbrace{\log_2(\log_2 n)}_{\text{binary-search the path}},$$

reduced from $2\log_2 n$.



10.4 Tournament Trees

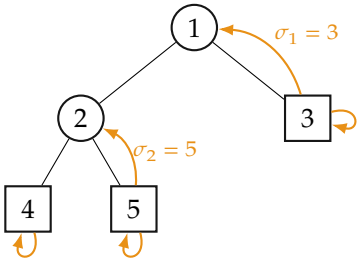
A tournament tree is stored in an array of size $2n-1$. Positions $1,\dots,n-1$ are matches (internal nodes), while positions $n,\dots,2n-1$ are players/keys (leaves). An array σ stores pointers to the winning player.



For a three-player example,

$$\sigma_4 = 4, \quad \sigma_5 = 5, \quad \sigma_3 = 3, \qquad \sigma_2 = 5, \quad \sigma_1 = 3,$$

where the orange entries are winners propagated upward.




```
MakeTree(n given keys):
  place the n keys in positions n,...,2n-1
  for i ← n to 2n − 1:
    σ_i ← i // match with itself
  for i ← n − 1 downto 1:
    if key[σ_2i] < key[σ_2i+1]:
      σ_i ← σ_2i // update winner
    else:
      σ_i ← σ_2i+1
```

Lazy deletion and winner updates

```
LazyDeleteMin:
  report key[σ_1]
  key[σ_1] ← ∞
  Update(σ_1, ∞)
```

This takes $\log_2 n$ comparisons.

```
Update(i,k): // player i, new key k
  while i ≠ 1:
    j ← sibling of i
      // if i is even, i is a left child; otherwise it is a right child
    if key[σ_j] < key[σ_i]:
      σ_⌊i/2⌋ ← σ_j
    else:
      σ_⌊i/2⌋ ← σ_i
    i ← ⌊i/2⌋
```

UPDATE assumes that the caller has already changed the player's key; the helper itself only propagates the stored winners toward the root.

Exercise

After practice, write a non-lazy deletion in $O(\log n)$ time.

Tournament-tree sorting therefore takes

$$T \le \underbrace{n-1}_{\text{MAKETREE}} + \underbrace{(n-1)\log_2 n}_{\text{successive lazy deletions}} .$$

Insertion

When the number of players grows from n to $n+1$, the old leaf at position n becomes an internal match. Move its player to position $2n$ and place the new player at $2n+1$:

```
Insert(k):
  key[2n] ← key[n]
  Update(2n+1, k)
```

Chapter 11

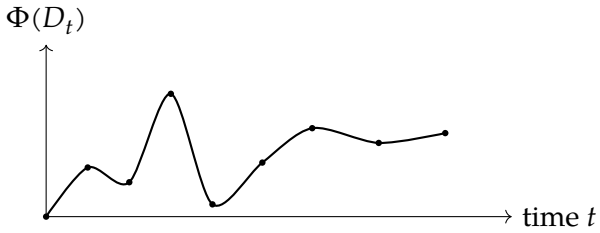
Amortized Analysis and Self-Adjusting Search Trees

11.1 Amortized Analysis

At time t, a data structure D_t has a nonnegative "score" or *potential*

$$\Phi(D_t) \geq 0,$$

which measures how far D_t is from an ideal state; $\Phi(D_t) = 0$ when it is ideal.



For the empty data structure,

$$\Phi(\emptyset) = 0.$$

Amortized cost by the potential method

If one operation changes the structure from D_t to $D_{t'}$, its amortized cost is

$$\begin{aligned}\hat{c} &= c + \Delta\Phi \\ &= c + \Phi(D_{t'}) - \Phi(D_t),\end{aligned}$$

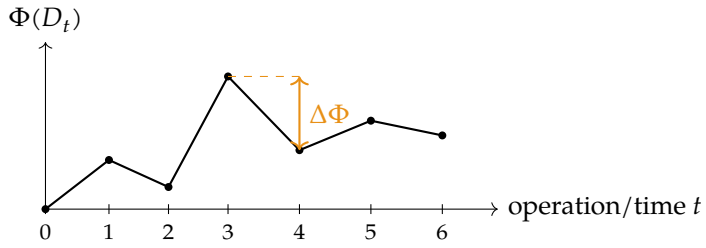
where c is the actual cost.

Over all operations up to time t, the potential changes telescope:

$$\begin{aligned}\sum \hat{c}_i &= \sum c_i + \sum \Delta\Phi_i \\ &= \sum c_i + \Phi(D_t) - \Phi(D_0).\end{aligned}$$

If the process starts with the empty/ideal structure, then $\Phi(D_0) = 0$, while $\Phi(D_t) \geq 0$. Therefore

$$\boxed{\text{total amortized cost} \geq \text{total actual cost}} \qquad \text{(starting at } t = 0\text{)}.$$



Note

We may choose the potential function in whichever way makes sense for the analysis.

Example: bitwise addition

```
x ← 0
for i ← 1 to n:
  x ← x + 1
```

An increment can propagate through a long suffix, as in

$$\cdots 101111 \ldots 111 + 1.$$

A potential based on the *minimum number of bits* is not useful. Instead, count the number of 1 bits in the current binary representation. For the binary counter, use

$$\Phi(x) = \#\{1 \text{ bits in } x\},$$

not the number of *trailing* 1s. The red counterexample is that adding one to a word ending in 0 can suddenly create a long suffix of ones:

$$111111110 + 1 = 111111111, \qquad \Delta\Phi = 9 \text{ for a trailing-ones potential: bad.}$$

The increment row is

$\Delta\Phi$	actual	amortized
$-k+1$	k	1

Example: stack with MultiPop

Let

$$\Phi(S) = |S|.$$

The operation k-Pop, also written MultiPop(k), removes k elements at once, or all remaining elements if fewer than k remain.

operation	actual	$\Delta\Phi$	amortized
Push	1	$+1$	2
k-Pop	$\min(k, \lvert S\rvert)$	$-\min(k, \lvert S\rvert)$	0

Thus any t operations have total cost at most $2t$.

Example: lazy deletion in a red-black tree

Deleted nodes remain as *ghosts*; when half the structure consists of ghosts, CleanUp rebuilds the red-black tree. The first attempted potential, one unit per ghost, does not pay for rebuilding:

~~potential = number of ghost elements~~.

The red correction is

$$\Phi = 2\,\#\text{ ghost elements}.$$

Operation	Actual	$\Delta\Phi$	Amortized
Search	$O(\log n)$	0	$O(\log n)$
Insert	$O(\log n)$	0	$O(\log n)$
LazyDelete	1	1 2	2 3
CleanUp	n	$-n/2$ $-n$	$n/2$ 0

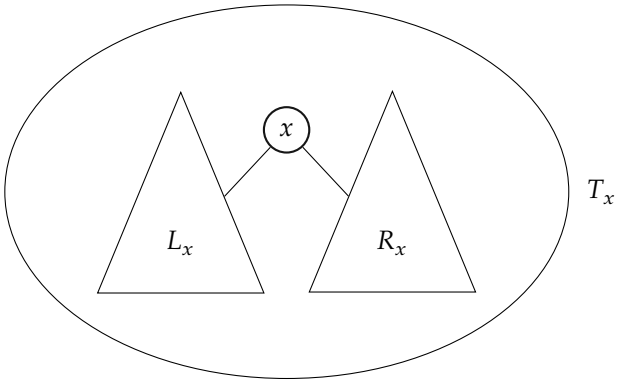
Exercise

Construct a red–black tree from a sorted list in $O(n)$ time.

11.2 Weight-Balanced Search Trees (Scapegoat Trees)

Store subtree sizes in every node. A BST is α-weight-balanced when, for every node x with left subtree L_x, right subtree R_x, and whole subtree T_x,

$$|L_x|, |R_x| \le \alpha |T_x|, \qquad \alpha \in \left[\frac{1}{2}, 1\right).$$



Along any downward path, the successive subtree sizes satisfy

$$n, \quad \le \alpha n, \quad \le \alpha^2 n, \quad \dots, \quad \le \alpha^k n.$$

The last subtree still contains at least one node, so $\alpha^k n \ge 1$. Hence

$$\left(\frac{1}{\alpha}\right)^k \le n, \qquad k \le \frac{\log n}{\log(1/\alpha)},$$

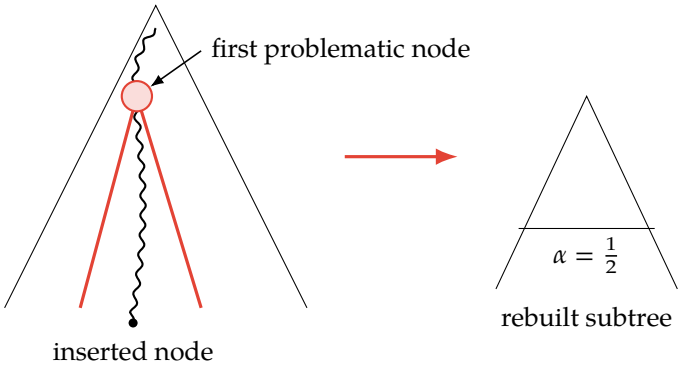
and the height obeys

$$h \le \frac{\log n}{\log(1/\alpha)}.$$

Restoring balance after a BST update

Classical BST insertion or deletion can break the weight-balance property. The solution is a cleanup/rebuild.

Step 1. During insertion, walk upward along the insertion path and find the first node whose balance property is broken. This is the first problematic node, or *scapegoat*.
Step 2. Take the entire subtree rooted at the scapegoat, including the newly inserted node.
Step 3. Its in-order sequence is already a correct BST order. Rebuild it as a perfectly balanced scapegoat tree, equivalently one with $\alpha = 1/2$, in $O(n)$ time.
Step 4. Put the rebuilt subtree back in its original position.



Potential analysis

When $\alpha = 1/2$, perfect balance gives

$$|L| \le \frac{1}{2}(|L| + |R| + 1) \quad \Longrightarrow \quad |L| - |R| \le 1.$$

Symmetrically $|R| - |L| \le 1$, so

$$||L| - |R|| \le 1.$$

Motivated by this, define the unscaled potential

$$\Phi := \sum_{\substack{\text{nodes } x \\ ||L_x| - |R_x|| \ge 2}} ||L_x| - |R_x||.$$

A perfectly balanced tree has potential 0.

At a scapegoat x,

$$\Phi \ge ||L_x| - |R_x||.$$

Without loss of generality, suppose the left subtree is the one breaking the property:

$$|L_x| > \alpha |T_x|.$$

Then

$$|R_x| \le |T_x| - \alpha |T_x| - 1 \le (1 - \alpha)|T_x|,$$

and hence

$$\Phi \ge |L_x| - |R_x| > (2\alpha - 1)|T_x|.$$

Before rescaling the potential, the cost table is

Operation	Actual	$\Delta\Phi$	Amortized
Classical Insert	$O(\log n)$	–	–
Classical Delete	$O(\log n)$	–	–
CleanUp	$\lvert T_x\rvert$	$\leq -(2\alpha-1)\lvert T_x\rvert$	–
Search	$O(\log n)$	0	$O(\log n)$

We want the amortized cleanup cost to be 0. For $\alpha > 1/2$, the red correction rescales the potential:

$$\Phi := \frac{1}{2\alpha-1} \sum_{\substack{\text{nodes } x \\ ||L_x|-|R_x||\geq 2}} ||L_x| - |R_x||.$$

With the rescaled potential, the updated table is

Operation	Actual	$\Delta\Phi$	Amortized
Classical Insert	$O(\log n)$	$\leq h = O(\log n)$	$O(\log n)$
Classical Delete	$O(\log n)$	$\leq h = O(\log n)$	$O(\log n)$
CleanUp	$\lvert T_x\rvert$	$\leq -\lvert T_x\rvert$	≤ 0
Search	$O(\log n)$	0	$O(\log n)$

For insertion or deletion, the potential changes by at most the height: only nodes on the path from the root to the inserted/deleted node are affected, and each imbalance

$$||L_x| - |R_x||$$

changes by at most 1.

Amortized bound for scapegoat trees

Starting from an empty tree, or from a perfectly balanced tree with initial potential 0, any sequence of m operations has

$$T(m \text{ actual operations}) \leq T_{\text{amortized}} \leq c\, m \log n,$$

where n is the maximum size of the tree during the sequence.

11.3 Splay Trees

A splay tree is an ordinary, possibly unbalanced BST with $\Theta(\log n)$ amortized time per operation. Every classical BST insertion, deletion, or search is followed by a *splay step*.

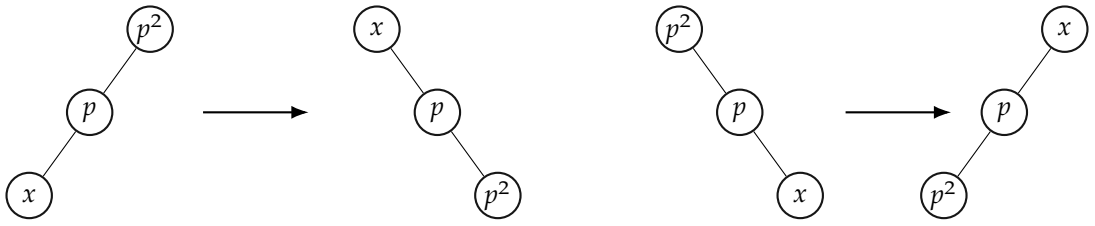
The active node x is

- the searched or inserted node; or
- the parent of a deleted node.

For further reading, see Tarjan's book on data structures.

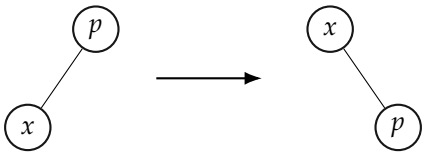
Splay rotations

Rotation 1: two levels up. When x, its parent p, and its grandparent p^2 lie in the same direction, perform the corresponding zig-zig double rotation. The mirror-image right-right case is identical.



left-left zig-zig right-right zig-zig

Rotation 2: one level up. If $p^2(x) = \text{nil}$, rotate x once above its parent p:



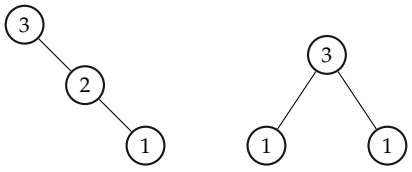
Potential for splay trees

Let $T(y)$ be the subtree rooted at y. Define

$$\Phi := \sum_y \log_2 |T(y)| = \log_2\left(\prod_y |T(y)|\right).$$

This potential is smaller for a more balanced tree. For example,

three-node chain: $\Phi = \log_2(3 \cdot 2 \cdot 1) = \log_2 6$, balanced tree: $\Phi = \log_2(3 \cdot 1 \cdot 1) = \log_2 3$.



Let

$\Delta =$ the distance from the active node x to the root, $n =$ the tree size before the operation.

The actual splay work is proportional to Δ.

Potential change of a classical insertion

Suppose the insertion path contains old subtree sizes S_1, S_2, S_3 and introduces a fourth node. After insertion, these sizes are $S_i + 1$. Therefore

$$\begin{aligned}\Phi(\text{after}) - \Phi(\text{before}) &= \log_2(S_1+1) + \log_2(S_2+1) + \log_2(S_3+1) + \log_2(S_4+1) \\ &\quad - \log_2 S_1 - \log_2 S_2 - \log_2 S_3 \\ &= \log_2(S_1+1) + (\log_2(S_2+1) - \log_2 S_1) \\ &\quad + (\log_2(S_3+1) - \log_2 S_2) + (\log_2(S_4+1) - \log_2 S_3) \\ &\le \log_2(S_1+1) \le \log_2(n+1),\end{aligned}$$

because $S_i \ge S_{i+1} + 1$. Delete and search are treated similarly. The corresponding classical-operation contributions are

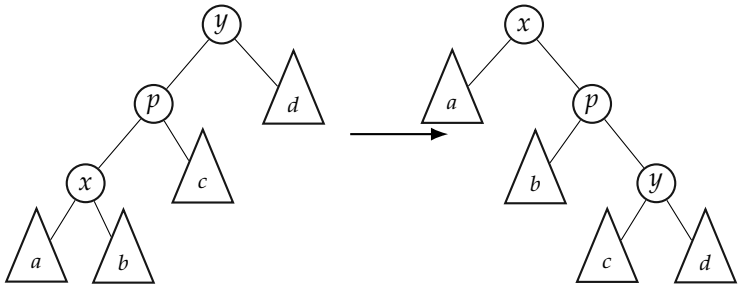
Operation	Actual splay distance	Classical $\Delta\Phi$
Insert + Splay	Δ	$\leq \log_2(n+1)$
Delete + Splay	Δ	≤ 0
Search + Splay	Δ	0

Potential change of a two-level splay

Use the arithmetic-geometric mean inequality

$$\frac{a+b}{2} \geq \sqrt{ab}.$$

For a left-left zig-zig step, let a, b, c, d be the sizes of the four hanging subtrees:



Only x, p, y change subtree size. Directly expanding the potential gives

$$\begin{aligned}\Delta\Phi &= \log_2(3+a+b+c+d) + \log_2(2+b+c+d) + \log_2(1+c+d) \\ &\quad - \log_2(3+a+b+c+d) - \log_2(2+a+b+c) - \log_2(1+a+b) \\ &= \log_2(2+b+c+d) + [\log_2(1+c+d) + \log_2(1+a+b)] \\ &\quad - [\log_2(2+a+b+c) + \log_2(1+a+b) + \log_2(1+a+b)].\end{aligned}$$

By the arithmetic-geometric mean inequality,

$$\begin{aligned}\Delta\Phi &\leq \log_2(2+a+b+c+d) + \log_2\left(\frac{a+b+c+d+2}{2}\right)^2 - 3\log_2(1+a+b) \\ &\leq 3\log_2(a+b+c+d+3) - 3\log_2(1+a+b) - 2 \\ &= 3\log_2|T(y)| - 3\log_2|T(x)| - 2.\end{aligned}$$

As the active node rises, the two-level bounds telescope. For example, grouping the path through $x, p^2(x), p^4(x), \ldots$ gives

$$\begin{aligned}\Delta\Phi &\leq (3\log_2|T(p^4(x))| - 3\log_2|T(p^2(x))| - 2) \\ &\quad + (3\log_2|T(p^2(x))| - 3\log_2|T(x)| - 2) + \cdots \\ &\leq 3\log_2|T(\text{root})| - 3\log_2|T(x)| - 2\cdot\frac{\Delta}{2} \\ &\leq 3\log_2 n - \Delta.\end{aligned}$$

The factor $\Delta/2$ counts the number of two-level splay steps.

Since the actual splay cost is Δ,

$$\Delta + \Delta\Phi \leq 3\log_2 n.$$

Combining this access bound with the classical-operation potential changes completes the table:

Operation	Actual	Total $\Delta\Phi$	Amortized
Insert + Splay	Δ	$\leq \log_2(n+1) + 3\log_2 n - \Delta$	$\leq 4\log_2(n+1)$
Delete + Splay	Δ	$\leq 3\log_2 n - \Delta$	$\leq 3\log_2 n$
Search + Splay	Δ	$\leq 3\log_2 n - \Delta$	$\leq 3\log_2 n$

Chapter 12

String Data Structures and Pattern Matching

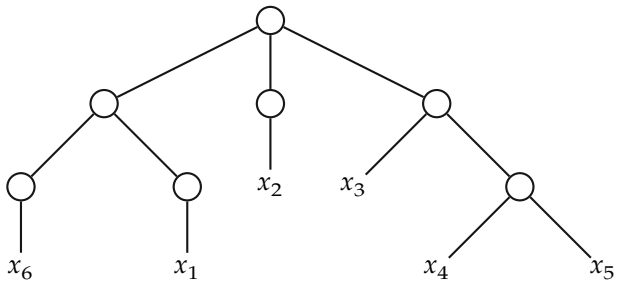
We consider strings $x_1, \dots, x_n$, where an x_i may be finite or infinite. Example alphabets include

$$\{0,1\}, \qquad \{A,C,G,T\}, \qquad \{0,\dots,9\}, \dots$$

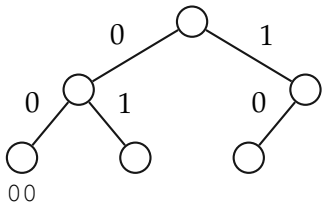
12.1 Tries

Definition

A *trie* is a k-ary position tree, where $|\text{Alphabet}| = k$. Strings correspond to paths, so the structure supports retrieval by following the symbols of a string.

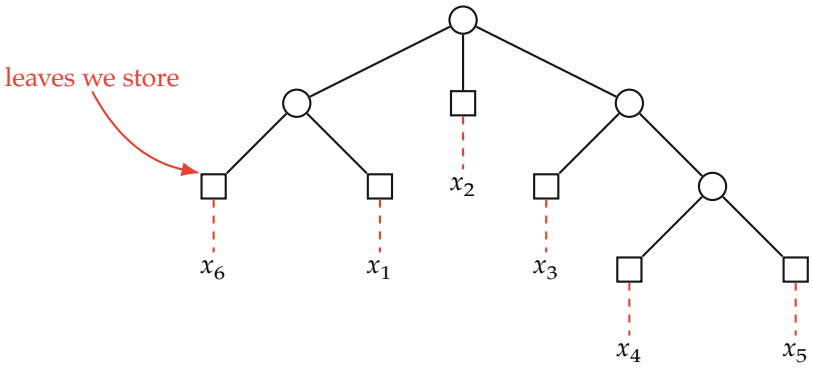


Recall the binary-tree case: a bit string describes a position, with a 0-edge going left and a 1-edge going right. A trie applies exactly the same idea to an arbitrary alphabet.



Finite representation. Do not store infinite strings explicitly. Store leaves that correspond bijectively to the strings. Red dashed tails indicate the omitted continuation beyond a stored leaf:

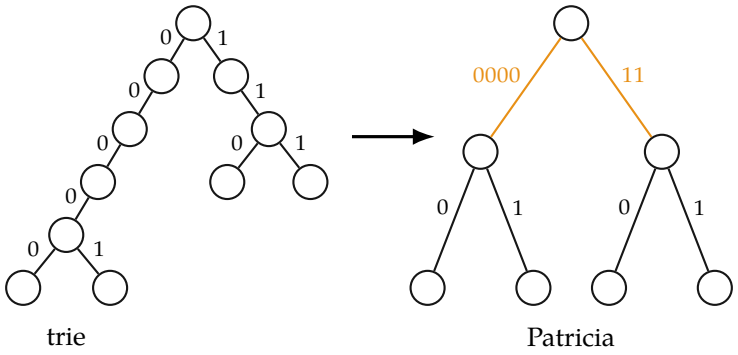
stored leaf $\dashrightarrow$ the remaining (possibly infinite) suffix.



Space question. Given n stored leaves, how much space is required?

12.2 Patricia Trees

A Patricia tree is a modification of a trie that compresses every maximal one-child path into a single labelled edge.



Patricia-tree space

For a binary Patricia tree with n leaves, every internal node has two children. Hence there are $n-1$ internal nodes and

$$n + (n-1) = 2n - 1$$

nodes in total.

Remark: the k-ary case. Let n_i be the number of nodes of degree i. Then

$$\#\text{nodes} = \sum_{i=0}^{\infty} n_i$$

and

$$\begin{aligned} \#\text{nodes} - 1 = \#\text{edges} &= 2n_2 + 3n_3 + 4n_4 + \cdots \\ &\geq 2(n_2 + n_3 + \cdots) = 2(\#\text{nodes} - n_0). \end{aligned}$$

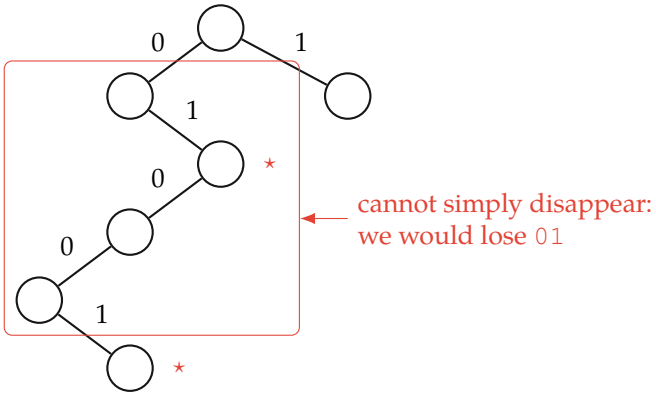
Therefore

$$\#\text{nodes} \leq 2n_0 - 1,$$

where n_0 is the number of leaves.

When one stored string is a prefix of another

Consider storing `01` and `01001`. The node reached after `01` is both the end of one stored string and an internal point on the path to the other:



Solution. Add a bit at a node indicating that a stored string ends there. Compression may then keep a marked endpoint while compressing the remaining path.

Exercise

Show that a Patricia representation of n strings, when prefixes are allowed, uses at most $2n$ nodes.

Representing the children of a trie node

1. Store an array of $k = |\text{Alphabet}|$ child pointers. Many entries may be nil.
2. **de la Briandais representation (better):** store a linked list of the existing children, avoiding nil pointers. The list has length at most k.

12.3 Suffix Tries and Suffix Trees

For input text $T[1, \ldots, n]$, form all suffixes

$$\begin{aligned} x_1 &= T[1 \ldots n], \\ x_2 &= T[2 \ldots n], \\ &\vdots \\ x_n &= T[n]. \end{aligned}$$

Store these n strings in a trie. A *suffix tree* is the Patricia version of this suffix trie, so its space is at most $2n$ nodes.

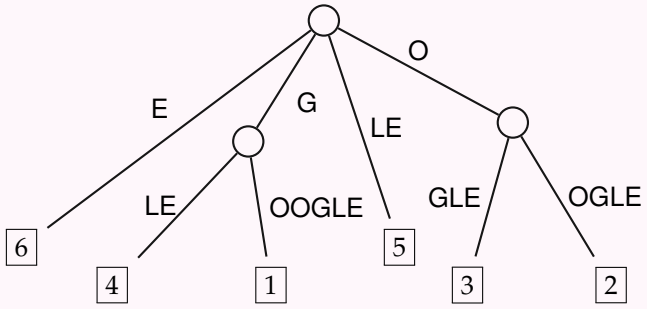
Suffix tree for $T =$ GOOGLE

The positions are

i	1	2	3	4	5	6
$T[i]$	G	O	O	G	L	E

and the six suffixes are

GOOGLE, OOGLE, OGLE, GLE, LE, E.



leaf	6	4	1	5	3	2
stored interval	[6,6]	[5,6]	[2,6]	[5,6]	[4,6]	[3,6]

For every compressed edge, store only an interval $[a, b]$ into the original text: a is the starting letter index and b is the ending letter index.

Construction and suffix arrays

$$\boxed{O(n) \text{ time}}$$

Ukkonen's algorithm constructs the Patricia suffix tree in linear time.

Now order the alphabet (for example, $A < \cdots < Z$) and order the children of every suffix-tree node accordingly. For a string of length n, the set of positions $\{1, \ldots, n\}$ is in bijection with the suffix strings

$$T[1 \ldots n],\ T[2 \ldots n], \ldots, T[n].$$

In an ordered suffix tree, a preorder traversal visits the suffix strings in lexicographic order. Thus an ordering of the suffix strings gives a permutation of $\{1, \ldots, n\}$.

Definition

The *suffix array* is the permutation of $\{1, \ldots, n\}$ obtained by listing the suffix starting positions in lexicographic order. It uses $O(n)$ space.

For $T =$ GOOGLE,

starting position	suffix
6	E
4	GLE
1	GOOGLE
5	LE
3	OGLE
2	OOGLE

$\Longrightarrow$ [6, 4, 1, 5, 3, 2].

Preorder traversal = lexicographic order.

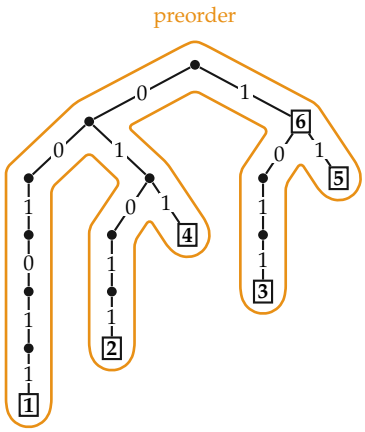
Binary suffix-array example and search

For $T =$ `001011`, the suffixes in lexicographic order are

1	`001011`
2	`01011`
4	`011`
6	`1`
3	`1011`
5	`11`

$\Longrightarrow \quad [\,1,\ 2,\ 4,\ 6,\ 3,\ 5\,].$

The same array is read off the suffix trie of T: a preorder traversal that visits 0-children before 1-children (the orange tour) meets the leaves in lexicographic order.



Operation: search. Binary-search the suffix array according to lexicographic order. This solves efficient search for a string S in a text T when T has already been preprocessed and stored as a suffix tree/array.

12.4 Pattern Matching When Neither Input Is Preprocessed

The next problem is also pattern matching, but now neither the text nor the pattern is known beforehand:

$$T[1\ldots n] \quad \text{(text)}, \qquad P[1\ldots m] \quad \text{(pattern)}, \qquad m \ll n.$$

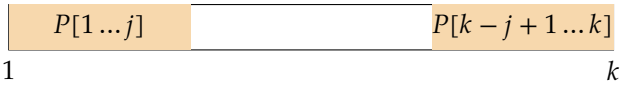
The algorithm has two steps.

Step 1: construct the magic table

For $k = 1, \ldots, m$, define

$$M_k := \begin{cases} \max\{\, 1 \le j < k \mid P[1\ldots j] = P[k-j+1\ldots k] \,\}, & \text{if such a } j \text{ exists,} \\ 0, & \text{otherwise.} \end{cases}$$

Thus M_k is the length of the longest proper prefix of $P[1\ldots k]$ that is also a suffix ending at k.



Magic-table example and program

The highlighted prefix/suffix example is

$$P = \texttt{0011010101001001},$$

with

k	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
M_k	0	1	0	0	1	0	1	0	1	0	1	2	3	1	2	3

For example, at $k = 13$ the highlighted length-3 prefix `001` matches the length-3 suffix ending at position 13.

BuildMagicTable(P, M)

```
j ← 0;  k ← 1;  M_1 ← 0
while k < m:
  if P[j + 1] = P[k + 1]:
    M_{k+1} ← j + 1
    j ← j + 1;  k ← k + 1
  else if j > 0:
    j ← M_j
  else:                                          // j = 0
    M_{k+1} ← 0
    k ← k + 1
```

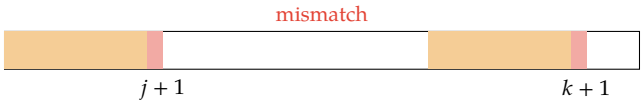
Only the middle `else if` is nontrivial. Throughout all three branches, the invariant is

$$P[1 \ldots j] = P[k - j + 1 \ldots k]. \tag{*}$$

After a mismatch $P[j + 1] \neq P[k + 1]$, suppose some other j' could extend:

$$P[1 \ldots j' + 1] = P[k - j' + 1 \ldots k + 1].$$

Removing the unequal final characters yields a shorter proper prefix/suffix relation, which leads to the fallback rule below.



Why the magic-table fallback works

The hypothetical match of length j' implies

$$\begin{aligned} P[1 \ldots j'] &= P[k - j' + 1 \ldots k] \\ &= P[j - j' + 1 \ldots j] \qquad \text{by } (*). \end{aligned}$$

Therefore $j' \leq M_j$ by the definition of M_j, and $P[1 \ldots j']$ is a prefix substring of $P[1 \ldots M_j]$.

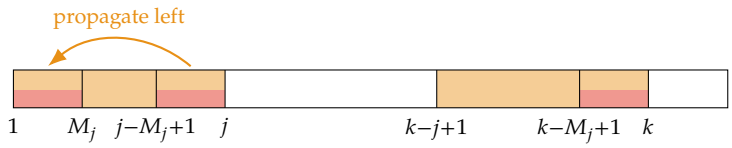
Moreover,

$$\begin{aligned} P[1 \ldots M_j] &= P[j - M_j + 1 \ldots j] \\ &= P[k - M_j + 1 \ldots k] \qquad \text{by } (*). \end{aligned} \tag{†}$$

Setting

$$\boxed{j \leftarrow M_j}$$

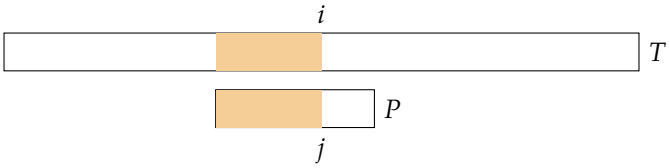
propagates this relation leftward and maintains the invariant.



At the next iteration, if $P[j+1] = P[k+1]$, the longest match has been found. Otherwise the same argument again gives a candidate of length at most M_j, and the algorithm falls back again. Eventually it either finds the required j' or reaches the case $j = 0$, so the procedure terminates.

Step 2: Knuth–Morris–Pratt

Use two moving pointers: i in the text T and j in the pattern P.



KMP fallback after a mismatch

Suppose the verified match is

$$P[1 \dots j-1] = T[i-j+1 \dots i-1].$$

If $P[j] = T[i]$, advance both i and j after one check. Otherwise, advance the pattern efficiently so that the algorithm does not become $O(mn)$.

By the definition of M_{j-1},

$$P[j-1-M_{j-1}+1 \dots j-1] = P[1 \dots M_{j-1}],$$

while the verified match gives

$$P[1 \dots j-1] = T[i-j+1 \dots i-1].$$

Consequently,

$$P[1 \dots M_{j-1}] = T[i-M_{j-1} \dots i-1].$$

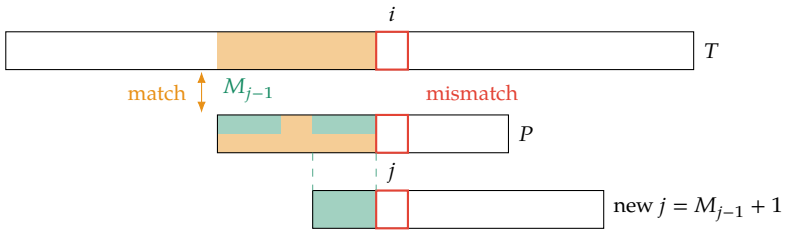
Thus, by moving

$$\boxed{j \leftarrow M_{j-1}+1},$$

we preserve the already matched prefix

$$P[1 \dots j-1] = T[i-j+1 \dots i-1]$$

and then test $P[j] = T[i]$ again.



By maximality of M_{j-1}, no intermediate shift between $M_{j-1}+1$ and the previous j could preserve a longer match; otherwise it would contradict the definition of M_{j-1}.

KMP-Algorithm(T, P, M)

```
i ← 1; j ← 1                                   // M is the magic table from step 1
while i ≤ n:
  if T[i] = P[j]:
    i ← i + 1; j ← j + 1
  else if j > 1:
    j ← M_{j-1} + 1
  else:                                                          // j = 1
    i ← i + 1
  if j = m + 1:
    return i − m and halt
return no match
```

KMP complexity

Complexity of BuildMagicTable(P, M)**.** At every iteration, either k increases or $k - j$ increases when $j \leftarrow M_j$. The first event occurs at most m times because $1 \leq k \leq m$. Also, $k - j$ starts at 0 and satisfies $k - j \leq k \leq m$. Therefore

$$T_m \leq 2m = O(m).$$

Complexity of KMP(T, P, M)**.** At every iteration, either i increases or $i - j$ increases. The pointer i starts at 1 and ends at n, while $i - j$ starts at 0 and

$$i \leq n, \quad j \geq 1 \quad \Longrightarrow \quad i - j \leq n - 1.$$

Therefore

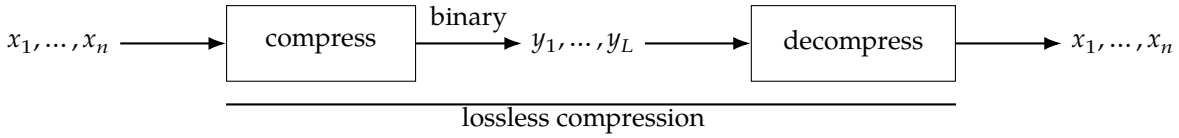
$$T_{n,m} \leq n + n = 2n = O(n).$$

Chapter 13

Information Theory, Huffman Coding, and Lempel–Ziv

13.1 Information theory and data compression

13.1.1 Lossless and lossy compression



Remark. Compression may also be lossy; converting images to JPEG is one example.

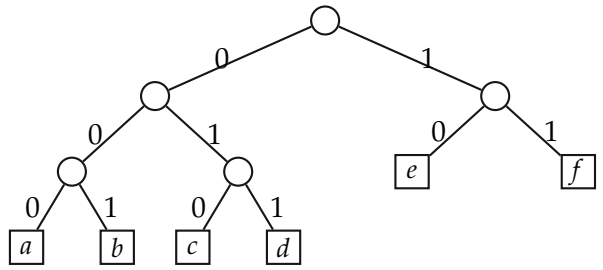
History before 1977.

- Standard coding maps one symbol to a sequence of bits.
- Fixed-width coding: ASCII uses 8 bits per symbol.
- Variable-width coding: Morse code needs a separator (and is therefore effectively ternary in this account); Braille is another example for blind readers.

13.1.2 Codes viewed as binary trees

In Braille, one symbol uses six binary positions, so two symbols use twelve. For a binary tree code, symbols correspond to leaves; this avoids the separation problem of Morse code. A codeword is a root-to-leaf path:

$$0 = \text{go left}, \qquad 1 = \text{go right}.$$



symbol	codeword
a	000
b	001
c	010
d	011
e	10
f	11

Concerns.

1. Space complexity.
2. The coder needs only parent pointers in the tree.
3. The decoder uses the tree to solve the separation problem.

For example, the stream

$$\texttt{00000100000110} = \texttt{000}\,|\,\texttt{001}\,|\,\texttt{000}\,|\,\texttt{001}\,|\,\texttt{10}$$

is decoded by:

1. starting at the root;
2. walking according to the next bit;
3. continuing until a leaf is reached, outputting its symbol, and restarting at the root.

13.1.3 Optimizing expected codeword length

Let p_i be the probability/proportion of symbol i in the input, with

$$\sum_i p_i = 1.$$

The average codeword length is

$$\sum_{\text{leaf } i} p_i \ell_i,$$

where ℓ_i is the length of the ith symbol's codeword.

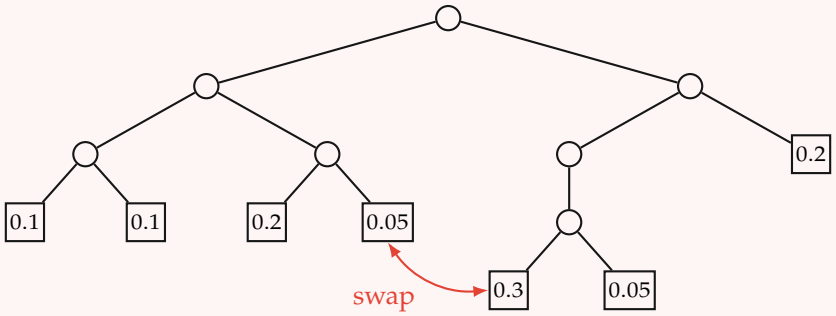
The Huffman objective

The optimization problem is

$$\text{minimize} \qquad \sum_i p_i \ell_i.$$

A non-optimal placement

The tree below places a symbol of probability 0.30 deeper than one of probability 0.05 (it also contains a one-child node). Exchanging the two leaves joined by the red swap arrow shortens the more probable symbol and lengthens the less probable one, improving the expected length.



Observations for optimality.

1. There should be no one-child nodes.
2. The two smallest probabilities can be made siblings, furthest from the root. This statement is not used directly as an algorithmic principle.

Closer to the root means a smaller ℓ_i; farther from the root means a larger ℓ_i.

Greedy merging example

symbol	a	b	c	d	e
p_i	0.10	0.15	0.20	0.25	0.30

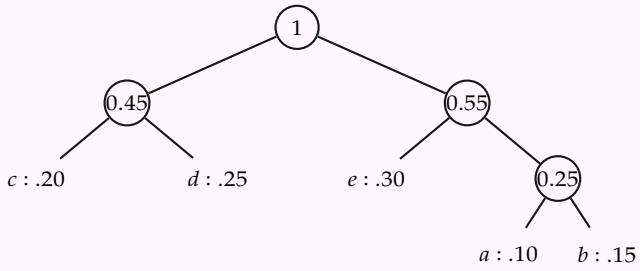
Repeatedly take the two smallest current weights and make them siblings:

$$0.10,\ 0.15 \longrightarrow 0.25,$$

$$0.20,\ 0.25 \longrightarrow 0.45,$$

$$0.25,\ 0.30 \longrightarrow 0.55,$$

$$0.45 + 0.55 = 1.$$



All that is done at each step is:

1. find the two smallest weights;
2. make them siblings.

13.2 Huffman Coding

13.2.1 Huffman Trees and the Greedy Algorithm

Definition

The tree constructed above is called a *Huffman tree*. Its construction algorithm is a *greedy algorithm*.

Divide-and-conquer, dynamic programming, and greedy

I would like to explain a bit the difference between the three large class of algorithms we have seen so far: divide-and-conquer, dynamic programming, and the greedy algorithm. You might find certain similarities between them, as they all deal with a subproblem in some sense, but what is the difference?

Divide-and-conquer algorithms mainly divide a large problem into smaller, independent subproblems, where we do not store the subproblem results; rather, we use them directly. In most examples we saw, the size is chopped by a proportion.

Dynamic programming methods mainly involve overlapping subproblems, with size usually reduced by one (there are exceptions). They are related in the sense that we typically need the result from the previous subproblem to compute the result for the larger problem. For the same reason, we need to store the results in a table, using either tabulation (filling the table bottom up) or memoization (filling it top down). The longest common subsequence implementation in OCaml is one example.

Greedy algorithms are the ones where, at every subproblem, we compute a solution for LOCAL OPTIMUM. That is why, for the rest of the greedy greedy algorithms—including Dijkstra's algorithm, minimum spanning trees, and the Prim–Dijkstra algorithm—we prove some result guaranteeing GLOBAL OPTIMUM from the local optimal choices we made. In contrast with dynamic programming, the latter always aims at global optimum by computing all possible solutions while iterating through all subproblems.

Construction of a Huffman tree

Store the tree in arrays indexed $1, \dots, 2n-1$:

indices	$1, \dots, n-1$	$n, \dots, 2n-1$
role	internal nodes	leaves

Index 1 is the root; left$[i]$ and right$[i]$ store the children of internal node i. There are $n-1$ internal nodes because there are no one-child nodes.

Huffman construction

```
make a priority queue PQ from
{(i, p_i) | i = n, ..., 2n − 1}
for i = n − 1 down to 1:
  (x, p) ← DeleteMin(PQ)
  (y, q) ← DeleteMin(PQ)
  left[i] ← x
  right[i] ← y
  Insert((i, p + q), PQ)
```

A priority queue is most useful for DeleteMin, giving

$$O(n \log n)$$

time.

Exercises

1. Show
$$\sum_i p_i \ell_i \leq \lceil \log_2 n \rceil$$
by comparing the optimal Huffman tree with a binary tree.
2. Show that the explicit table of codewords for a Huffman tree can use $\Omega(n^2)$ space.
3. Modify Huffman construction to build a tree in $\Theta(n)$ time for:
   (a) n nodes with weights 61 or 101.
   (b) n nodes with presorted weights.
   Hint: reduce the cost of DeleteMin by keeping two queues: one for original weights and one for weights of combined nodes.

What a compressed file stores

A compressed file contains:

1. the encoded message bits;
2. the shape of the Huffman tree;
3. the symbols in leaf order.

If the output message has length n, its encoded data occupies

$$\#\text{bits} = \left(\sum_i p_i \ell_i \right) n \leq cn = \Theta(n).$$

The binary-tree shape can be stored in at most

$$2(2n - 1)$$

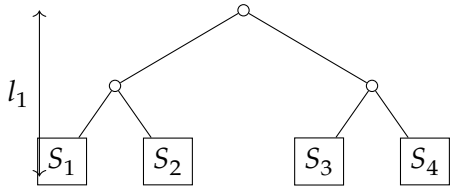
bits using a node/parenthesis-style encoding.

13.2.2 Optimal Merging of Sorted Files

Given: n sorted files with sizes $S_1, \ldots, S_n$. The goal is to merge them into one sorted file. A classical merge of files of sizes S_1 and S_2 takes time $S_1 + S_2$.

A complete merge schedule is a binary tree. If file i is a leaf at depth l_i, its items participate in l_i merges, so

$$\text{cost} = \sum_{\text{leaves } i} l_i S_i.$$



The optimized greedy algorithm is the Huffman-tree construction: recursively merge the two files of smallest size.

13.2.3 Generating a Weighted Random Integer

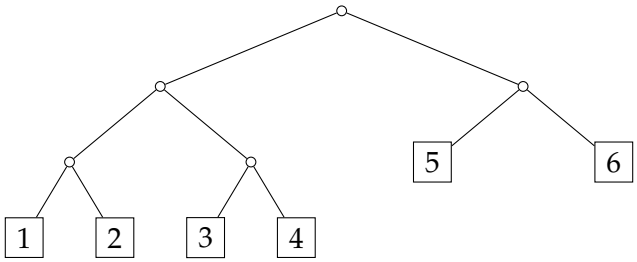
Suppose integer $i \in \{1, \dots, 6\}$ should be generated with probability p_i. Use a uniform random-number generator $U \sim \text{Unif}[0,1]$ efficiently.

Partition $[0,1]$ into consecutive intervals of lengths $p_1, \dots, p_6$. For example, if $U = 0.3$ lies in the interval of p_2, output 2.

Build a binary decision tree whose leaves are $1, 2, \dots, 6$. At a node representing leaves 1 through 6, compare U with the cumulative mass of the left subtree; for example,

$$U < p_1 + p_2 + p_3 + p_4 \quad \Longrightarrow \quad \text{go left},$$

and otherwise go right. Continue until reaching a leaf.



Charging one unit per comparison gives expected cost

$$\sum_i p_i l_i.$$

Thus the optimal decision tree is again a Huffman tree.

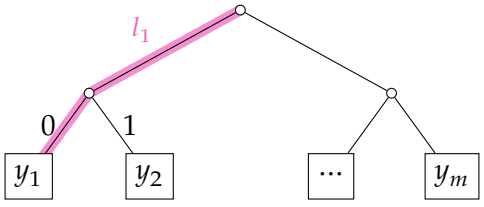
13.3 Shannon's Theory

Recall the lossless-compression pipeline

$$x_1, \dots, x_n \longrightarrow \boxed{\text{compress}} \longrightarrow y_1, \dots, y_L \in \{0,1\} \longrightarrow \boxed{\text{decompress}} \longrightarrow x_1, \dots, x_n.$$

View the binary codewords as root-to-leaf paths in a trie. If input symbol i appears with probability p_i and its codeword has length l_i, the average code length is

$$L = \sum_{\text{leaves } i} p_i l_i.$$



When inputs are strings rather than individual symbols, the l_i can be enormous. Two solutions are:

1. cut the input into manageable chunks and apply Huffman coding;
2. use Lempel–Ziv compression (introduced after 1977).

Entropy

Given probabilities $p_1, p_2, \ldots$, define

$$\mathcal{E} := \sum_{i=1}^{\infty} p_i \log_2 \frac{1}{p_i}.$$

Then $\mathcal{E} \geq 0$.

Example

For a fair coin, $p_0 = p_1 = 1/2$ and

$$\mathcal{E} = \sum_{i=0}^{1} p_i \log_2 \frac{1}{p_i} = 1.$$

For n equally likely outcomes, $p_1 = \cdots = p_n = 1/n$ and $\mathcal{E} = \log_2 n$.

Achievable average code length

$$\mathcal{E} \leq \underbrace{\min_{\text{binary trees}} \sum_i p_i l_i}_{\text{Huffman}} \leq \underbrace{\sum_i p_i l_i}_{\text{Shannon–Fano code}} \leq \mathcal{E} + 1.$$

13.3.1 Two Facts Used in the Entropy Bound

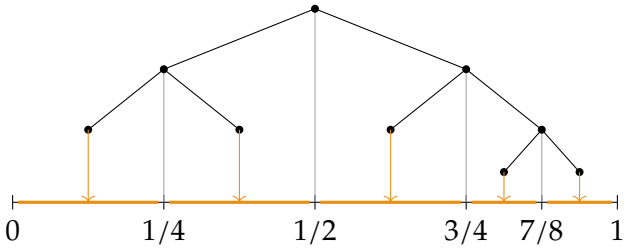
Fact 1. For every $x > 0$,

$$\ln x \leq x - 1.$$

Fact 2 (Kraft's theorem). For every binary tree,

$$\sum_{\text{leaves } i} \frac{1}{2^{l_i}} \leq 1.$$

Visual proof of Kraft's theorem. Start at the root and recursively assign the current node the midpoint of its interval. Each leaf at depth l_i receives the smallest dyadic interval belonging to it, of length 2^{-l_i}.



The leaf intervals are disjoint subsets of $(0,1)$, hence their total length is at most 1. In fact,

$$\sum_i 2^{-l_i} = 1 \quad \Longleftrightarrow \quad \text{there are no internal nodes with only one child.}$$

13.3.2 Proof of the Lower Bound

$$\begin{aligned}
\mathcal{E} - \sum_i p_i l_i &= \sum_i p_i \left(\log_2 \frac{1}{p_i} - l_i \right) \\
&= \sum_i p_i \frac{\ln\left(1/(p_i 2^{l_i})\right)}{\ln 2} \\
&\le \frac{1}{\ln 2} \sum_i p_i \left(\frac{1}{p_i 2^{l_i}} - 1 \right) \\
&= \frac{1}{\ln 2} \left(\sum_i \frac{1}{2^{l_i}} - 1 \right) \\
&\le 0,
\end{aligned}$$

where the first inequality uses Fact 1 and the last uses Kraft's theorem. Therefore

$$\mathcal{E} \le \sum_i p_i l_i.$$

13.3.3 Shannon–Fano Code

Given $p_1, p_2, \ldots$, define

$$l_i = \left\lceil \log_2 \frac{1}{p_i} \right\rceil.$$

Since Huffman coding is optimal,

$$\sum_i p_i l_i \ge \min_{\text{binary trees}} \sum_i p_i l_i.$$

It remains to show the upper bound:

$$\begin{aligned}
\sum_i p_i l_i &= \sum_i p_i \left\lceil \log_2 \frac{1}{p_i} \right\rceil \\
&\le \sum_i p_i \left(\log_2 \frac{1}{p_i} + 1 \right) \\
&= \mathcal{E} + \sum_{i=1}^{\infty} p_i = \mathcal{E} + 1.
\end{aligned}$$

Cumulative-probability construction

Rank the probabilities from high to low and begin with cumulative probability $q = 0$. For each p_i, write q in binary and take its first l_i binary digits as the Shannon–Fano codeword for symbol i. Then update $q \leftarrow q + p_i$ and continue.
For

$$(p_1, p_2, p_3, p_4) = (0.4, 0.3, 0.2, 0.1), \qquad (l_1, l_2, l_3, l_4) = (2, 2, 3, 4),$$

the construction is:

symbol	q	binary expansion of q	code
1	0	0.000000 …	00
2	0.4	0.011001100110 …	01
3	0.7	0.101100110011 …	101
4	0.9	0.111001100110 …	1110

To obtain a binary expansion of $0.d_1d_2\ldots$, repeatedly multiply the remaining decimal fraction by 2 and append the resulting integer part, 0 or 1.
For this example, $\mathcal{E} = 1.846439$ and the Shannon–Fano average length is $\sum_i p_i l_i = 2.4 < \mathcal{E} + 1$.

13.4 Lempel–Ziv Parsing

Let the alphabet be $\mathcal{A} = \{a, b, c\}$. The input sequence is

$$a\ a\ a\ b\ c\ a\ b\ b\ b\ a\ a\ a\ a\ b\ c\ c\ b\ a\ a\ b\ c\ c\ a.$$

Draw a divider after every shortest prefix that has not previously appeared as a whole phrase. The displayed parsing is

$$a \mid aa \mid b \mid c \mid ab \mid bb \mid aaa \mid abc \mid cb \mid aab \mid cc \mid a.$$

Label the phrases $1, 2, \ldots, 12$. The displayed output is

$$0a,\ 1a,\ 0b,\ 0c,\ 1b,\ 3b,\ 2a,\ 5c,\ 4b,\ 2b,\ 4c.$$

The final phrase a is displayed as phrase 12; no additional (parent, new character) pair is displayed for it.

If alphabet symbols use a fixed number of bits but phrase references have variable width, the receiver faces a separation problem when it receives the concatenated binary string. For phrase number k, reserve

$$\lceil \log_2 k \rceil$$

bits for the index of the previously seen phrase.

Thus the phrase-reference fields use successively $0, 1, 2, 3, \ldots$ bits as the phrase number grows. The receiver can count fields of the prescribed widths and recover the phrase boundaries.

Number of phrases

Show that a length-n string produces

$$c = O\left(\frac{n}{\log n}\right)$$

phrases. Hint: if the alphabet has size k, then for every $r \in \mathbb{N}$,

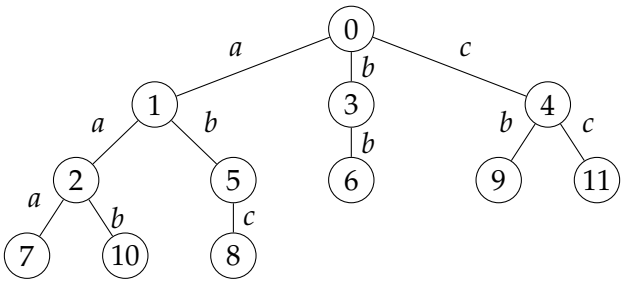
$$c \le \underbrace{k + k^2 + \cdots + k^{r-1}}_{\text{distinct phrases of length } <r} + \frac{n}{r} \le k^r + \frac{n}{r}.$$

Choose

$$r = \left\lfloor \frac{1}{2} \log_k n \right\rfloor.$$

13.4.1 Digital Search Tree for the Phrases

Store one node per phrase. A node's label is its phrase number, and an edge's label is the newly appended character.



For example, node 2 is reached by path aa, and its parent pointer leads to node 1, whose parent is node 0.

Example

Write an $O(n)$ algorithm for this process and compare its compression result with the result of `zip`.

Receiver. Keep a table of parent pointers and new characters:

piece	parent piece	new character
1	0	a
2	1	a
3	0	b
4	0	c
⋮	⋮	⋮

Following parent pointers reconstructs a phrase; for example $2 \to 1 \to 0$ spells aa in reverse traversal order.

Note

The alphabet $\{a, b, c\}$ itself cannot be encoded with variable-width symbols here: a fixed bit length is needed to avoid the separation problem.

Chapter 14

Graph Foundations, Search, and Shortest Paths

14.1 Graphs and Their Representations

14.1.1 Basic Graph Terminology

A graph is

$$G = (V, E), \qquad n := |V|, \qquad e := |E|,$$

where V is the set of vertices (nodes) and E is the set of edges.

Example

$$V = \{1, 2, 3, 4, 5\}, \qquad E = \{(4, 5), (1, 2), (2, 3), (1, 3)\}.$$

The vertices $1, 2, 3$ form a connected component containing a cycle, while $4, 5$ form a second component.



connected component with a cycle

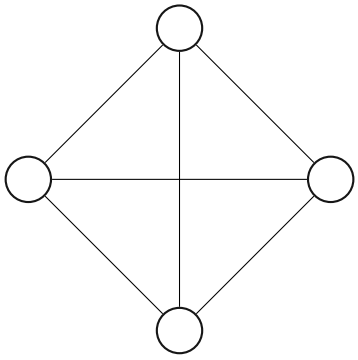
A *path* is a sequence of adjacent vertices. Edges may be undirected or directed. A directed edge from a vertex to itself is a loop; a cycle is a closed path through edges.

14.1.1.1 Complete and Bipartite Graphs

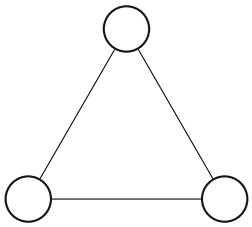
The complete undirected graph K_n contains every one of the

$$\binom{n}{2}$$

possible unordered vertex pairs as an edge.



K_4



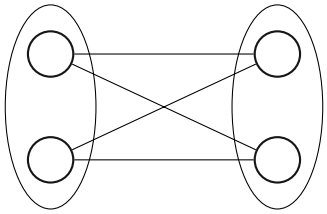
K_3

An undirected graph is bipartite when

$$V = A \cup B, \qquad A \cap B = \emptyset, \qquad E \subseteq A \times B.$$

Thus every edge has one endpoint in A and the other in B.

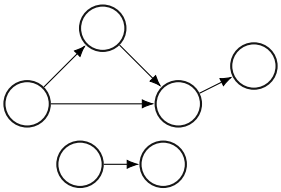
K_4 is not bipartite, whereas crossing edges in a drawing do not prevent a graph from being bipartite:



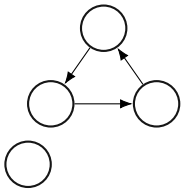
A B

14.1.2 Directed Acyclic Graphs

A directed acyclic graph (DAG) has no directed cycle. The left example below is a DAG; the right example is not.



DAG



contains a directed cycle

14.1.3 Graph Representations

14.1.3.1 Adjacency Matrix

This representation is described as "good for large graphs." For vertices $1, \ldots, 5$ in the example, the matrix entry A_{ij} is 1 exactly when $(i, j) \in E$:

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

If the graph is undirected and has no loops, the matrix is symmetric and its main diagonal is zero. It therefore suffices to store the entries above the main diagonal; the greyed entries on and below the diagonal are redundant. For the example,

$$(A_{12}, A_{13}, A_{14}, A_{15}, A_{23}, A_{24}, A_{25}, A_{34}, A_{35}, A_{45}) = (1100100001)_2.$$

The space usage is

$$\Theta(|V|^2).$$

14.1.3.2 Adjacency Lists

Store each vertex's neighbors in a linked list. For the running undirected example:

1	$3 \to 2$
2	$1 \to 3$
3	$1 \to 2$
4	5
5	4

An undirected adjacency-list representation stores $|V|$ list heads and $2|E|$ edge records, hence

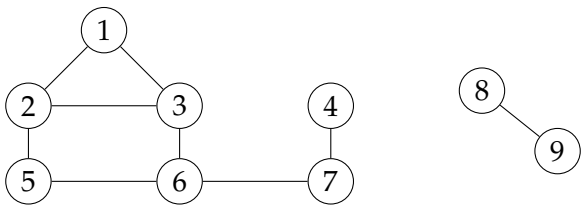
$$\Theta(|V| + |E|)$$

space. In the pointer-level picture, each list record contains a pointer to the neighboring vertex record and a pointer to the next list record.

14.2 Depth-First Search

14.2.1 The Algorithm

Consider the graph



Setup: node colors.

- white means "not touched";
- grey means "active";
- black means "done."

The setup will also be used in a future lecture.

Maintain a global time t. For vertices u, v:

- $d[v]$ is the discovery time, when v changes from white to grey;
- $f[v]$ is the finish time, when v changes from grey to black;
- $p[v]$ is the parent of v in the resulting DFS tree.

We write DFS(u) for the procedure that visits the connected component containing u.

To search the whole graph, initialize every vertex and launch one DFS call from each still-white vertex:

```
for every u:
  (color[u], p[u]) ← (white, nil);
time ← 0;
for every u:
  if color[u] = white, DFS(u).
```

```
DFS(u):
  color[u] ← grey;
  time ← time + 1;
  d[u] ← time;
  for every neighbor v of u:
    if color[v] = white:
      p[v] ← u; DFS(v);
  color[u] ← black;
  time ← time + 1;
  f[u] ← time.
```

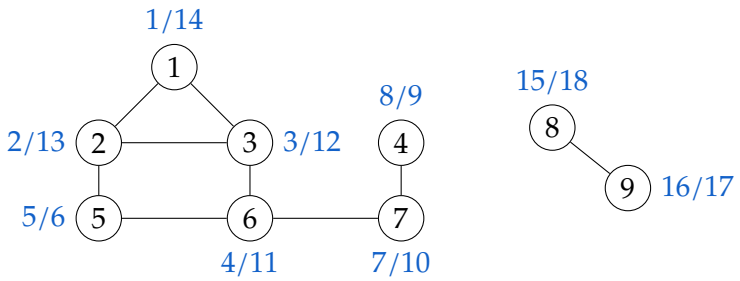
Across all calls, the vertex initialization and color/time assignments cost $\Theta(|V|)$. The neighbor loops examine $2|E|$ adjacency records in an undirected graph, so the total cost is

$$\Theta(|V| + |E|).$$

Why keep all the timestamps and colors?

The black-color mechanism and both $d[u]$ and $f[u]$ can be removed while leaving a functioning traversal. They become useful for proving properties, classifying edges, and finding strongly connected components.

For the graph introduced above, one run assigns the following discovery/finish times:



The resulting DFS forest has tree paths

$$1 \to 2 \to 3 \to 6, \qquad 6 \to 5, \qquad 6 \to 7 \to 4, \qquad 8 \to 9.$$

14.2.2 Classification of Edges

Classify an edge the first time it is seen. From u to v, if color$[v]$ = white, the edge is a *tree edge*.

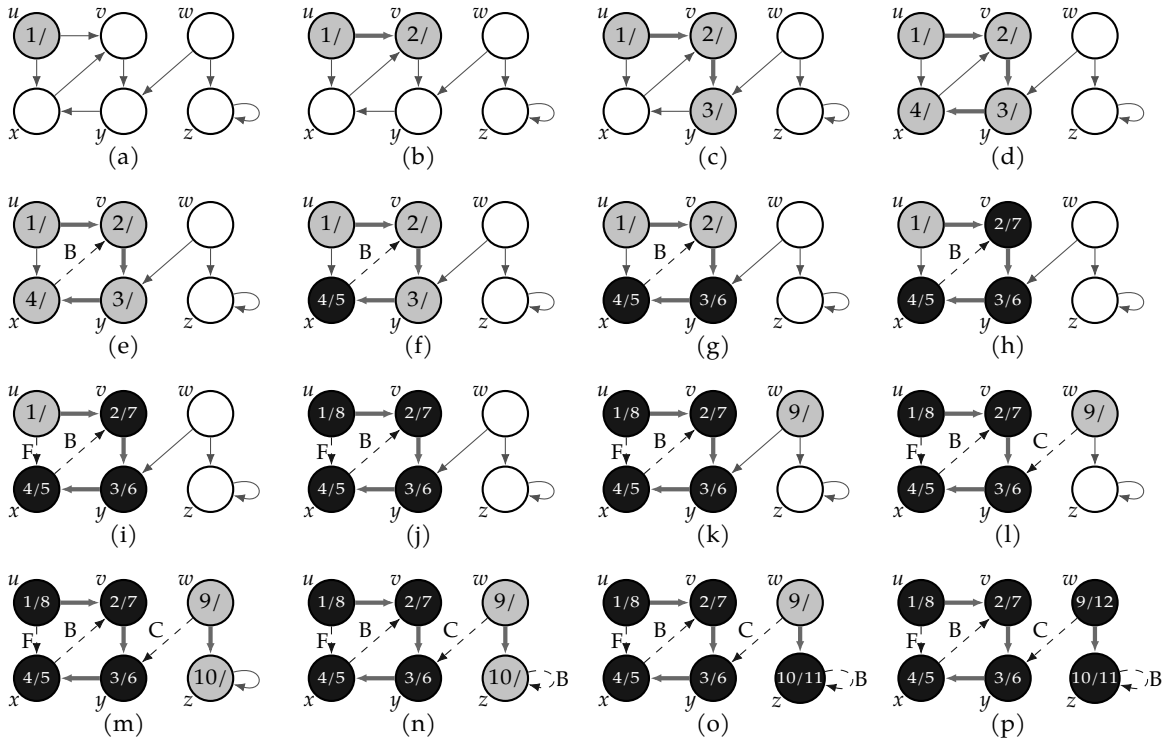
If v is grey when exploring an edge from the active vertex u, it is a *back edge*.

Undirected DFS edge classification (Cormen, p. 610)

In a depth-first search of an undirected graph G, every edge of G is either a tree edge or a back edge.
Proof. Let (u, v) be an arbitrary edge and suppose without loss of generality that $d[u] < d[v]$. The search must discover and finish v before it finishes u, while u is grey, because v occurs in u's adjacency list.
If the edge is first explored from u to v, then v is white until that moment; otherwise the same edge would already have been explored from v to u. Hence (u, v) becomes a tree edge. If it is first explored from v to u, then u is still grey, so (u, v) is a back edge. □

The benefit is structural: in an undirected DFS tree, a non-tree edge must connect an ancestor and a descendant. There are no cross edges between unrelated subtrees and no distinct forward-edge class.

The following progression (after CLRS, Figure 22.4) shows DFS on a directed graph. Thick edges are tree edges; a dashed edge has just been examined and is labeled B, C, or F according to whether it is a back, cross, or forward edge. Timestamps inside the vertices are discovery/finish times; grey vertices are active and black vertices are finished.

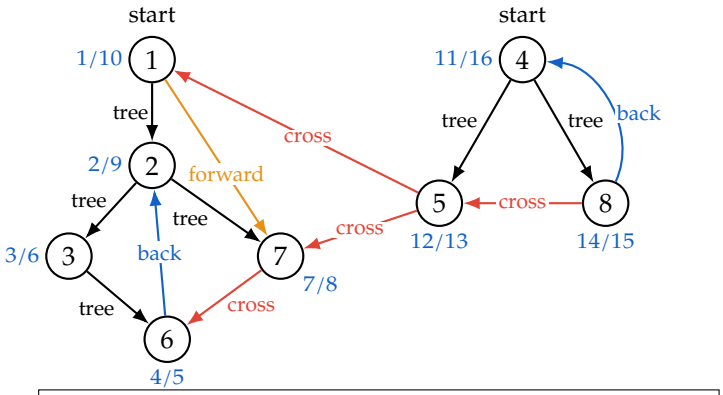


14.2.2.1 Two Additional Classes in Directed Graphs

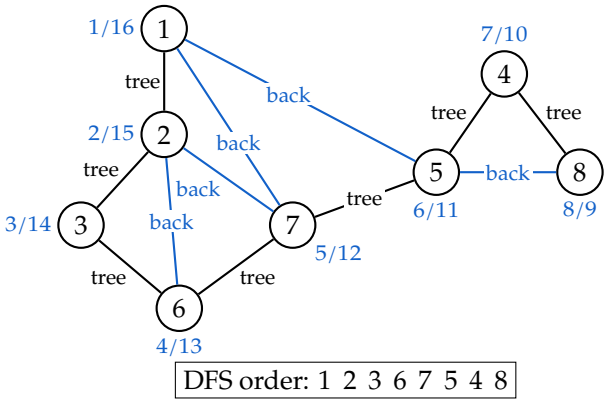
When v is already black as (u, v) is examined:

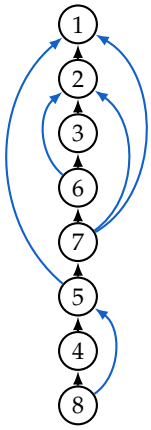
- (u, v) is a *forward edge* if v is a descendant of u in the DFS tree;
- otherwise (u, v) is a *cross edge*.

The in-class example is the directed graph below. Blue numbers are discovery/finish times, and the edges are colored and labeled by class: tree edges black, back edges blue, forward edges orange, and cross edges red. The search from vertex 1 gets stuck after visiting $1, 2, 3, 6, 7$, so a second search is started from vertex 4.

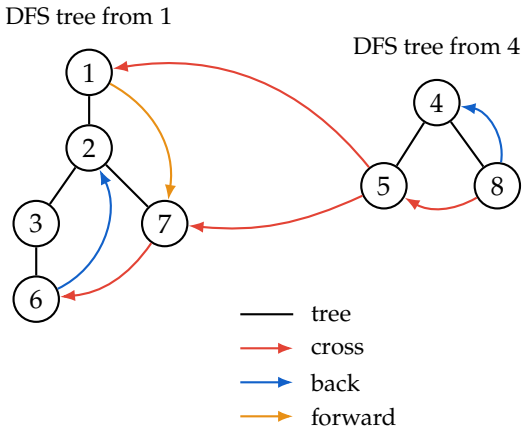


For comparison, make the same graph undirected and run DFS from vertex 1 again. Now every non-tree edge is a back edge, in agreement with the theorem above, and the search no longer gets stuck: it reaches 5, 4, and 8 from 7. The right-hand drawing shows the resulting DFS tree with its parent pointers and the five back edges.





The DFS forest of the directed example is drawn below with the non-tree edges colored by class. It has two trees because the directed search had to restart at vertex 4.



The undirected run therefore has DFS order

$$1, 2, 3, 6, 7, 5, 4, 8,$$

whereas the directed run produced $1, 2, 3, 6, 7$ and then $4, 5, 8$.

14.2.2.2 Equivalent Classification Used by Cormen

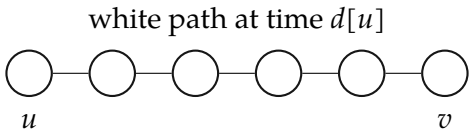
1. **Tree edges** are the edges in the depth-first forest G_π. Edge (u, v) is a tree edge if v is first discovered by exploring (u, v).
2. **Back edges** are edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. Self-loops, which may occur in directed graphs, count as back edges.
3. **Forward edges** are non-tree edges (u, v) connecting u to a descendant v in a depth-first tree.
4. **Cross edges** are all other edges. They may connect unrelated vertices in the same depth-first tree or vertices in different depth-first trees.

14.2.3 Cycle Detection

White-path theorem

A vertex v is a descendant of u in the DFS tree if and only if, at time $d[u]$, there exists a path from u to v consisting entirely of white vertices (apart from the newly discovered u).

The proof is by induction and is omitted.



Cycle criterion for undirected graphs

An undirected graph has no cycles if and only if its DFS has no back edges.

Practical detector

Inside the DFS neighbor loop, add

```
if color[v] = grey and p[u] ≠ v, report a cycle.
```

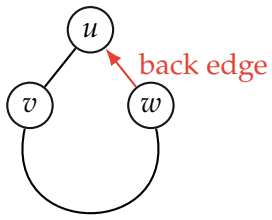
The parent of u is also grey, but the parent edge is a tree edge and must not be reported as a back edge.

Proof.

($\Rightarrow$) Suppose a back edge (v, u) exists, so v is a descendant of u. The DFS-tree path from u down to v, together with the graph edge (v, u), forms a cycle.

($\Leftarrow$) Suppose a cycle exists. Let u be the vertex on the cycle that is colored grey earliest, and let v, w be its two adjacent cycle vertices.

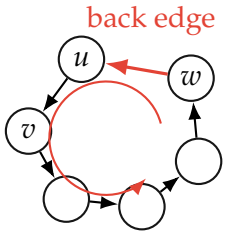
If one of v, w is already grey at time $d[u]$, the edge to it is a back edge. Otherwise both are white at $d[u]$. Depth-first search proceeds around one direction of the cycle. Eventually one of (v, w) or (w, v) is examined while its other endpoint remains active. That edge cannot be the edge that first discovered the endpoint from the opposite direction of the cycle, so it is a back edge. Thus every undirected cycle produces a back edge. □



Cycle criterion for directed graphs

A directed graph contains a directed loop/cycle if and only if every full DFS contains at least one back edge.

Proof of the directed cycle criterion (continued). The proof is almost exactly the same as for the preceding undirected corollary. The important directed distinction is that the closing edge itself must point from a descendant back to an ancestor:



Practical detector in a directed graph

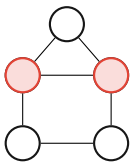
It is enough to test whether color$[v]$ = grey while examining an edge (u, v). A directed parent pointer represents exactly one directed tree edge, from the parent to the child. If the reverse direction is later present as an edge from the child to the parent, it is a different edge and is a back edge, not the same tree edge.

14.3 Euler Tours

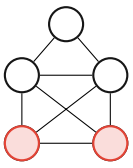
Tour and Euler tour

A *tour* is a path that starts at a vertex and returns to that vertex. An *Euler tour* traverses every edge exactly once and returns to its starting vertex.

Not every graph has an Euler tour. Neither graph below has one: in the left graph the two upper corners have odd degree 3; adding the two crossing diagonals (right) makes those even but gives the two bottom corners degree 3, so a closed tour is still impossible (only an open Euler path between the two odd-degree vertices exists). Odd-degree vertices are drawn in red.



no Euler tour
(two vertices of degree 3)



still no Euler tour
(two vertices of degree 3)

Entering and leaving

At every vertex, including the starting vertex, each arrival along an unused edge must be paired with a departure along another unused edge. A walk can become stuck if it enters a vertex after all

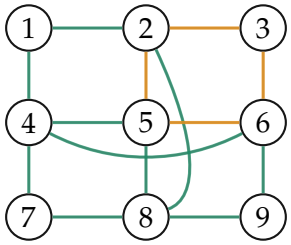
other incident edges have already been consumed.

Euler-tour characterizations

- An undirected graph has an Euler tour if and only if every vertex has even degree.
- A directed graph has an Euler tour if and only if every vertex has equal in-degree and out-degree.

Graphs satisfying the appropriate condition are called *Euler graphs* or *Eulerian graphs*.

The following Eulerian graph illustrates the construction. The green edges form the first call to EulerWalk; the unused orange cycle is inserted by a later call.



first Euler walk
second Euler walk

Euler-walk subroutine. Let AL be the adjacency lists, and let taking the front item of $AL[v]$ also delete that edge record.

Literal control flow

```
EulerWalk(u): // u is the start vertex
Q ← an empty queue; Enqueue(u,Q);
v ← u; AL ← the adjacency lists;
while v ≠ u:
  v ← and delete the front item of AL[v];
  Enqueue(v,Q);
Output(Q).
```

Since v is initialized to u, the loop guard is false before the first iteration, so this literal subroutine outputs only the initial queue. The intended walk is described as taking edges until it returns to u. This box keeps the literal control flow visible as an educational warning.

Note

In an Eulerian graph, a walk that consumes unused edges cannot get stuck at a different vertex: every arrival can be paired with a departure. Thus it eventually returns to u.

Deleting an undirected edge efficiently

Deleting an undirected edge requires removing its records from the adjacency lists of both endpoints. Design a data structure that supports this efficiently. Hint: use the same kind of cross-linking idea as in the augmented two-red–black-tree structure.

Assume that data structure. For $V = \{1, \ldots, n\}$, the full tour can be assembled as follows.

```
EulerTour(G):
S ← an empty stack;
Push(1, S);
while |S| > 0:
  v ← Pop(S);
  if |AL[v]| = 0, then Output(v);
  else Push(EulerWalk(v), S).
```

The running time is $O(|E|)$, given the efficient deletion structure from the exercise. The idea is that whenever a vertex in the current walk still has unused incident edges, another Euler walk can be started there and spliced into the current walk.

In the running example, the first walk stored on the stack is

$$1, 4, 5, 8, 9, 6, 4, 7, 8, 2, 1.$$

At vertex 5, the walk $5, 2, 3, 6, 5$ is substituted. The outputs therefore form the Euler tour

$$\boxed{1, 4, 5, 2, 3, 6, 5, 8, 9, 6, 4, 7, 8, 2, 1}.$$

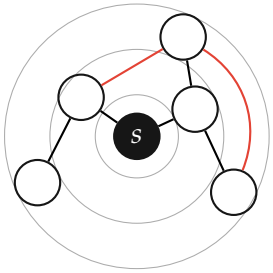
14.4 Breadth-First Search

For vertices u, v, define the unweighted distance

$$\text{dist}(u, v) = \begin{cases} \text{minimum number of edges in a path from } u \text{ to } v, & \text{if a path exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

Let s be the start, or *source*, vertex.

Breadth-first search exposes concentric distance levels around s and produces a BFS tree as a side product. Every non-tree edge joins vertices on the same level or on adjacent levels only.



BFS-tree edges in black;
red: non-tree edges, which join vertices
on the same level or on adjacent levels only.

Use the same three colors as for DFS:

$$\text{white} = \text{not seen}, \qquad \text{grey} = \text{active/in the queue}, \qquad \text{black} = \text{done}.$$

Let $p[u]$ be the parent of u in the BFS tree and let

$$d[u] = \text{dist}(u, s).$$

```
BFS(s):
Q ← an empty queue;
for every u:
  (p[u], d[u], color[u]) ← (nil, ∞, white);
d[s] ← 0;  color[s] ← grey;  Enqueue(s, Q);
while |Q| > 0:
  u ← Dequeue(Q);
  for every neighbor v of u:
    if color[v] = white:
      d[v] ← d[u] + 1;
      p[v] ← u;
      color[v] ← grey;
      Enqueue(v, Q);
  color[u] ← black.
```

The neighbor loops inspect $2|E|$ adjacency records in an undirected graph, and all remaining work is $O(|V|)$. Hence

$$T_{\text{BFS}} = O(|V| + |E|).$$

Relation to the COMP 250 implementation

In COMP 250, the color is a field/attribute called `visited`. For this BFS, the final assignment to black may be omitted, making the pseudocode match the COMP 250 code.

Example

Write a BFS-inspired algorithm that checks whether a graph is bipartite. Hint: two-color the graph level by level, assigning each newly discovered vertex the color opposite its parent; reject if an edge has equally colored endpoints.

14.5 Weighted Graphs and Shortest Paths

A weighted graph consists of

$$G = (V, E), \qquad w[u, v] \geq 0 \quad \text{(usually)},$$

where $w[u, v]$ is the weight of edge (u, v). Then

$$\text{dist}(u, v) = \text{the minimum total weight of a path from } u \text{ to } v,$$

also called the shortest-path distance.

Typical shortest-path problems are:

- given a source s and target t, find a shortest path $s \rightsquigarrow t$;
- given s, find a shortest path $s \rightsquigarrow u$ for every u;
- find all-pairs shortest paths $u \rightsquigarrow v$ for all u, v.

14.5.1 Dijkstra's Algorithm

Dijkstra's algorithm applies to weighted graphs with *positive* weights. It is greedy: optimize locally, then continue with the next subproblem.

Let S be the set of vertices whose shortest-path answers have been finalized. Define

$$d[u] = \text{length of a shortest } s\text{–}u \text{ path whose internal vertices lie in } S,$$
$$p[u] = \text{the predecessor of } u \text{ on a path attaining } d[u].$$

For $u \notin S$, $d[u]$ is only tentative and need not yet be optimal.

Greedy-choice property

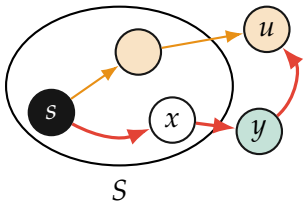
Among all vertices outside S, a vertex u with minimum tentative value $d[u]$ is a correct vertex to finalize next.

Proof. Suppose instead that the true shortest path from s to u is shorter than $d[u]$. Along this path, let y be the first vertex outside S, and let x be its predecessor, so $x \in S$.

When x was finalized, relaxation made $d[y]$ no greater than the length of the shortest path prefix from s to y. Because all remaining edge weights are positive, that prefix is no longer than the entire path to u. Hence

$$d[y] \le \text{dist}(s, y) \le \text{dist}(s, u) < d[u],$$

contradicting the choice of u as the minimum-d vertex outside S. □



path attaining $d[u]$ (internal vertices in S)
hypothetical shorter path
y: first vertex of that path outside S

```
Dijkstra(s):
for every u:
  (p[u], color[u], d[u]) ← (nil, white, ∞);
d[s] ← 0;
make a priority queue PQ containing every pair (u, d[u]);
while |PQ| > 0:
  // continue with the loop body below
```

Here white means not in the partial solution S, while grey means finalized/in S.

```
while |PQ| > 0:
  u ← DeleteMin(PQ);
  color[u] ← grey;
  for every neighbor v of u:
    if color[v] = white:
      if w[v, u] + d[u] < d[v]:
        d[v] ← w[v, u] + d[u];
        p[v] ← u;
        DecreaseKey((v, d[v]), PQ).
```

The greedy-choice property proves that changing u to grey at deletion time cannot cause a shorter path to u to be missed.

Correcting the inconsistent update

The incorrect update is

$$d[v] \leftarrow w[v,u] + d[v],$$

which adds the edge weight to the old tentative value of v. The relaxation test immediately above it uses $w[v,u] + d[u]$, so the consistent and standard update is

$$d[v] \leftarrow w[v,u] + d[u].$$

The corrected line is used in the pseudocode above.

The operation counts are

$$\text{MakePQ} : O(|V|), \qquad \text{DeleteMin} : |V| \text{ times}, \qquad \text{DecreaseKey} : |E| \text{ times},$$

plus $O(|V| + |E|)$ other work.

Priority queue	all Delete-Min operations	all Decrease-Key operations	total
Unordered list	$\|V\|^2$	$\|E\|$	$O(\|V\|^2)$
Binary heap	$\|V\| \log \|V\|$	$\|E\| \log \|V\|$	$O(\max\{\|E\|, \|V\|\} \log \|V\|)$
Fibonacci heap	$\|V\| \log \|V\|$	$\|E\|$	$O(\|E\| + \|V\| \log \|V\|)$

For the unordered-list bound, $|E| \leq \binom{|V|}{2} \leq |V|^2$ in a simple undirected graph.

Reference-only topic

A Fibonacci heap is designed in particular to support DecreaseKey in $O(1)$ actual and amortized time. It was not covered that year and was not examinable; the amortized-analysis material is included for reference.

Chapter 15

Fibonacci Heaps

15.1 Overview and Structure

The Fibonacci heap was conceived by Fredman and Tarjan in 1984 and first published in a journal in 1987.

Overview. Before the analysis, contrast the time complexities of operations on a standard binary heap and a Fibonacci heap. Most operations are familiar from priority queues; the novel operations are defined below.

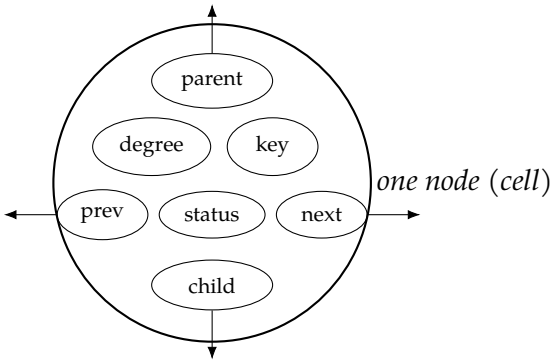
	Binary heap (actual time)	Fibonacci heap (amortized time)
Insert	$\Theta(\log_2 n)$	$\Theta(1)$
Delete	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Delete-Min	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Decrease-Key	$\Theta(\log_2 n)$	$\Theta(1)$
Meld	$\Theta(\log_2 n \times \log_2 m)$	$\Theta(1)$

Table 4: Comparing a binary heap and a Fibonacci heap.

Notice the improvements for Insert, Decrease-Key, and Meld. In the final row, n and m are the sizes of the two heaps being melded.

The data structure. A Fibonacci heap has the following terminology and properties.

- It is a priority queue consisting of a *forest* of heap-ordered trees: every node's key is at least the key of its parent.
- We interact with a heap H through a pointer to the root of the tree having the minimum key.
- The *degree* of a node is its number of immediate children, and every node stores its degree.
- Every node has a child pointer and a parent pointer.
- Each sibling group is organized as a doubly linked list.
- The *root-list* is the doubly linked list of the roots of all trees in the forest.
- Every node stores a *marked* or *unmarked* status, discussed below.



Remark on marking and maximum degree

Marking will ensure that the maximum degree is $O(\log_2 n)$. Throughout the preliminary analysis, assume and later prove that the maximum degree is at most $c \log_2 n$ for some constant c. A node x is marked if it has lost a child since the last time x was made the child of another node.
Here "maximum degree" means the maximum degree among all nodes in the Fibonacci heap.

Root-list nodes are unmarked. For the moment assume that every node is unmarked; the complete marking rules are summarized after the relevant operations.

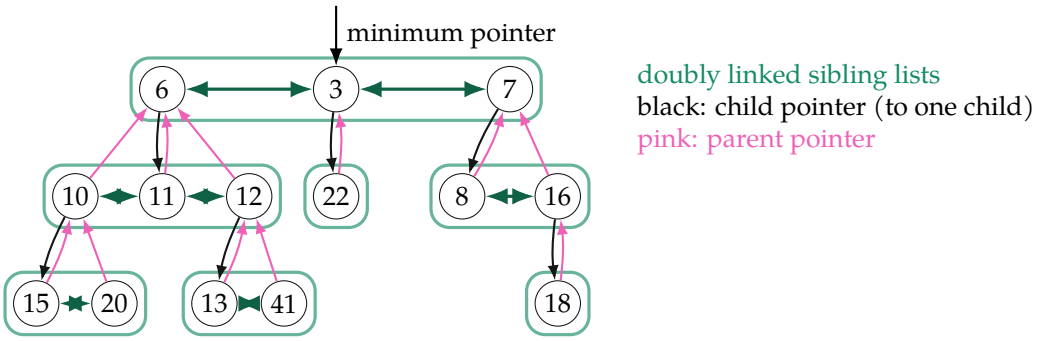


Figure 6: A sample Fibonacci heap.

15.2 Simple Operations and the Potential Function

Simple operations.

- Meld(F_1, F_2) combines the root-lists in $O(1)$ and finds the global minimum in $O(1)$.
- Insert(x, F) creates a one-element Fibonacci heap and applies Meld, so it costs $O(1)$.

Here F_1 and F_2 are Fibonacci heaps. To find the global minimum, compare only their two previous minima.

Potential. Define

$$\boxed{\Phi = \alpha(\text{size of root-list}) + \beta(\text{number of marked nodes})}.$$

The scaling factors α and β will be chosen later. The simple operations above take $O(1)$ amortized time.

Note

Performing Insert n times can create a heap that is entirely one long root-list. Such a list is troublesome when the minimum is removed and the second-smallest node must be found; the remaining operations build in the remedy.

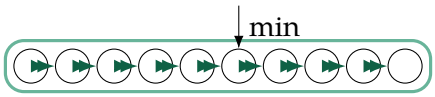


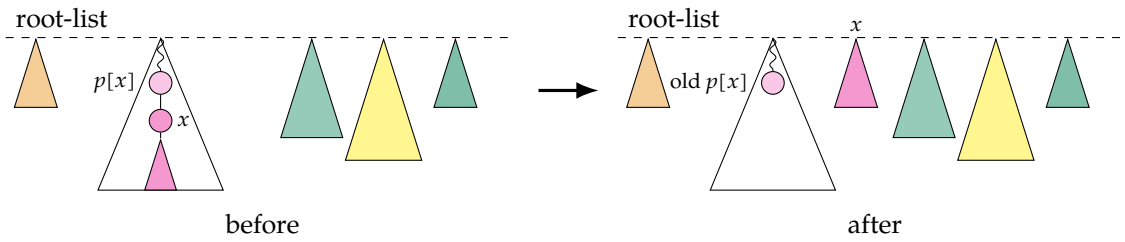
Figure 7: A Fibonacci heap that is entirely a root-list.

15.3 Decrease-Key, Delete-Min, and Delete

Other operations. We now discuss the less trivial operations from Table 4 and their suboperations.

Decrease-Key changes the key of a node x to a lower value k, moving it to a higher priority. Assume k is at most the current key of x.

```
Decrease-Key(x,k,F):
1. Set the key of x to k.
2. If x is not in the root-list and the heap order
  between x and p[x] is upset,
  add x and its subtree to the root-list.
3. If step 2 moved x, test whether it is the new minimum;
  adjust the minimum pointer accordingly.
```



Figures 8–9: Decrease-Key cuts x and its subtree into the root-list when key$[x] <$ key$[p[x]]$.

This operation has $O(1)$ actual time and $O(1)$ amortized time.

Removing nodes. Delete-Min removes the minimum root and moves each of its children, with its subtree, into the root-list. A new operation, Clean-Up-Root-List, then makes finding the next minimum manageable.

```
Delete-Min(F):
1. Report and delete the minimum-key root.
  Add each immediate child to the root-list
  as the root of its subtree.
2. Perform Clean-Up-Root-List.
```

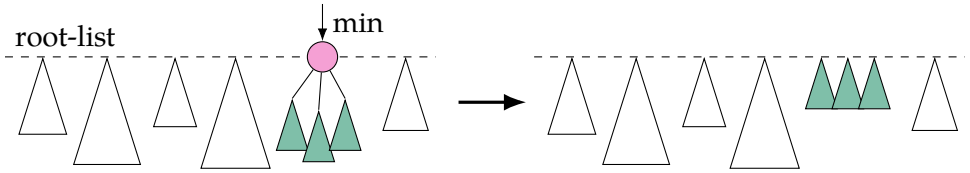


Figure 10: The first step of Delete-Min.

This first step has actual time $O(1)$ and

$$\Delta\Phi \leq \alpha c \log_2 n,$$

because the root-list grows by no more than the maximum degree, which is at most $c \log_2 n$ under the earlier assumption.

The root-list increase in the first step of Delete-Min is at most the maximum degree, hence at most $c \log_2 n$.

```
Delete(x,F):
1. Remove x from the heap.
  Add its immediate children to the root-list
  as roots of their respective subtrees.
2. Perform Clean-Up-Root-List.
```

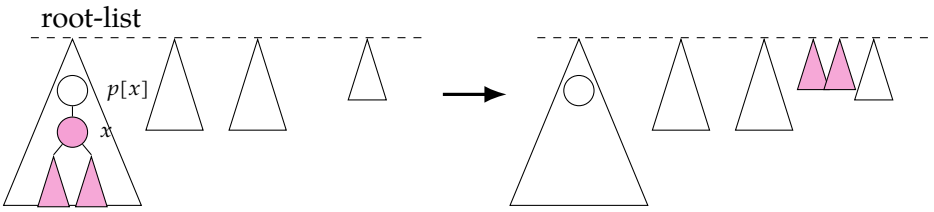


Figure 11: The first step of Delete.

Delete may also be expressed as two familiar operations:

$$\text{DecreaseKey}(x, -\infty, F) \quad \text{followed by} \quad \text{DeleteMin}(F).$$

Thus the actual time is $O(1)$ and $\Delta\Phi \leq c \log_2 n$ for some c.

Let B be an array of buckets, each initialized to nil. Bucket $B[i]$ points to a root of degree i; the pointer is updated as consolidation proceeds.

Bucket-array size

"B should be smaller than the maximum degree in the root-list + $\log_2$(size of the root-list)." Managing the size of B is left as an exercise.

```
Clean-Up-Root-List(F):
1. Empty the root-list into a stack S.
2. while |S| > 0:
3.   x ← Pop(S);
4.   δ ← degree[x];
5.   if B[δ] = nil:
6.     B[δ] ← x;
7.   else: // root-list shrinks by 1
```

```
8.     if key[x] < key[B[δ]], exchange the keys of x and B[δ];
9.     make x a child of B[δ];
10.     y ← B[δ];
11.     degree[y] ← δ + 1;
12.     Push(y, S);
13.     B[δ] ← nil.
14. Make a new root-list from the non-nil buckets in B.
15. // Its size is at most
16. // 1 + maximum degree = O(log_2 n).
17. Find the new minimum in O(log_2 n) time.
```

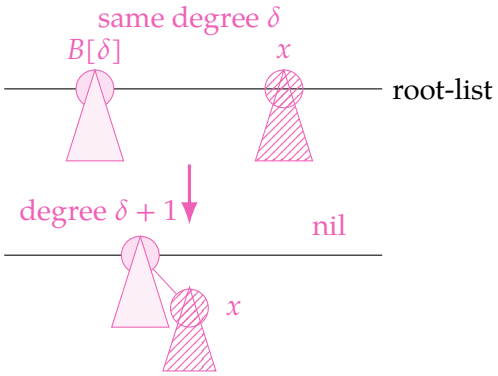


Figure 12: Clean-Up-Root-List links two roots of the same degree.

For Clean-Up-Root-List,

$$\begin{aligned} \text{actual time} &\leq \text{size of root-list before}, \\ \Delta\Phi &\leq \alpha c \log_2 n - \alpha(\text{size of root-list before}). \end{aligned} \tag{11}$$

Consequently, if $\alpha = 1$,

$$\text{amortized time} \leq \alpha c \log_2 n.$$

15.4 Marked Nodes and Cascading Cuts

Marked nodes. The marking rules are:

1. nodes in the root-list are unmarked;
2. a node that has lost a child, through either Decrease-Key or Delete, is marked;
3. no node may lose two children while remaining a child itself.

Cascading-Cut. Whenever a node is about to lose a second child, traverse upward through the tree. At every marked node on this path, cut it from its parent and put it in the root-list with its subtree; its parent thereby becomes marked. Stop upon reaching an unmarked node, which then becomes marked.

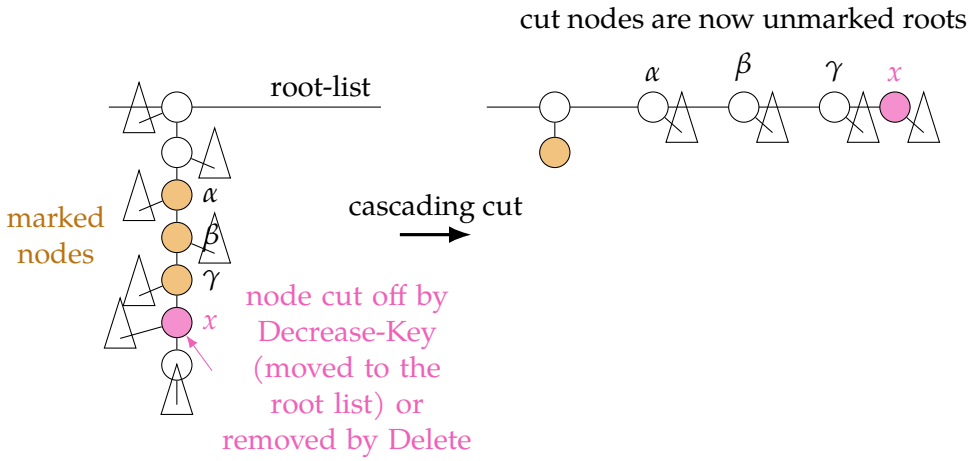


Figure 13: Cascading-Cut.

Question

"This needs to be performed (only) in Decrease-Key step 2 if we cut x off from $p[x]$?"

Let k be the number of marked nodes moved. Then

$$\begin{aligned} \text{actual time} &= k, \\ \Delta\Phi &= \alpha k - \beta k. \end{aligned} \tag{12}$$

Thus the amortized time is 0 when $\beta = \alpha + 1$. Since $\alpha = 1$, choose $\beta = 2$.

The root-list grows by k, while the k moved nodes become unmarked. The node at which Cascading-Cut stops becomes marked, but that contribution is canceled by the starting pink node becoming unmarked. The operation is therefore free in the amortized accounting. It also establishes the repeated maximum-degree assumption used above.

15.5 The Maximum-Degree Bound

Maximum degree of a Fibonacci heap

For a Fibonacci heap of size n, the maximum degree is at most

$$\frac{\log_2 n}{\log_2 \varphi} \approx 1.44 \log_2 n, \qquad \varphi = \frac{1+\sqrt{5}}{2}.$$

The number φ is the limiting ratio of successive Fibonacci numbers, hence the name *Fibonacci heap*.

Linking lemma

The ith child of a node x has degree at least $i - 2$.
Proof. Let y be the ith child linked to x. At the time of that link, x already had at least $i - 1$ children. Clean-Up-Root-List links roots only when they have the same degree, so y then had degree at least $i - 1$. Since y can lose only one child before Cascading-Cut moves it, its present degree is at least $i - 2$. □

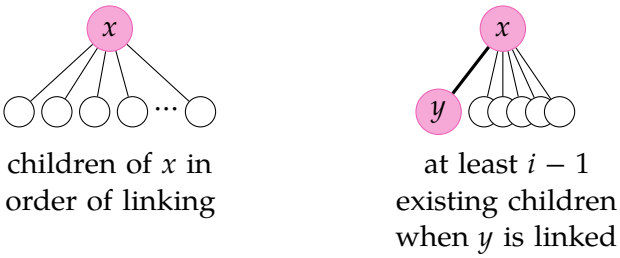


Figure 14: The linking lemma.

Proof of the maximum-degree theorem. Let D_k be the number of descendants, including the node itself, of a node of degree k. We prove by induction on k that

$$D_k \geq \varphi^k.$$

For $k = 0$, $D_0 = 1 = \varphi^0$. For $k = 1$, $D_1 \geq 2 \geq \varphi$. Now suppose $k \geq 2$. Ordering the children by the time they were linked and applying the lemma gives

$$\begin{aligned} D_k &\geq D_{k-2} + D_{k-3} + \cdots + D_0 + 2 \\ &\geq \varphi^{k-2} + \varphi^{k-3} + \cdots + \varphi^0 + 2 \\ &= \frac{\varphi^{k-1} - 1}{\varphi - 1} + 2. \end{aligned} \tag{13}$$

The first inequality is the linking lemma, the second is the induction hypothesis, and the equality is the finite geometric sum. The constant +2 accounts for the node itself and its first child.

The final expression in (13) is at least φ^k because

$$\frac{\varphi^{k-1} - 1}{\varphi - 1} + 2 \geq \varphi^k \iff \varphi^{k-1} - 1 + 2\varphi - 2 \geq \varphi^{k+1} - \varphi^k.$$

Using $\varphi^{k+1} = \varphi^k + \varphi^{k-1}$, this reduces to

$$2\varphi \geq 3,$$

which is true since $\varphi \approx 1.618$.

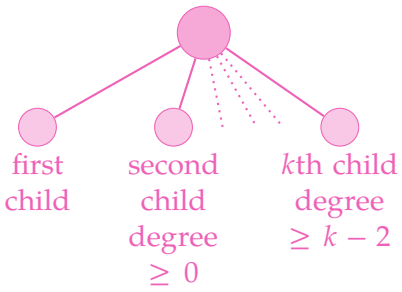


Figure 15: A node and its children, representing the first line of (13).

Finally $D_k \leq n$, so

$$\varphi^k \leq D_k \leq n.$$

Taking base-2 logarithms yields

$$k \leq \frac{\log_2 n}{\log_2 \varphi},$$

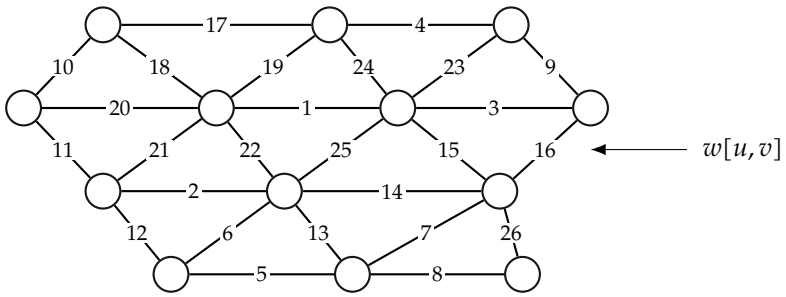
as required. □

Chapter 16

Spanning Trees and Advanced Directed-Graph Algorithms

16.1 Minimum spanning trees

Let $G = (V, E)$ be a weighted connected graph, with edge weight $w[u, v]$ on $(u, v) \in E$. The weighted graph used in class is shown first; the second drawing highlights its minimum spanning tree in pink.



The 26 edge weights are the integers $1, \ldots, 26$, each used once, so the minimum spanning tree is unique.

Spanning tree

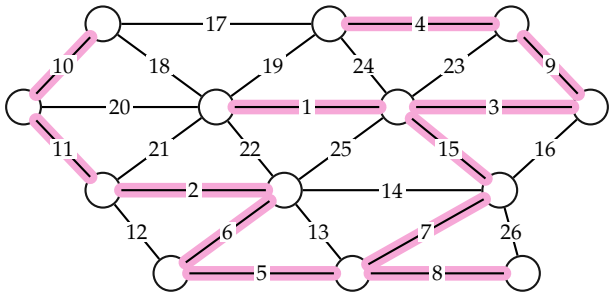
A spanning tree of (V, E) is a graph (V, E^*) that is a tree, where

$$E^* \subseteq E.$$

Minimum spanning tree (MST)

A minimum spanning tree is a spanning tree whose total edge weight is minimum:

$$\mathrm{wt}(T) = \sum_{e \in E(T)} w(e).$$



The pink tree uses the edges of weight $1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 15$, of total weight 81.

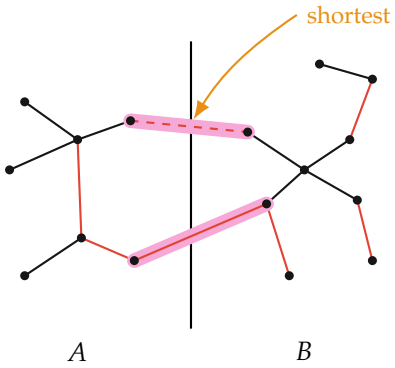
Cut property

Suppose S is a partial solution and A, B are an arbitrary partition relative to S. An edge (u, v) is a *bridge* of this cut when

$$u \in A, \qquad v \in B, \qquad (u, v) \in E.$$

Then a shortest bridge across the cut is part of *an* MST.

Proof (exchange argument, begun). Consider an MST that does not contain the chosen shortest bridge. Since the tree must eventually connect A to B, it contains some other highlighted bridge across the same cut.



black: the components of the partial solution S; red: the remaining edges of an MST T that avoids the shortest bridge
pink: the two bridges across the cut $A \mid B$ (the dashed one is not in T)

Cut-property proof. Replace the other A–B bridge in the tree by the shortest bridge. Removing the old bridge separates the tree along the cut, and inserting the new one reconnects it, so the result is again a spanning tree. Its weight is no larger. Thus there is an MST containing the shortest bridge. □

16.1.1 Greedy strategies for MSTs

16.1.1.1 Prim–Dijkstra

The partial solution S grows one vertex at a time. For every vertex outside S, maintain

$$d[v] = \text{weight of the shortest bridge from } v \text{ to a vertex in } S, \qquad p[v] = \text{its endpoint in } S.$$

Unlike shortest-path Dijkstra, $d[v]$ is not the distance from a fixed source; it is updated as the current tree S grows.

Prim–Dijkstra

```
for every u ∈ V:
  (d[u], p[u], color[u]) ← (∞, nil, white)
choose a start vertex s;  d[s] ← 0
MakeNull(PQ)
put every pair (u, d[u]) in PQ
while |PQ| > 0:
  u ← DeleteMin(PQ)
  color[u] ← gray
  for every neighbor v of u:
    if color[v] = white and w[u,v] < d[v]:
      p[v] ← u;  d[v] ← w[u,v]
      DecreaseKey((v, d[v]), PQ)
```

The selected edges are $(p[v], v)$ for $v \neq s$. The algorithm has the same priority-queue implementation choices as Dijkstra's shortest-path algorithm.

Incomplete complexity accounting

The bound $O(|V| + |E|)$ counts the graph-scanning portion; with an ordinary comparison priority queue, the priority-queue operations must also be counted (as in the corresponding implementation of Dijkstra).

16.1.1.2 Kruskal's algorithm

First order the edges by increasing weight, at cost $O(|E| \log |E|)$, and then scan them using a disjoint-set test.

Kruskal's algorithm

```
for edges (u,v), from smallest to largest weight:
  if set(u) ≠ set(v):
    output (u,v)
    Join(set(u), set(v))
```

At every stage the partial solution S is a forest. Its connected components are the disjoint sets. The cut property justifies adding the lightest edge whose endpoints lie in different components.

Set management. The full disjoint-set lecture is skipped here. One elementary representation stores a component label at every array position, for example

$$[\,3, 3, 3, 3, 4, 4, \ldots\,].$$

When joining, always relabel the smaller set into the larger set (in the example, flip the 4's into 3's).

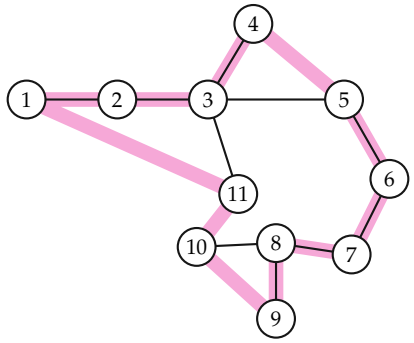
Exercise

Show that, over all joins, this weighted-union rule takes $O(|V| \log |V|)$ time.

16.1.2 A 2-approximation for metric TSP

A metric instance satisfies the triangle inequality. For comparison, the non-metric traveling-salesperson problem can be solved by dynamic programming in $O(n^2 2^n)$ time. The following method returns a tour whose length is at most twice optimum.

1. Find a minimum spanning tree in $O(n^2)$ time.
2. Run DFS on the MST and retain the discovery (preorder) order; this takes $O(n)$ on the tree.
3. Visit the vertices in that order, shortcutting already visited vertices to make a tour.



DFS on a tree is a preorder traversal. By the triangle inequality, shortcutting cannot increase length, so

$$\text{length(shortcut tour)} \le \text{length(full tree walk)} = 2\,\text{length(MST)} \le 2\,\text{OPT}_{\text{TSP}}\,.$$

Deleting the last edge of any TSP tour leaves a spanning tree; therefore its length is at least the MST length.

The approximation proof ends with

$$\text{length(MST)} \le \text{length(TSP)}.$$

16.2 All-pairs shortest paths: Floyd–Warshall

For every pair i, j, we want

$$d[i,j] = \text{shortest-path distance}, \qquad p[i,j] = \text{first vertex on a shortest path } i \to j.$$

The tables use $O(n^2)$ space. Let the vertices be $\{1, 2, \dots, n\}$, and define

$$d[i,j,k] = \text{shortest } i \to j \text{ path whose possible intermediate vertices are all at most } k.$$

The base cases are

$$d[i,j,0] = \begin{cases} w[i,j], & (i,j) \in E, \\ \infty, & \text{otherwise}, \end{cases} \qquad p[i,j,0] = \begin{cases} j, & (i,j) \in E, \\ \text{nil}, & \text{otherwise}. \end{cases}$$

In general, $p[i,j,k]$ is the first vertex on the path represented by $d[i,j,k]$.

Floyd–Warshall

```
for k = 1 to n:
  for i = 1 to n:
    for j = 1 to n:
      a ← d[i,j,k−1]
      b ← d[i,k,k−1] + d[k,j,k−1]
      d[i,j,k] ← min(a,b)
      if a < b: p[i,j,k] ← p[i,j,k−1]
      else: p[i,j,k] ← p[i,k,k−1]
```

Negative weights

1. Prim–Dijkstra and Kruskal work with negative edge weights. Floyd–Warshall additionally requires no negative cycles (cycles of negative total weight).
2. Dijkstra for a shortest path between two vertices does *not* work with negative weights. Its greedy proof relies on any alternative path leaving the partial solution and then traversing nonnegative edges; negative edges can reduce that path and invalidate the commitment. Uniformly increasing all weights does not fix this, because paths have different numbers of edges. A cleverer reweighting exists. Bellman–Ford handles negative weights, but it too requires no negative cycles.

Exercises

1. How do you find an MST minimizing the maximum edge weight in the tree?
2. What $k \in \mathbb{N}$ makes a k-ary heap most efficient for MST and Dijkstra's algorithms?

16.3 The transitive-closure problem

Definition

Let $G = (V, E)$ be a directed graph. Its transitive closure is (V, E^*), where

$$E^* = \{(i, j) : \text{there exists a path from } i \text{ to } j\}.$$

The goal is to compute this closure. A naive method runs DFS once from every vertex. One DFS costs $O(|V| + |E|)$, which can be $O(|V|^2)$, so the total can be $O(|V|^3)$.

Adjacency-matrix observation. Let $A = (a_{ij})$ be the adjacency matrix, with

$$a_{ik} = \begin{cases} 1, & (i,k) \in E, \\ 0, & (i,k) \notin E. \end{cases}$$

Then

$$(A^2)_{ij} = \sum_{k=1}^{n} a_{ik} a_{kj}$$

is exactly the number of length-2 walks from i to j. Walks may revisit vertices. More generally, $(A^r)_{ij}$ counts the r-step walks from i to j.

A direct computation of

$$I + A + \cdots + A^n$$

using a fast $O(n^{2.81})$ matrix multiplication for each power still costs $n \cdot O(n^{2.81})$.

Repeated squaring. Observe successively that

$$(I + A)_{ij} \neq 0 \iff i \rightsquigarrow j \text{ in 0 or 1 steps},$$

$$((I + A)^2)_{ij} \neq 0 \iff i \rightsquigarrow j \text{ in 0 through 2 steps},$$

and in general

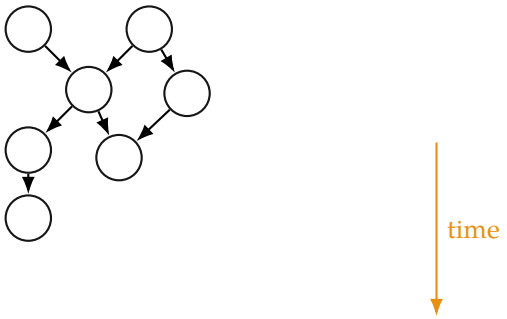
$$((I + A)^n)_{ij} \neq 0 \iff i \rightsquigarrow j \text{ in 0 through } n \text{ steps}.$$

Fast exponentiation computes $(I+A)^n$ using $O(\log_2 n)$ matrix multiplications. With the Strassen exponent, this is

$$O(n^{2.81} \log_2 n).$$

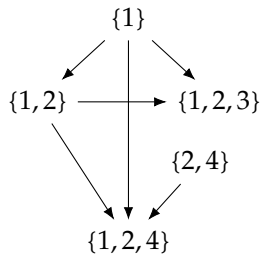
16.4 Directed acyclic graphs (DAGs)

A DAG can usually be drawn with all edges progressing in one direction—for example, downward as time advances:

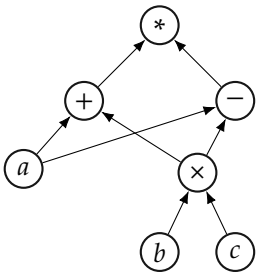


16.4.1 Examples of DAGs

Inclusion graph



Expression DAG



The expression DAG exposes the shared subexpression instead of duplicating it.

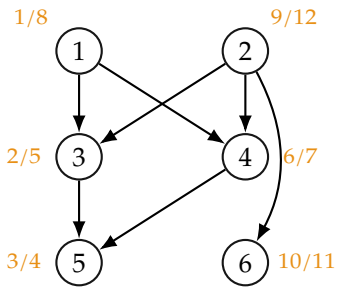
16.4.2 Topological sorting / consistent labeling

Input: a DAG. **Output:** an ordering in which every edge is aligned in one direction. The ordering need not be unique.

DFS method

```
run DFS and compute each finish time f[v]
list the vertices from high f[v] to low f[v]
```

In the example, the orange labels are discovery/finish times:



$\Longrightarrow \quad 2, 6, 1, 4, 3, 5$

Why it works. Consider an edge (u, v). If v is black when DFS examines the edge, then $f(v) < f(u)$. If v is white, the search from u discovers and finishes v before finishing u, again giving $f(v) < f(u)$.

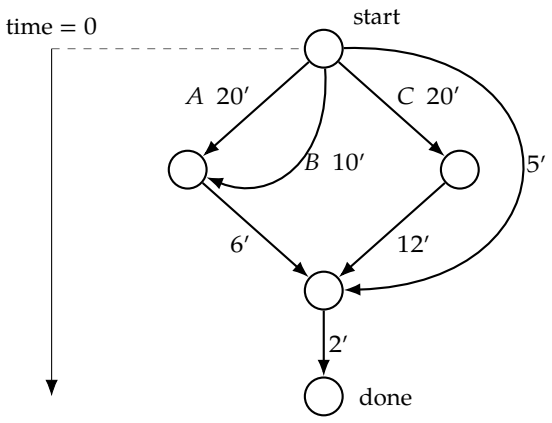
As DFS continues, the white vertex v is completely explored before u finishes. Hence (u, v) points in the correct direction in decreasing finish-time order.

Exercise

Why could v not be gray? Hint: cycles.

16.4.3 Activity monitors (PERT)

Tasks $A, B, C, \ldots$ are edges, and each edge carries the time needed for that task. In the activity network below, the parallel edges A and B and the long curved edge of length $5'$ encode the task dependencies:



Given a DAG $G = (V, E)$ and edge times $T[u, v]$, find the time from start to done. The key recurrence is

$$\text{Time}[v] = \max_{u:(u,v)\in E} (\text{Time}[u] + T[u, v]).$$

PERT / longest path in a DAG

Time[start] $\leftarrow 0$
`topologically sort the DAG`
`for every` v, `from high` $f[v]$ `to low` $f[v]$`:`
  Time$[v] \leftarrow \max_{u:(u,v)\in E}(\text{Time}[u] + T[u,v])$

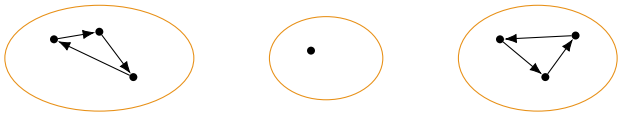
16.5 Strongly Connected Components

16.5.1 Decomposing every directed graph

Define

$$u \equiv v \quad \Longleftrightarrow \quad \text{there is a path } u \rightsquigarrow v \text{ and a path } v \rightsquigarrow u.$$

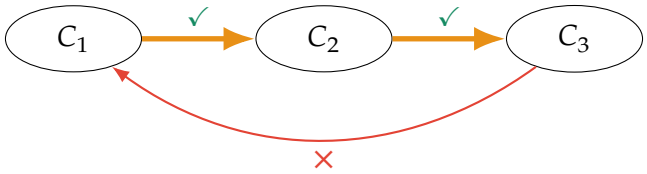
This is an equivalence relation, so G decomposes into equivalence classes. Each class contains vertices that are mutually reachable.



There can be no pair of opposite-direction connections between two distinct equivalence classes: paths in both directions would merge them into one class. Thus the equivalence classes themselves form a DAG.

Strongly connected components (SCCs)

The mutual-reachability equivalence classes are the strongly connected components of G. Contracting every SCC to one vertex produces the *condensation DAG*.



16.5.2 Kosaraju–Sharir's algorithm

1. Run DFS on $G = (V, E)$, computing all finish times $f[v]$.
2. Reverse every edge to obtain $G^R = (V, E^R)$.

Both steps together take $O(|V| + |E|)$.

Edge reversal gives these adjacency lists:

	G		G^R
1	$3 \to 2$	1	$2 \to 3$
2	$1 \to 4$	2	$1 \to 4$
3	$4 \to 1$	3	1
4	2	4	$2 \to 3$

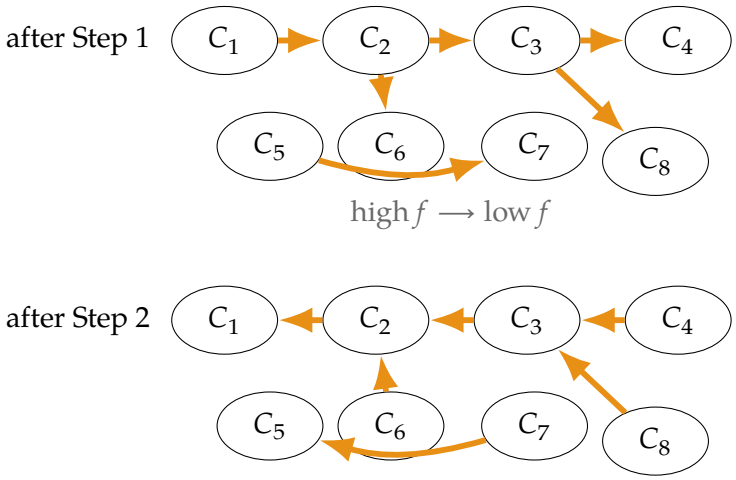
Aside: sorting adjacency lists in linear time

If the reversal is performed in vertex order $1 \to 4$ in this example, the adjacency lists of G^R come out sorted. Applying the same construction again, $G \to G^R \to G$, gives a linear $O(|V| + |E|)$-time method for sorting adjacency lists in this setting.

Reversing edges does not change which pairs of vertices are mutually reachable. Hence G^R has exactly the same SCC decomposition as G.

Step 3. Run DFS on G^R, choosing new DFS roots in order from highest to lowest $f[v]$, where the finish times are those computed in Step 1 on G.

This peels off one component at a time. A highest-finish component in the original condensation DAG becomes unable to reach another SCC after all condensation edges are reversed. Consequently, every DFS tree in Step 3 stays inside one SCC.



Two useful consequences are

$$\#\text{SCCs} = \#\text{ DFS restarts in Step 3},$$

and the components are peeled off in topologically sorted order. In Step 3, the DFS searches taken from high to low original finish time terminate without leaving their SCC, because the relevant condensation edges now point inward.

16.6 NIM: A Game on a DAG

Given groups of sticks, a move removes as many sticks as desired from one group at a time.

Unary bars describe the groups, while the compact notation lists their sizes:

$$| \quad || \quad ||| = 123, \qquad || \quad || = 22.$$

Good and Bad positions

A position is Good if, in one move, it can give the opponent a Bad position. A position is Bad if every legal move gives the opponent a Good position.

For the example beginning at $(1, 3, 5)$, part of the move DAG is classified in alternating layers:

Bad	1
Good	$11, 12, 13, 14, 15, 2, 3, 4, 5$
Bad	$111, 22$
Good	$112, 113, 114, 115, 122, 23, 24, 25$
Bad	$\vdots$

Edges point upward from a position to positions reachable by removing sticks.

Let $G = (V, E)$, where V is the set of all positions and E is the set of all legal moves.

Classification algorithm

```
topologically sort V; MakeNull(Q)
for vertices v from high f[v] to low f[v]:
  Enqueue(v, Q)
while |Q| > 0:
  u ← Dequeue(Q)
  Kind[u] ← Bad
  for every v such that (u,v) is a move:
    if Kind[v] = Bad:
      Kind[u] ← Good
```

Against a perfect player, a winning strategy must start from a Good position and then follow the move DAG by repeatedly choosing a move to a Bad position.

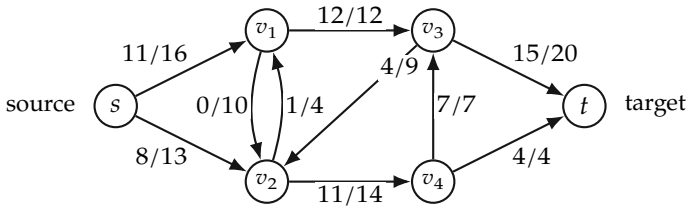
Chapter 17

Network Flow

17.1 Flow Networks

Motivating examples include water flow and electricity. The edges of a capacitated directed network have directions and capacity bounds.

Each pipe/edge has a direction and a capacity. In the worked network (the running example of CLRS), an edge label is written as flow/capacity; the flow shown has value $11 + 8 = 19$:



Write $c(u, v)$ for the capacity of the directed edge $u \to v$.

Flow

A flow is a function

$$f : V \times V \longrightarrow \mathbb{R},$$

where $f(u, v)$ is the flow from u to v across (u, v), satisfying:

1. **Antisymmetry:** $f(u, v) = -f(v, u)$.
2. **Capacity:** $f(u, v) \leq c(u, v)$.
3. **Kirchhoff's junction law:** for every $u \notin \{s, t\}$,

$$\sum_{v \text{ a neighbor of } u} f(u, v) = 0.$$

The value of the network flow is

$$|f| = \sum_{(s,u) \in E} f(s, u) = \sum_{(u,t) \in E} f(u, t).$$

The maximum-flow problem asks us to maximize this value.

Cut

A cut is a partition $S, T \subseteq V$ with

$$S \cap T = \emptyset, \qquad S \cup T = V, \qquad s \in S, \quad t \in T.$$

For $A, B \subseteq V$, the flow from A to B is

$$f(A,B) = \sum_{\substack{u \in A,\, v \in B \\ (u,v) \in E}} f(u,v).$$

Useful identities for set-to-set flow are

$$\begin{aligned} f(A,A) &= 0 && \text{because } (u,v) \text{ is paired with } (v,u), \\ f(u, V \setminus \{u\}) &= f(u,V) = 0 && \text{for } u \neq s,t, \text{ by Kirchhoff's law,} \\ f(A,B) &= -f(B,A), \\ A \cap B = \emptyset &\implies f(A \cup B, C) = f(A,C) + f(B,C). \end{aligned}$$

If S, T is a cut, then the network-flow value equals the flow across the cut:

$$f(s,V) = f(S,T).$$

Indeed, add the zero Kirchhoff sums for every $u \in S \setminus \{s\}$:

$$\begin{aligned} f(s,V) &= f(s,V) + \sum_{u \in S \setminus \{s\}} f(u,V) \\ &= f(S,V) \\ &= f(S,S) + f(S,T) \\ &= 0 + f(S,T). \end{aligned}$$

17.2 Residual Networks

The greedy step is to find an augmenting path by DFS. For a current flow f, the residual network has the same vertex set,

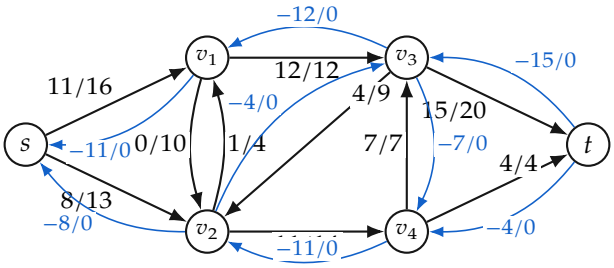
$$V^* = V,$$

and residual capacity

$$c_f(u,v) = c(u,v) - f(u,v).$$

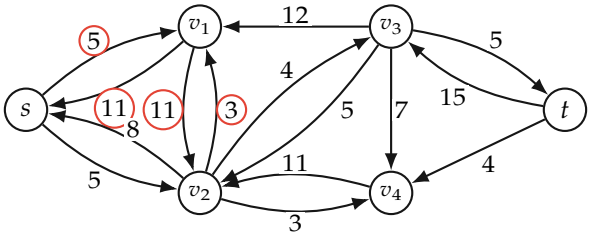
If (u,v) does not exist in the original network, take $c(u,v) = 0$.

In the left drawing, the blue arrows record the reverse direction of every edge: by antisymmetry it carries flow $-f(u,v)$ and has capacity 0, so its residual capacity is $0 - (-f(u,v)) = f(u,v)$. The right drawing is the resulting residual network. Red annotations emphasize that residual capacities are not flows.



flow network with the implicit reverse edges in blue

residual capacities ≠ flow! $5 = 16 - 11$, $11 = 0 - (-11)$



$11 = 10 - (-1)$, $3 = 4 - 1$

the residual network

Interpretation

Residual capacity measures how much more we can push through that pipe in that direction. In particular, a reverse residual edge records how much of an earlier choice can be cancelled.

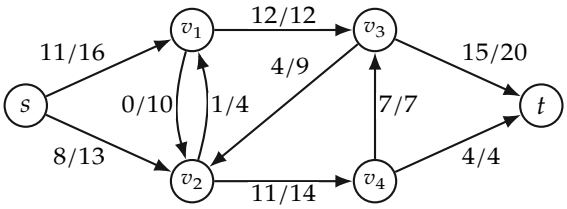
An *augmenting path* is a path in the residual network from s to t, and every edge on it must have positive residual capacity.

17.3 Augmenting Paths and Ford–Fulkerson

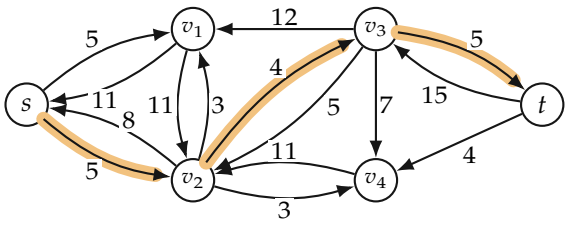
An s–t path in the residual network is augmenting when every edge on it has positive residual capacity. If such a path exists, the flow in the original network can be increased by

$$\Delta = \min_{e \in P} c_f(e).$$

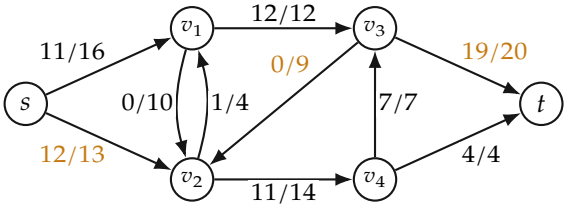
The colored augmentation below increases the flow from 19 to 23.



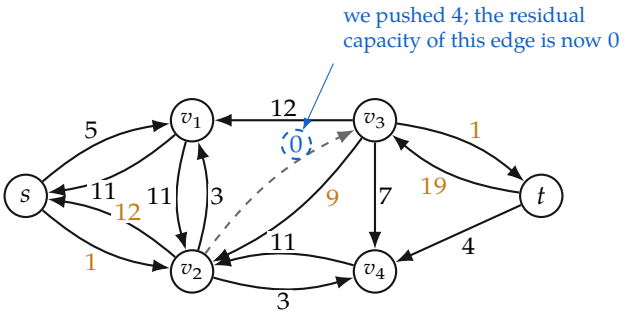
(a) current flow, value 19



(b) residual network; augmenting path, $\Delta = 4$



(c) new flow, value 11 + 12 = 23 (19 before)



(d) new residual network

In the residual network (b) the orange path

$$s \to v_2 \to v_3 \to t$$

has residual capacities 5, 4, 5, so $\Delta = 4$. Pushing 4 units along it raises $s \to v_2$ to 12/13, cancels the 4 units that were flowing from v_3 to v_2 (that edge becomes 0/9), and raises $v_3 \to t$ to 19/20; the new value is 11 + 12 = 23, as drawing (c) shows. Drawing (d) is the new residual network.

In the handwritten notes the edge from v_3 to v_2 was relabeled 9/9 after this augmentation. The correct label is 0/9: the new residual network, which shows residual capacity 9 on $v_3 \to v_2$ and 0 on $v_2 \to v_3$, confirms that no flow remains on that edge.

Ford–Fulkerson

```
let f be the trivial flow: f(u,v) = 0 for all u,v ∈ V
while there is an augmenting path P from s to t
     in the residual network with c_f > 0:
   find P by DFS in the residual network
   construct the path flow f* of value
       min_{e∈P} c_f(e)
   f ← f + f*
return f
```

Correction property

If f is a legal flow in the original network and f^* is a legal flow in its residual network, then $f + f^*$ is a legal flow in the original network.

Flow conservation follows from the definition of flow and from adding two conserving flows. For capacity, use $f^*(u,v) \le c_f(u,v)$ in the calculation below.

For every original edge (u, v),

$$\begin{aligned} f(u,v) + f^*(u,v) &\le f(u,v) + c_f(u,v) \\ &= f(u,v) + c(u,v) - f(u,v) \\ &= c(u,v). \end{aligned}$$

The remaining correction property follows from the way path flow is pushed into the residual network: initially $f^* = 0$, and each loop iteration adds precisely the legal residual path flow.

17.4 Cuts and the Max-Flow/Min-Cut Theorem

A cut (S, T) partitions V, with $s \in S$ and $t \in T$. Its capacity is

$$c(S,T) = \sum_{\substack{u \in S,\ v \in T \\ (u,v) \in E}} c(u,v).$$

Theorem

The following statements are equivalent:
1. the residual network has no augmenting s–t path;
2. the current flow is a maximum flow;
3. the flow value equals $c(S, T)$ for some cut (S, T).

Max-flow/min-cut

$$\max_f |f| = \min_{\substack{(S,T) \text{ cut} \\ s \in S,\ t \in T}} c(S,T).$$

In general, every feasible flow satisfies $|f| \le c(S, T)$ for every cut. In the worked example the highlighted minimum cut has capacity

$$12 + 7 + 4 = 23,$$

equal to the maximum flow.

For a flow with no augmenting path, let

$$S = \{v : v \text{ is reachable from } s \text{ in the residual network}\}, \qquad T = V \setminus S.$$

17.4.1 Proof of max-flow/min-cut

Ford–Fulkerson stops exactly when t is not reachable from s in the residual network. Thus $s \in S$, $t \in T$, $S \cap T = \emptyset$, and $S \cup T = V$: (S, T) is a cut.

For $u \in S$ and $v \in T$, residual reachability cannot cross (u, v). Therefore

$$c_f(u,v) = 0 = c(u,v) - f(u,v),$$

so every forward edge of the cut is saturated and

$$c(S,T) = f(S,T).$$

Combining the universal bound with this cut gives

$$|f| \le \min_{(S',T')} c(S',T') \le c(S,T) = f(S,T) = |f|.$$

All inequalities are equalities, proving

$$\max |f| = \min c(S,T).$$

17.5 Complexity

With integral capacities, the flow starts at 0 and increases by at least 1 each iteration, so the number of iterations is at most the maximum-flow value. Building/updating the residual network costs $O(|E|)$, and DFS costs $O(|V| + |E|) = O(|E|)$ in a connected network. Hence

$$T_{\text{Ford–Fulkerson}} = O(|E| \cdot \text{maximum-flow value}).$$

Edmonds–Karp uses BFS to choose an augmenting path with the fewest edges. The number of augmentations is $O(|V||E|)$, each costing $O(|E|)$, hence

$$T_{\text{Edmonds–Karp}} = O(|V||E|^2).$$

Final-exam format

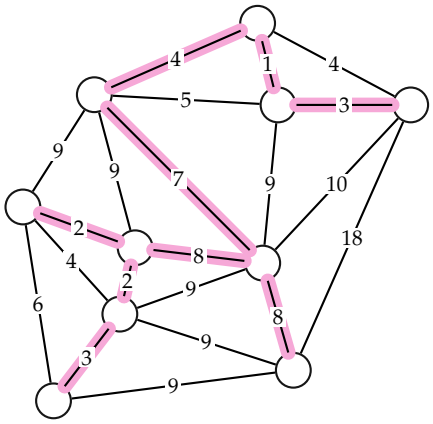
Eleven questions; questions 1 and 6 ask for full algorithms; three questions ask for high-level algorithm design; approximately 30% of the material is from before the midterm and 70% from after the midterm.

Appendix A

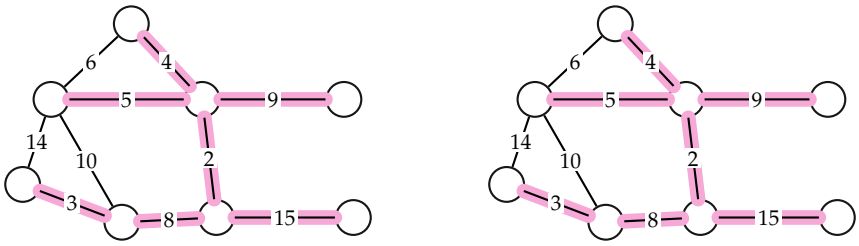
Worked Practice Examples

A.1 Minimum Spanning Trees: Prim–Dijkstra and Kruskal

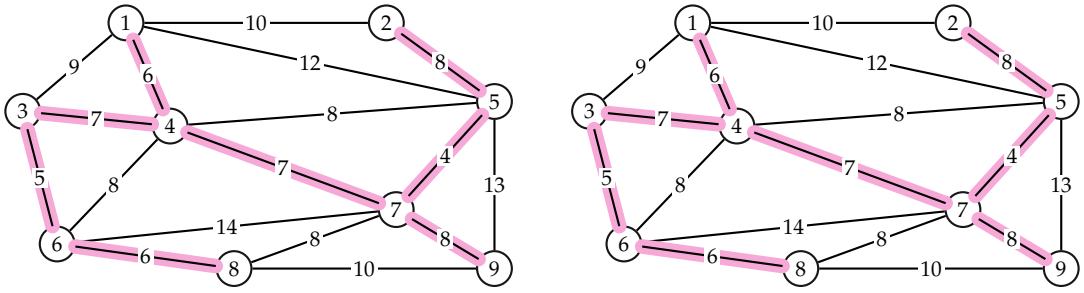
Each graph below was used to practice the two greedy algorithms by hand; the pink edges are the chosen tree. When the same graph is drawn twice, the two runs (one per algorithm) select the same edges, as they must whenever the minimum spanning tree is unique.



ten vertices; tree weight $1 + 2 + 2 + 3 + 3 + 4 + 7 + 8 + 8 = 38$



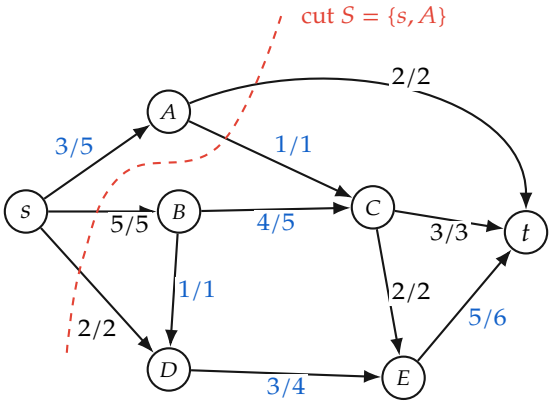
eight vertices, drawn twice (one run per algorithm); tree weight $2 + 3 + 4 + 5 + 8 + 9 + 15 = 46$



nine labeled vertices, drawn twice; tree weight $4 + 5 + 6 + 6 + 7 + 7 + 8 + 8 = 51$

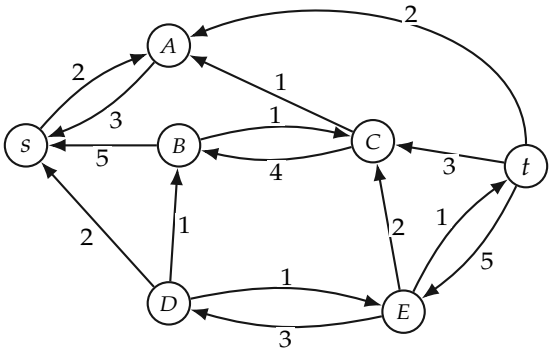
A.2 Maximum Flow, Residual Network, and Minimum Cut

Labels are flow/capacity; the blue labels mark the edges whose flow was changed by the last augmentation. The flow has value 10. The dashed cut $S = \{s, A\}$ has capacity $1 + 2 + 5 + 2 = 10$, so the flow is maximum and the cut is minimum.



flow $= 3 + 5 + 2 = 10$, cut capacity $= 1 + 2 + 5 + 2 = 10$

The residual network of this flow is drawn below. From s only A is reachable (through the residual edge $s \to A$ of capacity 2), so $S = \{s, A\}$ is exactly the set of vertices reachable from s, as in the proof of the max-flow/min-cut theorem, and there is no augmenting path.



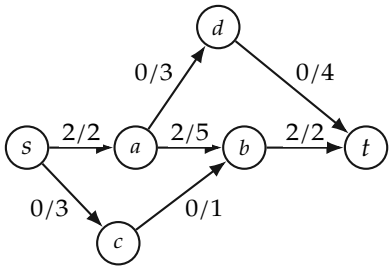
residual network: t is not reachable from s

A.3 Augmenting Along a Reverse Residual Edge

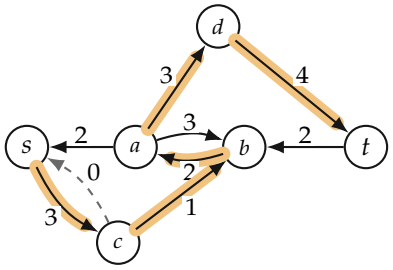
The second practice example shows why the reverse residual edges are essential. Start from the flow of value 2 in the first drawing. In its residual network the only s–t path, highlighted in orange, is

$$s \to c \to b \to a \to d \to t,$$

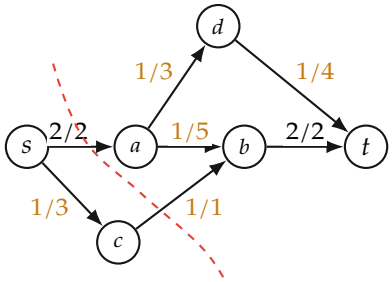
and it uses the reverse edge $b \to a$, which cancels one unit of the flow on (a, b). Pushing $\Delta = 1$ along it gives the flow of value 3 in the third drawing. In the new residual network t is no longer reachable from s; the dashed cut $S = \{s, c\}$ has capacity $2 + 1 = 3$, equal to the flow.



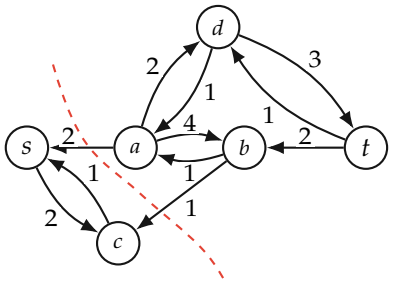
flow of value 2



residual network; e.g. $2 = 0 - (-2)$ on $a \to s$, $3 = 3 - 0$ and $0 = 0 - 0$ between s and c



new flow of value 3



new residual network: no s–t path; cut = 2 + 1 = 3

In the residual networks, $f(u, v) = -f(v, u)$ and $c(u, v) = 0$ whenever $(u, v) \notin E$, which is why every original edge contributes a reverse residual edge whose capacity equals the current flow on that edge.